

大家好！我们是牛面 Offer 团队！

牛面Offer，是你专属的【AI+后端】面试突击教练。无论校招、社招，还是转行求职，我们都能为你提供一站式求职备战：从刷题训练、简历优化，到面试模拟、技巧提升，全覆盖助力通关。

这篇PDF也为你整理了高频面试真题，搭配精炼答案，直接告诉你怎么答、怎么说，还提供了每道题的详细分析，帮你高效准备、从容应对，轻松拿 Offer！

更多优质题目尽在牛面Offer小程序，可以扫下方二维码学习2000+免费题库：

牛面 一站式学习与面试

2000+热门题库免费刷！题目全栈覆盖，助你拿offer! 

海量题库刷不停，
微信扫码立即开始！



趣味闯关

告别死记硬背，闯关练习陪你
轻松进步，每一关都在悄悄变强。

1000+精品关卡等你来战



技术题库

春招、秋招最热题型全覆盖
多名大厂老鸟精心把关筛选

2k+道命中率超高必刷题



简历押题

大厂标准提取核心问题
问题-思路-答案三位一体

个性化精准预测



模拟面试

专项突破，垂直领域靶向强化
简历深挖，六大维度全面拷问

真实场景完整演练

 牛面题库 让面试准备更高效 → niumianoffer.com 

Java大厂后端面试题-Top100

Java大厂后端面试题-Top100

MySQL：基础

1. Mysql存储引擎对比？

解析：

存储引擎：InnoDB，MyISAM，MEMORY等

参考口述回答：

在这里，我主要对比两种最常用的存储引擎：InnoDB和MyISAM，同时简要提及其他几种引擎的特点。

1. InnoDB存储引擎：

- **事务支持**：InnoDB是事务型数据库的首选引擎，它提供了提交、回滚和崩溃恢复能力来保护用户数据，确保数据的完整性。它遵循ACID原则，即原子性、一致性、隔离性和持久性。
- **行级锁定**：InnoDB支持行级锁定，这意味着在多个用户并发访问数据库时，它只会锁定被访问的行，而不是整个表。这大大提高了数据库的并发性能。
- **外键约束**：InnoDB还支持外键约束，可以确保数据的参照完整性。
- **聚簇索引**：InnoDB使用聚簇索引，即数据和主键索引存储在一起，这有助于提高某些查询的效率。

2. MyISAM存储引擎：

- **表级锁定**：MyISAM不支持事务和行级锁定，只支持表级锁。这意味着在读写操作时，会对整个表进行锁定，可能导致并发性能较差。
- **全文索引**：MyISAM支持全文索引，适合进行文本搜索。
- **高速读取**：MyISAM通常用于只读或大量读取的应用场景，如Web站点的归档数据或只读的数据仓库。其查询速度相对较快。
- **压缩存储**：MyISAM还支持压缩表，可以节省存储空间。

3. 其他存储引擎：

- **Memory**：将所有数据保存在RAM中，提供极快的访问速度，适用于需要快速查找引用的场景。但服务器关闭时数据会丢失。

总的来说，选择哪种存储引擎取决于具体的应用需求和性能要求。例如，对于需要高并发写入和数据完整性的应用，InnoDB是更好的选择；而对于只读或文本搜索为主的应用，MyISAM可能更合适。在实际应用中，我们需要根据具体情况进行权衡和选择。

推荐学习资料：

2. 一条SQL执行流程?

解析

连接器->查询缓存->解析SQL->执行SQL

参考口述回答

第一阶段：连接器

当用户尝试连接到MySQL数据库时，首先会经过连接器。连接器的主要任务是验证客户端的身份，例如检查用户名和密码是否正确。如果验证通过，连接器还会到权限表中查询该用户所拥有的权限。连接成功后，用户的会话信息会被缓存起来，这样在后续的连接请求中，如果连接参数相同，就可以重用之前的会话信息，避免重复进行身份验证。

第二阶段：缓存

在MySQL中，为了提高查询效率，有一个查询缓存的机制。当接收到一个查询请求时，MySQL会先检查查询缓存，看看是否之前已经执行过相同的查询。如果是，那么MySQL会直接从缓存中获取结果并返回给客户端，从而跳过后续的解析和执行阶段。但需要注意的是，查询缓存的维护也是有一定开销的，特别是在数据变更频繁的场景下，查询缓存可能会导致性能下降，因为每次数据变更都会导致相关的查询缓存失效。

第三阶段：解析SQL

如果查询缓存中没有找到匹配的结果，MySQL就需要对SQL语句进行解析。解析器会先对SQL语句进行词法和语法分析，将其转换成一个“解析树”。这个过程中会检查SQL语句的语法是否正确，如果不正确，就会返回错误信息给客户端。

第四阶段：预处理与优化

解析完SQL语句后，MySQL会对其进行预处理和优化。预处理主要是检查解析树中的表和列是否存在，数据类型是否正确等。优化器则负责根据解析树生成一个高效的执行计划。优化器会考虑多种可能的执行路径，并选择其中成本最低的一种。这个成本是基于对数据的统计信息来计算的，比如表的大小、索引的分布情况等。

第五阶段：执行SQL

根据优化器生成的执行计划，MySQL开始执行SQL语句。如果是查询操作，MySQL会按照执行计划中的步骤逐步检索数据，并最终返回给客户端。如果是更新操作（如INSERT、UPDATE或DELETE），MySQL会先对数据进行修改，并可能触发相关的约束和触发器。在这个过程中，MySQL还需要维护数据的完整性和一致性。

第六阶段：返回结果

最后，MySQL会将执行结果返回给客户端。对于查询操作，这通常是一个结果集，包含了满足条件的记录；对于更新操作，可能是一个表示操作成功或失败的状态码。

以上就是MySQL中一条SQL语句的执行过程。

推荐学习资料

《小林Coding》 | 图解MySQL：# 执行一条 select 语句，期间发生了什么？

MySQL：索引

3. 索引的分类？

解析

多个角度：数据结构，物理存储，特性，字段个数

参考口述回答

首先，从数据结构的角度来看，MySQL的索引主要分为以下几类：

1. B+tree索引：

- 这是最常用的索引类型，特别是在InnoDB和MyISAM存储引擎中。
- B+tree索引能够处理全键值、键值范围和前缀查询。
- 在InnoDB中，它使用的是B+tree结构，这是一种平衡的树结构，能够保持数据的有序性，从而加速查询性能。

2. Hash索引：

- 主要在MEMORY存储引擎中使用，但也可以在其他引擎中通过特定方式实现。
- Hash索引基于哈希表实现，适用于等值查询，但不适合范围查询和排序操作。
- 它的查找速度非常快，但缺点是数据不是按照索引值顺序存储的。

3. Full-text索引：

- 主要用于文本搜索，支持在MyISAM和InnoDB存储引擎上创建。
- 它允许你在文本字段上进行高效的全文搜索。

接下来，从物理存储的角度来看，索引可以分为：

- **聚簇索引**：在InnoDB中，表数据实际上是按照主键的顺序存储的，这种索引称为聚簇索引。因此，主键查询在InnoDB中是非常高效的。
- **二级索引（辅助索引）**：除了主键索引外，其他所有的索引都可以看作是二级索引。在InnoDB中，二级索引的叶子节点存储的是主键值，而不是实际的数据记录。

此外，从字段特性的角度，索引还可以分为：

- **主键索引**：基于表的主键字段创建的索引，具有唯一性。
- **普通索引**：允许在索引的列中插入重复值和空值。
- **唯一索引**：确保索引列中的数据是唯一的，类似于主键索引，但允许有空值。

最后，从字段个数的角度，索引可以分为：

- **单列索引**：仅包含表中的一个列。
- **联合索引（复合索引、组合索引）**：包含表中的多个列，可以提高多个列的查询效率。

需要注意的是，不同的存储引擎支持的索引类型和特性可能会有所不同。例如，MEMORY存储引擎默认使用Hash索引，而InnoDB和MyISAM则主要使用B+tree索引。在选择索引类型和存储引擎时，需要根据具体的应用需求和性能要求来进行权衡。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[# 索引的分类](#)

《小林Coding》 | 图解MySQL：[# 索引的分类](#)

4. B+树索引结构介绍？为什么使用B+树而不是B树？

解析

参考口述回答

1. B+树结构介绍

首先，它是一种多叉树结构，这意味着每个节点可以有多个孩子节点，不像二叉树那样每个节点只能有两个孩子。这样的设计使得树的高度降低，进而提高了检索效率，因为我们在查找数据时需要的比较次数减少了。

其次，B+树的内部节点，也就是非叶子节点，是不存储数据的，它们只存储键值和指向子节点的指针。而所有的数据都存储在叶子节点上。这样的设计让内部节点可以容纳更多的键值，进一步降低了树的高度。

再者，B+树的叶子节点之间是通过指针相连的，形成了一个有序链表。这种结构特点让范围查询变得非常简单和高效，因为我们可以直接通过叶子节点的指针进行顺序访问。

最后，B+树还具有良好的平衡性。在插入或删除数据时，B+树会通过分裂或合并节点来保持平衡，从而确保查询性能的稳定。

1. 为什么用B+树而不是B树？

一、树的高度

B+树相较于B树，其树的高度更低。这是因为B+树的非叶子节点不存储数据，只存储索引，因此可以容纳更多的子节点，使得整个树的结构更加扁平化。这种设计减少了查询时需要经过的层级数，最重要是减少了磁盘IO的次数，从而提高了查询效率。相比之下，B树的每个节点都存储数据，导致节点容纳的子节点数量有限，树的高度相对较高。

二、插入删除效率

在插入和删除操作方面，B+树也表现出更高的效率。由于B+树的非叶子节点不存储数据，因此在插入或删除数据时，只需要调整索引结构，而不需要频繁地移动大量数据。这大大简化了插入和删除操作的过程，并提高了效率。相比之下，B树在插入或删除数据时可能需要更复杂的操作来保持树的平衡。

三、范围查询效率

B+树在范围查询方面具有显著优势。由于B+树的叶子节点包含所有数据，并且叶子节点之间通过指针相连，形成了一个有序链表结构。这种结构使得范围查询变得非常简单和高效，因为我们可以直接通过叶子节点的指针顺序访问范围内的数据。而在B树中，范围查询可能需要更复杂的中序遍历操作。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[# MySQL索引为何使用B+树结构？](#)

5. 二级索引？查找过程？覆盖索引？索引下推？

解析

参考口述回答

1. 存储内容

在数据库中，二级索引通常存储在单独的数据结构中，如B+树。与聚簇索引不同，二级索引的叶子节点并不包含整行数据，而是存储了索引列的值和对应的主键值。这意味着，当我们通过二级索引查找数据时，我们首先找到的是与查询条件匹配的主键值，而不是直接找到完整的记录。

1. 查找过程

- 定位索引列的值：**在二级索引中，根据查询条件中的索引列值，数据库可以快速定位到匹配的索引记录。这个过程是通过在B+树中进行搜索实现的，时间复杂度相对较低。
- 获取主键值：**一旦找到匹配的索引记录，我们可以从中获取到对应的主键值。这个主键值是连接二级索引和聚簇索引的桥梁。
- 回表操作：**有了主键值后，我们需要回到聚簇索引中进行查找，这个过程被称为“回表”。在聚簇索引中，主键值直接与数据记录相关联，因此我们可以通过主键值直接找到完整的数据记录。

2. 覆盖索引

如果查询的列都包含在二级索引中，那么我们就可以直接使用二级索引进行查询，而无需进行回表操作。这种情况被称为“覆盖索引”。覆盖索引可以显著提高查询效率，因为它减少了磁盘I/O操作和数据查找的复杂性。

1. 索引下推

在某些情况下，数据库可以使用一种称为“索引下推”的优化技术来进一步提高查询效率。索引下推允许数据库在二级索引层面就进行部分过滤操作，从而减少需要回表的数据量。这种优化可以在某些复杂的查询条件下显著提高性能。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：# (五)MySQL索引应用篇：建立索引的正确姿势与使用索引的最佳指南！

6. 最左匹配原则？

解析

参考口述回答

一、什么是最左匹配原则

最左匹配原则是MySQL中联合索引查询时遵循的一个重要原则。它指的是在使用联合索引进行查询时，查询条件中必须包含索引的最左边（即第一个）字段，才能有效地利用索引进行查询优化。如果查询条件没有包含最左边的字段，那么索引将不会被使用，这可能会导致查询性能下降。

二、最左匹配原则的原理

最左匹配原则的原理基于B+树索引的结构特点。在MySQL中，索引是以B+树的形式存在的，联合索引也不例外。B+树索引是按照从左到右的顺序进行构建的，因此，在查询时也需要按照相同的顺序进行匹配。

当查询条件中包含了联合索引的最左字段时，数据库可以快速定位到符合条件的记录所在的位置，从而提高查询效率。如果查询条件没有包含最左字段，那么数据库就需要进行全表扫描，这会大大降低查询性能。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[#联合索引的最左前缀原则](#)

7. 索引失效场景？

解析

非最左匹配；模糊查询；计算或函数操作；数据类型转换；!=操作符；隐式类型转换等

参考口述回答

1. 联合索引非最左匹配：当我们使用联合索引时，必须遵循最左匹配原则。也就是说，查询条件中必须使用索引的最左边的一列，否则索引将不会起作用。例如，如果我们有一个基于列A和列B的联合索引，但查询条件只涉及列B，那么该索引将不会被使用。

2. 不恰当的模糊查询：在使用LIKE进行模糊查询时，如果查询模式以通配符开头（例如，LIKE '%xyz'），那么索引将不会起作用。因为数据库无法有效利用索引来加速这种模式的搜索。只有当模糊查询的模式是以确定字符开头时（例如，LIKE 'xyz%'），索引才能被有效利用。

3. 对索引列进行计算或函数操作：如果在查询中对索引列进行了计算或应用了函数，那么索引将失效。例如，如果我们对一个索引列进行了加法或乘法运算，或者使用了某种函数如UPPER()或LOWER()来处理索引列的值，那么数据库将无法使用索引来加速查询。

4. 数据类型转换：如果查询条件中的数据类型与索引列的数据类型不匹配，导致数据库需要进行类型转换才能进行比较，那么索引也可能失效。例如，如果索引列是字符串类型，但查询条件中使用了数字值进行比较，那么数据库可能需要进行类型转换才能执行查询，这会导致索引失效。

5. 使用OR操作符：当查询条件中包含多个使用OR连接的子条件时，如果子条件涉及的列不是全部被索引覆盖，那么索引可能会失效。因为数据库可能无法有效地利用单个索引来满足所有OR条件。

6. 使用IS NOT NULL或!=操作符：在某些数据库中，使用IS NOT NULL或!=操作符进行查询时，可能会导致索引失效。因为这些操作符需要扫描整个表来找到不匹配的行。

7. **隐式类型转换**：当在查询过程中发生隐式数据类型转换时（例如，将字符串与数字进行比较），这可能会导致索引失效。因为转换后的值可能无法与索引中的值直接匹配。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[#索引失效那些事儿](#)

《小林Coding》 | 图解MySQL：[#索引失效有哪些？](#)

MySQL：事务

8. 事务的ACID说一下？四种特性如何得到保证的？

解析

特性：原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）和持久性（Durability）
保证：redolog;undolog; MVCC；锁

参考口述回答

1. ACID介绍

事务的ACID特性是数据库管理系统在处理事务时所必须遵循的四个基本原则，它们分别是原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）和持久性（Durability）。下面我将逐一介绍这四个特性及其在实现层面是如何得到保证的。

首先是**原子性**。原子性意味着事务是一个不可分割的工作单位，事务中的操作要么全部完成，要么全部不完成。实现原子性的关键在于事务管理机制，它确保了一组相关的数据库操作要么全部执行成功，要么在发生错误时全部回滚到事务开始前的状态。这主要通过数据库管理系统的日志机制和回滚技术来实现。在事务执行过程中，系统会记录所有的操作日志，一旦发生错误，系统可以利用这些日志来回滚到事务开始前的状态，从而保证原子性。（**原子性是通过 undolog 来保证的**）

接下来是**一致性**。一致性要求事务执行前后，数据库的状态必须保持一致。这意味着，如果事务在执行过程中因为某些原因而中断，数据库系统必须能够恢复到事务开始前的状态，以保持数据的一致性。数据库管理系统通过一系列完整性约束、触发器和级联更新等操作来维护数据的一致性。同时，在事务结束时，系统会对数据库的状态进行检查，以确保所有的约束条件都得到满足，如果有不一致的情况，事务会被回滚。（）

再来说说**隔离性**。隔离性是指多个并发事务之间不会互相影响。为了实现隔离性，数据库管理系统采用了锁机制来控制对数据的并发访问。通过锁定正在被访问的数据，可以确保在事务完成之前，其他事务无法修改这些数据。此外，数据库还提供了不同的隔离级别，如读未提交、读已提交、可重复读和串行化等，以满足不同应用场景下的隔离需求。（**隔离性是通过 MVCC（多版本并发控制） 或锁机制来保证的**）

最后是**持久性**。持久性意味着一旦事务提交，其对数据库中数据的修改将是永久性的。为了实现持久性，数据库管理系统采用了写前日志（Write-Ahead Logging）技术。在事务提交前，系统会先将事务的修改记录在日志中，并确保这些日志已经持久化到磁盘上。这样，即使数据库系统发生故障或重启，已经提交的事务的修改

也不会丢失，因为系统可以根据日志来恢复数据到一致的状态。（持久性是通过 redo log（重做日志）来保证的）

2. ACID四种特性如何得到保证的的？

- 持久性是通过 redo log 来保证的；
- 原子性是通过 undo log 来保证的；
- 隔离性是通过 MVCC（多版本并发控制）或锁机制来保证的；
- 一致性则是通过持久性+原子性+隔离性来保证；

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[# 事务的ACID原则](#)

《小林Coding》 | 图解MySQL：[# 事务有哪些特性？](#)

9. 事务隔离级别？脏读，不可重复读，幻读？

解析

参考口述回答

1. 读未提交（Read Uncommitted）：

- 这是最低的隔离级别。在此级别下，一个事务可以读取另一个事务尚未提交的数据。
- 这种级别的问题在于，它可能导致“脏读”（Dirty Read）问题，即读取到未经确认的临时数据。
- 由于这个原因，这个隔离级别在实际应用中很少使用。

2. 读已提交（Read Committed）：

- 在这个级别，每个查询都会在事务执行时获取当前可见的数据。
- 它可以避免脏读，因为只能读取已提交的数据。
- 但这种级别可能导致“不可重复读”（Non-repeatable Read）问题，即同一个查询在多次执行时可能读取到不同的数据，因为其他事务可能在此期间修改了数据。

3. 可重复读（Repeatable Read）：

- 这是MySQL的默认事务隔离级别。
- 在这个级别下，事务在开始时“快照”当前状态，因此它看到的是一致的数据视图，在事务执行期间不会改变。
- 这可以避免脏读和不可重复读问题。
- 但这种级别可能导致“幻读”（Phantom Read）问题，即当事务读取某个范围内的记录时，另一个并发事务插入了一些新的记录，导致前一个事务在再次读取该范围的记录时，会看到一些“幻影”记录。
- InnoDB通过多版本并发控制（MVCC）和间隙锁来解决这个问题。

4. 串行化 (Serializable) :

- 这是最高的隔离级别。
- 在这个级别下，事务是完全串行执行的，避免了脏读、不可重复读和幻读问题。
- 它通过在事务执行期间对数据加锁来实现，这可以防止其他事务对数据进行修改和插入。
- 但这种严格的控制可能会导致大量的锁竞争和性能开销。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[#MySQL事务的隔离机制](#)

《小林Coding》 | 图解MySQL：[# 事务的隔离级别有哪些？](#)

10. 详细介绍多版本控制MVCC?

解析

基本概念；不同隔离级别下的实现；工作原理

参考口述回答

一、MVCC的基本概念

MVCC允许多个事务同时读取同一行数据，而不会彼此阻塞，每个事务看到的数据版本是该事务开始时的数据版本。这意味着，如果其他事务在此期间修改了数据，正在运行的事务仍然看到的是它开始时的数据状态，从而实现了非阻塞读操作。

二、不同隔离级别下的MVCC实现

1. **读未提交 (Read Uncommitted)** : 在这个隔离级别下，事务可以读取到其他未提交事务的修改。然而，这通常不被推荐，因为它可能导致脏读。在MVCC的上下文中，这个级别并不会创建数据的多个版本。
2. **读已提交 (Read Committed)** : 在这个级别，每次查询都会生成一个新的数据快照。当事务进行读取操作时，它只能看到在该事务开始之前已经被提交的数据。这意味着，如果数据在事务执行过程中被其他事务修改并提交，那么当前事务在下次查询时将看到这些更改。MVCC通过为每个事务提供其开始时的数据版本来实现这一点。
3. **可重复读 (Repeatable Read)** : 这是MySQL的默认隔离级别。在这个级别下，事务在开始时创建一个数据快照，并且在整个事务过程中都使用这个快照。即使其他事务在此期间修改了数据并提交，当前事务仍然看到的是它开始时的数据状态。MVCC通过保存数据的多个版本来实现这一点，每个版本都与一个特定的事务相关联。
4. **串行化 (Serializable)** : 这是最严格的隔离级别。在这个级别下，事务是完全串行执行的，避免了所有并发问题。然而，这也意味着性能可能受到影响，因为事务必须等待其他事务完成才能继续执行。在MVCC的上下文中，这个级别可能会退化为单版本控制，因为所有事务都是串行执行的。

三、MVCC的工作原理

MVCC的工作原理是基于数据版本的管理。每当数据被修改时，数据库不是直接更新原始数据，而是创建一个新的数据版本。每个版本都与一个事务ID相关联，这个ID表示哪个事务创建了该版本。当事务进行读取操作

时，数据库会根据该事务的隔离级别和当前活跃的事务列表来决定哪个版本的数据应该被返回。

此外，MVCC还依赖于一种称为“Read View”的机制来确定哪些版本的数据对当前事务是“可见”的。Read View本质上是一个事务ID的列表，它表示在当前事务开始时尚未提交的其他事务。通过这个列表，数据库可以判断哪些数据版本对当前事务是不可见的（因为它们是由尚未提交的事务创建的）。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[# \(九\)MySQL之MVCC机制：为什么你改了的数据我还看不见](#)

《小林Coding》 | 图解MySQL：[#Read View 在 MVCC 里如何工作的？](#)

MySQL：优化

11. 慢SQL排查？优化？

解析

排查：监控；慢查询日志；执行计划 优化：优化索引；优化SQL；优化数据库参数；分库分表；缓存

参考口述回答

一、慢SQL排查

- 1. 监控数据库性能：**首先，我会密切关注数据库的性能指标，如CPU利用率、内存利用率和磁盘I/O等。比如，如果CPU或内存使用率过高，可能就意味着有慢SQL存在。这些指标能帮助我初步定位可能存在问题的地方。
- 2. 查看慢查询日志：**数据库通常会记录执行时间较长的查询，即慢查询日志。通过查看这些日志，我可以找到那些执行时间较长的SQL语句，这是定位慢SQL的直接方法。
- 3. 使用执行计划：**对于疑似慢SQL的语句，我会使用数据库的执行计划工具进行分析。执行计划能展示SQL语句的执行步骤和涉及的表、索引等信息，从而帮助我找到性能瓶颈。

二、慢SQL优化

- 1. 检查并优化索引：**索引是提高查询性能的关键。我会检查现有的索引是否合理，是否能够支持SQL语句的快速执行。如果发现索引缺失或不合理，我会考虑添加、修改或删除索引。
- 2. 改写SQL语句：**有时候，慢SQL的产生是因为SQL语句本身存在问题，如查询条件不合理或表连接方式不正确。在这种情况下，我会尝试改写SQL语句，比如优化查询条件，合理使用索引，或者重构查询语句以提高性能。
- 3. 调整数据库参数：**数据库的性能也与其参数的设置有关。我会根据实际情况调整数据库的参数，如MySQL的innodb_buffer_pool_size和max_connections等，以提高性能。
- 4. 考虑分库分表：**如果数据库中的数据量过大，也可能导致SQL执行时间较长。在这种情况下，我会考虑对数据库进行分库分表，以分散数据存储和查询的压力。
- 5. 使用缓存：**对于一些热点数据，我会考虑使用缓存来提高访问速度。缓存可以将数据存储在内​​存中，减少数据库的访问次数，从而提高系统的性能。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：#(十八)MySQL排查篇：该如何定位并解决线上突发的Bug与疑难杂症？

《掘金专栏》 | 全解MySQL数据库：#(十七)SQL优化篇：如何成为一位写优质SQL语句的绝顶高手！

12. 介绍下分库分表？

解析

参考口述回答

1. 为什么要要进行分库分表？

一、解决数据库连接资源不足问题

在高并发的场景下，大量请求同时访问数据库，可能会导致数据库连接资源紧张。通过分库，可以将连接分散到多个数据库上，从而有效地减轻单个数据库的连接压力。

二、缓解磁盘IO的性能瓶颈

当单库数据量过大时，磁盘IO可能成为性能瓶颈。分库可以将数据分散到不同的物理存储上，降低单个数据库的磁盘IO负载，提高整体性能。

三、解决单表数据量过大问题

随着数据的持续增长，单表的数据量可能会达到数百万、数千万甚至更多。这会导致查询效率下降，尤其是当查询条件未命中索引时，全表扫描的耗时将显著增加。分表可以将大表拆分成多个小表，减少单个表的数据量，从而提高查询效率。

四、提高并发性能

分库分表能够将数据和请求分散到多个数据库和表上，从而提高系统的并发处理能力。每个数据库或表只处理部分数据，使得并发请求可以更加均匀地分布，减少了资源争用和锁冲突的可能性。

五、提升系统可扩展性

通过分库分表，可以更容易地扩展系统的数据处理能力。当需要增加更多的数据存储空间或处理能力时，只需简单地添加更多的数据库或表即可。这种水平扩展的方式比垂直扩展（如升级硬件）更加灵活和经济高效。

六、增强系统的容错能力

分库分表还可以提高系统的容错能力。当某个数据库或表出现故障时，其他数据库或表仍然可以继续运行，从而保证了系统的整体稳定性。此外，通过数据冗余和备份策略，可以进一步降低数据丢失的风险。

2. 分库分表的方案？

1. 垂直分库分表

- 垂直分库：根据业务的耦合性，将一个大的数据库拆分成多个小的数据库，每个数据库服务于不同的业务。这有助于将不同业务的数据进行隔离，提高系统的模块化和可维护性。

- **垂直分表**：将一个大表中的某些列拆分到另一个表中。这通常用于将不经常使用的数据或大字段（如文本、图片等）拆分出去，以提高查询效率和减少IO压力。

2. 水平分库分表

- **水平分库**：将数据按照某个字段（如用户ID、订单ID等）的取值范围分配到不同的数据库中。这种方法可以均匀地分散数据压力，提高系统的可扩展性。
- **水平分表**：将数据按照某个字段的取值范围分配到同一个数据库的不同表中。这通常用于解决单表数据量过大的问题，通过将数据分散到多个表中，可以提高查询性能和插入/更新的效率。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[# \(二十一\)MySQL之高并发大流量情况下海量数据分库分表的正确姿势](#)

MySQL：锁

13. Mysql中有哪些锁？

解析

全局锁 表级锁：表锁，元数据锁，意向锁，AUTO-INC锁

行级锁：记录锁，间隙锁，临键锁；

乐观锁与悲观锁

参考口述回答

一、全局锁

全局锁是对整个数据库实例加锁，通常在执行某些特定的全局操作时需要使用，如备份、数据库迁移等。在MySQL中，可以通过执行`FLUSH TABLES WITH READ LOCK`语句来获取全局锁。这种锁会阻塞其他会话对数据库的写操作，直到全局锁被释放。需要注意的是，获取全局锁会对数据库的正常运行产生影响，因此应谨慎使用，并在操作完成后及时释放。

二、表级锁

表级锁是锁定整个表，从而阻止其他用户并发访问。在MySQL中，表级锁主要包括以下几种：

1. **表锁**：最简单的表级锁，直接对整个表进行加锁。这种锁的优点是开销小、加锁快、无死锁，但缺点是锁定粒度大，发生锁冲突的概率最高，并发度最低。
2. **元数据锁（MDL）**：当对一个表结构进行修改时（例如`ALTER TABLE`），MySQL会自动为该表加上元数据锁，以防止其他事务在该表上进行DDL或DML操作。这种锁主要用于保证表结构的完整性。
3. **意向锁**：在InnoDB存储引擎中，当事务准备在某个数据行上加共享锁或排他锁之前，会先在表上获取相应的意向锁。意向锁是兼容的，主要用于在行锁和表锁之间建立一种层级关系。
4. **AUTO-INC锁**：当插入操作中涉及到自增字段时，MySQL会使用AUTO-INC锁来确保自增值的唯一性。这种锁在插入完成后会立即释放。

三、行级锁

行级锁是锁定表中的某一行或某些行，从而允许其他事务并发访问表中的其他行。在MySQL中，行级锁主要包括以下几种：

1. 记录锁（Record Locks）：直接锁定索引记录，常在更新操作时使用。当某个事务正在对一条记录进行修改时，其他事务无法对该记录进行修改或删除操作。
2. 间隙锁（Gap Locks）：锁定一个范围，但不包括记录本身。这种锁主要用于防止幻读（Phantom Reads），即在一个事务读取某个范围内的记录时，另一个事务插入新的记录到这个范围内。
3. 临键锁（Next-Key Locks）：是记录锁和间隙锁的结合，锁定一个范围并包括记录本身。这种锁可以确保范围内的记录和间隙都被锁定，从而提供更严格的并发控制。

1. 数据库中的乐观锁与悲观锁

1. 乐观锁（Optimistic Lock）

乐观锁的核心思想是认为在大多数情况下数据并不会发生冲突，因此在数据更新时不主动加锁，而是在提交数据更新时检查是否存在冲突。它常用于在高并发环境下减少锁的开销。

实现方式：

- **版本号机制**：最常见的乐观锁实现方式是在数据表中增加一个版本号字段（`version`），每次更新数据时同时更新版本号。

例子：

假设我们有一张用户表`user`，包括字段`id`、`name`和`version`：

在更新数据时，只有`version`字段与之前读取的一致，更新操作才会成功，否则说明数据在此期间被其他事务修改过，更新失败，需要重新读取和处理。

适用场景：

- 适合于大量读操作而较少写操作的场景，这样可以提升系统的并发性能。
- 高并发环境下避免频繁加锁带来的性能开销。

2. 悲观锁（Pessimistic Lock）

悲观锁的核心思想是认为对数据的冲突会频繁发生，因此在对数据进行操作之前主动加锁，避免其他事务访问该数据。悲观锁在操作过程中会保持锁定状态，直到事务提交或回滚。

实现方式：

- **数据库中的行级锁**：常见的实现方法是使用SQL语句加锁，如`SELECT ... FOR UPDATE`。

例子：

假设我们有一张用户表`user`，包括字段`id`和`name`：

在执行`SELECT ... FOR UPDATE`语句时，会对返回的行加锁，其他事务在该行被锁定期间无法更新数据。

适用场景：

- 适合于写操作频繁且数据竞争激烈的场景，可以有效防止并发写操作的冲突。
- 需要确保数据的强一致性，不允许丢失任何更新时。

总结

乐观锁：

- **原理：**假设不会发生冲突，通过版本号等机制在提交时检查冲突。
- **优点：**无锁方式，不会造成死锁，高并发下性能较好。
- **缺点：**需要重新读写数据处理冲突，适用于读多写少的场景。

悲观锁：

- **原理：**假设会发生冲突，通过锁机制在数据操作前锁定资源。
- **优点：**可以确保数据的强一致性，避免并发冲突。
- **缺点：**锁的开销较大，容易造成锁等待或死锁，适用于写多读少的场景。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[# \(八\)MySQL锁机制](#)

《小林Coding》 | 图解MySQL：[# MySQL 有哪些锁？](#)

《博客园》：[数据库中的乐观锁与悲观锁](#)

14. Mysql中的死锁排查？

解析 参考口述回答

一、死锁的排查

1. **查看错误日志：**MySQL的错误日志中可能会记录有关死锁的信息。通过查看这些日志，可以初步判断是否存在死锁以及死锁发生的时间和原因。
2. **使用SHOW ENGINE INNODB STATUS命令：**这个命令可以提供InnoDB存储引擎的详细信息，包括最近的死锁信息。在返回的结果中，可以查找"LATEST DETECTED DEADLOCK"部分，这里会详细描述死锁的情况，包括涉及的事务、锁定的资源等。
3. **性能监控工具：**使用如Percona Monitoring and Management (PMM)、InnoDB Plugin等工具可以帮助监控和分析数据库性能，并在发生死锁时提供警报。
4. **慢查询日志：**如果死锁是由于某些查询执行时间过长导致的，那么慢查询日志可能会记录这些查询。通过分析慢查询日志，可以找到可能导致死锁的长时间运行的查询。

二、死锁的解决

1. **调整事务隔离级别：**适当降低事务的隔离级别可以减少锁的竞争，但也可能带来其他问题，如不可重复读或幻读。需要根据具体业务需求权衡利弊。
2. **优化查询语句：**对涉及多表操作、复杂连接或子查询的语句进行优化，以减少锁的持有时间和范围。
3. **控制事务大小：**尽量将大事务拆分成多个小事务，以减少锁的持有时间。同时，避免在事务中执行不必要的操作。

4. **设置合理的锁超时时间**：通过设置合理的锁超时时间（如`innodb_lock_wait_timeout`参数），可以在一定程度上避免长时间的锁等待和死锁。
5. **手动介入**：如果以上方法都无法解决死锁问题，或者需要立即解决已经发生的死锁，可以考虑手动终止其中一个事务来打破死锁状态。这通常需要谨慎操作，并确保不会影响到业务的正常运行。

推荐学习资料

《掘金专栏》 | 全解MySQL数据库：[# \(十\)全解MySQL之死锁问题分析](#)

《掘金专栏》 | 全解MySQL数据库：[# \(十八\)MySQL排查篇：该如何定位并解决线上突发的Bug与疑难杂症？](#)

MySQL：日志

15. 介绍下binlog | redolog | undolog日志？

解析

参考口述回答

首先是**binlog**，也就是二进制日志。

- 作用：binlog记录了数据库所有的DDL（数据定义语言）和DML（数据操作语言）语句事件，这些语句会导致数据的改变。它主要用于数据复制和数据恢复。
- 格式：MySQL支持三种格式的binlog，分别是Statement-Based Replication(SBR)，记录SQL语句本身；Row-Based Replication(RBR)，记录行的改变；以及Mixed-Based Replication(MBR)，根据情况选择SBR或RBR。
- 工作原理：当一个事务被提交时，MySQL会将该事务的所有修改操作作为一个事件写入到binlog中。并且，MySQL会定期将内存中的binlog缓存刷新到磁盘上的binlog文件中。

接下来是**redo log**，也就是重做日志。

- 作用：redo log是InnoDB存储引擎特有的，用于保证事务的持久性和崩溃恢复。即使数据库崩溃，也可以通过redo log的回放来恢复已提交的事务。
- 结构：redo log是一个循环的、固定大小的日志文件，通常由多个物理文件组成。
- 写入过程：当事务执行修改操作时，MySQL首先将修改操作记录到内存中的redo log缓冲区，然后定期将这些日志记录刷写到磁盘上的redo log文件中。

最后是**undo log**，即回滚日志。

- 作用：undo log主要用于事务的回滚和并发控制。当事务需要回滚时，可以使用undo log中保存的旧值将数据恢复到事务开始之前的状态。此外，undo log还用于多版本并发控制（MVCC），以提供事务的隔离性和并发访问的一致性。
- 工作原理：对于每个事务来说，其都有一个对应的undo log。当我们开启事务并对表进行操作后，这些操作都会被记录到undo log中。如果事务执行错误或需要回滚，就可以根据undo log中的信息将数据恢复到原来的状态。

推荐学习资料

《小林Coding》 | 图解MySQL: # MySQL 日志: undo log、redo log、binlog 有什么用?

JAVA: 基础

16. 讲一下深拷贝浅拷贝的区别?

解析

复制的深度; 关联对象的处理;

参考口述回答

首先, 我们来说一下浅拷贝。浅拷贝是指仅仅复制对象的引用, 而不是对象本身。换句话说, 新旧对象还是共享同一块内存。在浅拷贝中, 如果原对象内部的子对象发生变化, 这种变化也会反映到拷贝对象中, 因为两者引用的是同一个子对象。实现浅拷贝, 我们需要实现Cloneable接口并重写Object类中的clone()方法, 如果没有重写clone()方法, 则会直接调用Object的clone()方法, 而Object的clone()方法是一个受保护的方法, 不能直接调用。

接下来, 我们谈谈深拷贝。与浅拷贝不同, 深拷贝会将对象完整地拷贝出来, 开辟新的内存来存放新对象, 新对象与原对象完全独立, 不共享内存, 修改新对象不会影响到原对象, 反之亦然。在深拷贝中, 对于对象中的每一个字段, 如果字段是值类型, 那么对该字段执行逐位复制; 如果字段是引用类型, 则递归地复制该字段所引用的对象。要注意的是, 实现深拷贝可能需要考虑的问题更多, 比如需要避免循环引用的问题, 否则可能会导致无限递归。

总结一下, 深拷贝和浅拷贝的主要区别在于复制的深度和对关联对象的处理方式。浅拷贝只复制第一层属性, 对象的属性是基本数据类型时, 直接进行值传递; 如果是引用数据类型, 则复制的是内存中的地址, 而不是真正的对象, 新旧对象还是共享同一块内存。而深拷贝则是对对象的所有层次进行复制, 包括对象内部的引用对象也会被完整地复制出来, 新旧对象完全独立, 不共享内存。

推荐学习资料

《JavaGuide》 | Java基础面试题: ##深拷贝和浅拷贝的区别了解吗?

17. 讲一下Java中的异常?

解析

Throwable; Error; Exception; try-catch; finally; throw; throws

参考口述回答

首先, 我们来看看Java异常的分类。在Java中, 所有的异常类都继承自Throwable类。Throwable类有两个主要的子类: Error和Exception。

1. Error:

- `Error`类表示严重问题，通常是JVM无法或不应该尝试处理的问题，如`OutOfMemoryError`。这些错误通常是Java运行时系统的内部错误或资源耗尽的情况，对于这类错误，应用程序一般无法处理或恢复。

2. Exception:

- `Exception`类是程序需要处理的异常情况。它又分为两大类：检查型异常（Checked Exceptions）和非检查型异常（Unchecked Exceptions）。
 - 检查型异常：这类异常在编译时必须被处理，要么通过`try-catch`块捕获，要么在方法签名中声明抛出。例如，`IOException`、`FileNotFoundException`、`ClassNotFoundException`等。
 - 非检查型异常：这类异常是运行时异常，继承自`RuntimeException`类。它们通常是由程序错误导致的，如空指针访问、数组越界等。常见的运行时异常有`NullPointerException`、`ArrayIndexOutOfBoundsException`、`ArithmeticException`等。运行时异常不需要在方法签名中声明，也应该尽量避免。

Java异常的处理方式:

- **try-catch块**: 使用`try`块包含可能引发异常的代码，并使用`catch`块捕获并处理这些异常。可以在`catch`块中执行特定的错误处理逻辑，如记录日志、回滚事务或执行清理操作。
- **finally块**: `finally`块包含的代码无论是否发生异常都会被执行，通常用于执行清理操作，如关闭文件流或数据库连接。
- **throw关键字**: Java允许程序员显式地抛出一个异常，使用`throw`关键字后跟一个异常对象。这通常用于在特定条件下触发异常处理流程。
- **throws关键字**: 在方法签名中使用`throws`关键字可以声明该方法可能抛出的异常类型，从而告知调用者需要处理这些异常。

处理异常时还需要注意以下几点:

1. 尽量避免使用空的`catch`块，因为它会吞噬异常，使得问题难以追踪和调试。
2. 在处理异常时，尽量提供有用的错误信息和堆栈跟踪，以便更好地定位问题。
3. 合理规划异常处理策略，避免过多的嵌套和复杂的逻辑结构。

推荐学习资料

《JavaGuide》| Java基础面试题: [##Java 异常](#)

18. String不可变是如何实现的?

解析

私有字段; 没有提供修改方法; 操作返回新对象; `final`修饰类避免被继承

参考口述回答

Java中的String类被设计为不可变的，这意味着一旦一个String对象被创建，它的内容就不能被改变。这种设计提供了很多好处，比如安全性、可预测性和性能优化。

String的不可变性主要是通过以下几个方面来实现的：

1. **私有字段**：String类内部使用私有字符数组来存储字符串的内容。由于这个字段是私有的，外部代码无法直接访问或修改它，从而保护了字符串内容的安全性。
2. **没有提供修改方法**：String类没有提供任何可以修改其内部状态的方法。例如，没有提供方法来改变字符串中的某个字符或添加/删除字符。
3. **返回新的对象**：当对String对象进行操作（如连接、替换等）时，String类会创建一个新的String对象来保存结果，而不是修改原始对象。这样，原始String对象的状态始终保持不变。
4. **final修饰类**：为了防止子类可能通过继承来破坏String的不可变性，Java中的String类被声明为final，这意味着它不能被继承。这样，就没有子类可以重写String类的方法或添加可能破坏不可变性的新方法。

推荐学习资料

《JavaGuide》| Java基础面试题： [##String 为什么是不可变的？](#)

19. 字符串拼接：加号，StringBuffer,StringBuilder对比？

解析

性能；线程安全；使用场景

参考口述回答

一、加号 (+) 拼接

1. 性能：当使用加号进行字符串拼接时，每次拼接都会创建一个新的String对象，这会导致大量的中间对象创建和内存分配。如果拼接操作频繁或字符串较大，这种方式可能会带来较大的性能开销。
2. 线程安全：加号拼接本身是线程安全的，因为它不涉及共享状态或并发操作。
3. 使用场景：加号拼接适用于简单的、少量的字符串拼接操作，或者在不需要考虑性能的场合。

二、StringBuffer拼接

1. 性能：StringBuffer在进行字符串拼接时，不会像加号那样每次都创建新的对象。它内部维护了一个可变的字符序列，通过append()方法可以将新的字符串添加到已有字符串的末尾，从而避免了频繁的内存分配。因此，在处理大量或频繁的字符串拼接时，StringBuffer的性能通常优于加号拼接。
2. 线程安全：StringBuffer是线程安全的。它内部使用了同步机制来确保在多线程环境下数据的一致性。这意味着在多线程应用中，你可以安全地使用StringBuffer进行字符串拼接，而不用担心数据混乱或不一致的问题。
3. 使用场景：当你需要在多线程环境下进行字符串拼接，或者需要频繁地进行大量的字符串拼接操作时，StringBuffer是一个很好的选择。

三、StringBuilder拼接

1. 性能：StringBuilder与StringBuffer类似，都维护了一个可变的字符序列来进行字符串拼接。然而，与StringBuffer不同的是，StringBuilder没有提供同步机制，因此它在单线程环境下的性能更高。实际上，在处理大量或频繁的字符串拼接时，StringBuilder的性能通常是最优的。
2. 线程安全：由于StringBuilder没有提供同步机制，所以它不是线程安全的。在多线程环境下使用StringBuilder进行字符串拼接可能会导致数据混乱或不一致的问题。
3. 使用场景：当你只需要在单线程环境下进行字符串拼接，并且追求最高的性能时，可以选择使用StringBuilder。特别是在处理大量的、频繁的字符串拼接操作时，如日志记录、文本处理等场景，StringBuilder是最佳的选择。

推荐学习资料

《JavaGuide》| Java基础面试题：String、StringBuffer、StringBuilder 的区别？

20. 基础数据类型及其大小？

解析

- 1字节：byte
- 2字节：short;char
- 4字节：int float
- 8字节：long;double
- 不定：boolean

参考口述回答

java中有8种基础数据类型，它们分别是：

1. **byte**：占用1字节，也就是8位，它的取值范围是-128到127。
2. **short**：占用2字节，即16位，可以表示的数值范围是-32768到32767。
3. **int**：占用4字节，32位，能表示的数值范围是从负的2的31次方到正的2的31次方减1。
4. **long**：占用8字节，64位，数据范围为负的2的63次方到正的2的63次方减1。
5. **float**：占用4字节，32位，可以表示的数据范围在3.4e-45到1.4e38之间。
6. **double**：占用8字节，64位，数据范围在4.9e-324到1.8e308之间。
7. **char**：占用2字节，16位，用于存储Unicode码，可以表示一个字符。
8. **boolean**：它只有两个取值，true和false，其大小没有明确的规定。

推荐学习资料

《JavaGuide》| Java基础面试题：String、StringBuffer、StringBuilder 的区别？

JAVA：集合框架

21. 集合里面的List、Set、Map差异你是怎么理解的？

解析：

数据结构和使用场景

参考口述回答：

首先，从数据结构的角度来看：

1. List（列表）：

- **有序性**：List中的元素按照它们被添加的顺序进行存储，这意味着元素的顺序与添加顺序相同，因此可以通过索引访问每个元素。
- **可重复性**：List允许存储相同的元素多次，即可以在List中添加重复的元素，且它们可以保持各自的位置和顺序。
- **常用实现类**：如ArrayList、LinkedList等，其中ArrayList基于动态数组实现，适用于查找和随机访问操作。

2. Set（集合）：

- **无序性**：Set中的元素是无序的，不能通过索引或位置来访问元素。
- **元素唯一性**：Set集合中的元素是唯一的，不会存在重复的元素。当试图添加重复元素时，新元素不会被添加进集合中。
- **常用实现类**：如HashSet、LinkedHashSet和TreeSet，其中HashSet基于哈希表实现，提供了较快的插入、查询和删除操作速度。

3. Map（映射）：

- **键值对存储**：Map集合是双列集合，每个元素由键和值组成（键值对）。键是唯一的，而值可以重复。
- **无序性**：Map的键是无序的，不可以通过索引访问。
- **常用实现类**：如HashMap、LinkedHashMap和TreeMap。其中，HashMap基于哈希表实现，提供了快速的键值对查找；LinkedHashMap保持了插入顺序；TreeMap则基于红黑树实现，键通过自然排序或自定义比较器进行排序。

其次，从使用场景的角度来看：

- **List**：适用于需要按照特定顺序存储和访问元素的场景，如日志记录、任务列表等。
- **Set**：适用于需要快速判断某个元素是否存在于集合中的场景，如用户唯一性校验、过滤重复数据等。
- **Map**：适用于需要根据键快速查找对应值的场景，如缓存系统、用户配置管理等。

综上所述，List、Set和Map在数据结构和使用场景上存在显著差异。List强调有序性和元素可重复性；Set强调元素唯一性和无序性；而Map则侧重于键值对的存储和快速查找。

推荐学习资料：

《JavaGuide》| 集合面试题：##Java 集合概览

22. HashMap底层如何实现的？

解析

数据结构；hash计算；hash冲突；扩容机制；线程安全；

参考口述回答

一、数据结构：

- HashMap内部使用了一个数组加链表和红黑树的组合。数组用于存储键值对的桶（bucket），而链表和红黑树则用于解决哈希冲突。
- 当发生哈希冲突时，HashMap会在数组的同一个桶中形成链表。为了提升性能，在Java 8中，当链表长度超过一定阈值（默认为8）时，链表会转化为红黑树。

二、哈希计算与桶定位：

- HashMap使用哈希函数来计算键的哈希值，并根据这个哈希值确定键值对应该存放在数组的哪个桶中。
- 这个哈希函数被设计为尽量减少冲突，并确保键值对能够均匀分布到各个桶中。

三、冲突解决与数据结构转换：

- 当插入新的键值对时，如果计算出的桶已经有元素存在（即发生哈希冲突），则新的键值对会被添加到该桶对应的链表中。
- 如果链表长度过长（超过8），并且HashMap的数组长度大于或等于64，则链表会转换为红黑树，以提高搜索效率。

四、扩容机制：

- HashMap内部有一个扩容机制，当元素数量达到数组长度与加载因子（默认为0.75）的乘积时，就会触发扩容操作。
- 扩容时，HashMap的容量会翻倍，并且所有的元素都需要重新散列到新的数组中。这个过程是为了保持哈希表的均匀分布和减少冲突。

五、线程安全：

- HashMap并不是线程安全的。如果在多线程环境下使用HashMap，可能会导致数据不一致的问题。如果需要线程安全的哈希表，可以考虑使用ConcurrentHashMap。

推荐学习资料

《JavaGuide》| 集合面试题：# HashMap底层如何实现的？

23. HashMap put流程？

解析

1. 初始化判断与数组准备
2. 计算hash值并确定桶的位置

3. 处理hash冲突

4. 扩容判断与执行

参考口述回答

HashMap的put方法流程在Java 8中可以归纳为以下几个主要步骤：

1. 初始化判断与数组准备：

- 首次调用put方法时，会检查HashMap内部的数组（称为桶数组或table）是否为空或者长度为0。
- 如果是，那么会进行初始化操作，根据设定的初始容量（或默认值16）来分配数组空间，并设定加载因子（默认为0.75）。

2. 计算hash值并确定桶的位置：

- 对传入的key计算hash值，HashMap会结合key的hashCode()和一些额外的位操作来确保散列的均匀性。
- 使用这个hash值来确定键值对应该存放在数组的哪个位置，这通常通过高效的位操作实现。

3. 处理hash冲突与数据结构选择：

- 如果计算出的桶位置已经有元素存在（即发生hash冲突），HashMap会检查该位置的数据结构。
- 如果该位置是链表结构，新的键值对会被添加到链表的末尾。在添加前会检查链表长度：
 - 如果链表长度小于8，直接插入链表尾部。
 - 如果链表长度达到8，并且当前HashMap的数组长度大于或等于64，则将链表转换为红黑树，然后在红黑树中插入新的键值对。
- 如果该位置已经是红黑树结构，新的键值对会被直接插入到红黑树中。

4. 扩容判断与执行：

- 在插入新元素后，HashMap会检查当前元素数量（即size）是否超过了容量（数组长度）与加载因子（默认为0.75）的乘积。
- 如果超过了这个乘积（即达到了扩容阈值），就会触发扩容操作。
- 扩容时，HashMap的容量会大致翻倍（通常是当前容量的两倍），并且所有的元素都需要重新散列到新的数组中。

推荐学习资料

《JavaGuide》| 源码分析：[HashMap put流程](#)

24. ConcurrentHashMap是怎么实现线程安全的？

解析

Java7的分段锁比较有点特点，一般需要结合Java8来回答

参考口述回答

在Java 7中，ConcurrentHashMap采用了**分段锁**的机制来实现线程安全。它将整个Map划分为多个段（Segment），每个段都相当于一个小型的HashMap，并且有自己的锁。当线程占用锁访问其中一个段数据的时候，其他段的数据也能够被其他线程访问，从而实现真正的并发访问。简单来说，就是多个线程可以同时操作不同的段，从而实现了高效的并发性能。

然而，在Java 8中，ConcurrentHashMap的实现方式发生了显著变化。Java 8中引入了更细粒度的同步控制——使用了一种称为“同步控制块 SCB”的机制，也被称为“桶锁”或“节点锁”。这种锁机制允许多个线程在高并发的情况下，更精细地控制数据的访问和修改。

具体来说，在Java 8的ConcurrentHashMap中，每个桶（bucket）都关联一个锁（**synchronized**）。当线程尝试访问或修改某个桶中的数据时，它只需要获取该桶对应的锁，而不是锁定整个Map或某个大的数据段。这种设计显著减少了线程之间的锁竞争，从而提高了并发性能。此外，Java 8的ConcurrentHashMap还采用了**CAS**（Compare-and-Swap）操作来支持无锁化（lock-free）的读写，进一步提升了性能。

推荐学习资料

《JavaGuide》| 集合面试题：[#ConcurrentHashMap](#) 线程安全的具体实现方式/底层具体实现

JAVA：并发编程

25. Java多个线程可能会有哪些线程安全问题，这些问题可以通过哪些方法进行解决？

解析

这个问题一般面试官会作为考察Java并发的话题切入问题，回答的时候可以整体回答，每一项不用太深入细节。面试官可能接着会进行追问，再回答细节。

参考口述回答

线程安全问题：

1. 数据竞争和一致性问题：

- 当多个线程同时访问和修改同一个共享数据时，可能会导致数据的不一致性。例如，两个线程同时对一个计数器进行增加操作，可能会因为线程调度导致计数不准确。

2. 原子性问题：

- 某些操作需要多个步骤完成，若这些步骤被多个线程交错执行，则可能导致数据状态错误。比如，一个线程正在写入数据，而另一个线程在此期间读取该数据，可能会读取到一个不一致的中间状态。

3. 可见性问题：

- 一个线程修改了共享变量的值，但其他线程可能无法立即看到这个修改，因为它们可能缓存了旧的值。这会导致数据不一致和其他线程做出错误的决策。

4. 有序性问题：

- 由于指令重排序或JVM优化，代码的执行顺序可能与编写顺序不同，这在多线程环境中可能导致难以预料的结果。

解决方法：

1. synchronized关键字：

- 使用synchronized关键字可以确保同一时间只有一个线程能够执行某个代码块或方法，从而防止数据竞争。它保证了可见性和原子性。

2. volatile关键字：

- volatile关键字用于声明变量，确保变量的更新对所有线程立即可见。它解决了可见性问题，但不保证原子性。

3. Lock接口和ReentrantLock类：

- Java提供了Lock接口和ReentrantLock类作为更灵活的线程同步机制。它们允许更精细的锁控制，包括尝试锁、定时锁、可中断锁等。

4. 原子类（Atomic Classes）：

- Java的java.util.concurrent.atomic包提供了一组原子变量类，如AtomicInteger、AtomicLong等。这些类的每个方法都是原子的，可以在多线程环境中无锁地安全操作。

5. 线程安全的数据结构：

- 使用Java提供的线程安全的数据结构，如ConcurrentHashMap、CopyOnWriteArrayList等，可以避免显式的同步操作。

6. 避免共享状态：

- 尽可能地减少共享状态是避免并发问题的最佳方法。例如，使用ThreadLocal来为每个线程提供独立的变量副本。

7. Happens-Before关系：

- 利用Java内存模型中的Happens-Before关系来确保多线程之间的操作顺序和可见性。

综上所述，解决Java多线程中的线程安全问题需要从数据竞争、原子性、可见性和有序性等多个方面入手。通过合理地使用同步机制、线程安全的数据结构和避免共享状态等方法，我们可以有效地解决这些问题并构建出健壮的多线程应用。

推荐学习资料

《JavaCore》：[Java并发安全性问题和解决思路](#)

26. Java线程的生命周期有了解吗？（类似问题：线程状态与切换）

解析

参考口述回答

Java线程的6种状态分别是：新建状态(New)、就绪状态(Runnable)、运行状态(Running)、阻塞状态(Blocked)、等待状态(Waiting)、以及终止状态(Terminated)。另外，还有一个特殊的等待状态，即定时等待状态(Timed Waiting)，这是等待状态的一种特例。

接下来，我将详细介绍这些状态之间的切换关系：

1. 新建状态(New)：

- 当我们使用`new`关键字创建一个线程对象时，该线程就处于新建状态。此时，线程还没有开始执行，也没有被启动。

2. 新建状态 -> 就绪状态(Runnable)：

- 当我们调用线程的`start()`方法时，线程会从新建状态进入到就绪状态。这意味着线程已经被启动，并且已经做好了执行的准备，但是否真正执行取决于操作系统的调度。

3. 就绪状态 -> 运行状态(Running)：

- 当线程获得CPU资源并开始执行任务时，它会从就绪状态进入到运行状态。在这个状态下，线程正在执行其`run()`方法中的代码。需要注意的是，只有一个线程可以处于运行状态，其他线程则需要等待或竞争资源。

4. 运行状态 -> 阻塞状态(Blocked)：

- 当线程试图获取一个内部的锁（不是`java.util.concurrent`库中的锁），而该锁被其他线程持有时，该线程会进入阻塞状态。当其他线程释放锁时，阻塞状态的线程会重新变为就绪状态，等待CPU的调度执行。

5. 运行状态/就绪状态 -> 等待状态(Waiting)/定时等待状态(Timed Waiting)：

- 线程可以通过调用`wait()`、`join()`或`LockSupport.park()`等方法进入等待状态，或者通过调用`Thread.sleep(long millis)`、设置了超时参数的`wait(long timeout)`、`join(long timeout)`或`LockSupport.parkNanos(long nanos)`、`LockSupport.parkUntil(long deadline)`等方法进入定时等待状态。在这些状态下，线程会暂停执行一段时间或者被特定条件唤醒，然后再次进入就绪状态等待CPU的调度执行。

6. 等待状态/定时等待状态/阻塞状态 -> 就绪状态：

- 当等待状态或定时等待状态的线程被其他线程通过`notify()`或`notifyAll()`方法唤醒，或者定时等待时间到达时，这些线程会重新进入就绪状态。同样地，当阻塞状态的线程获取到锁时，也会重新变为就绪状态。

7. 运行状态 -> 终止状态(Terminated)：

- 当线程执行完毕或者因异常而结束时，它会进入到终止状态。这意味着线程的执行已经结束，并且无法再次被启动。

推荐学习资料

《JavaGuide》| 并发面试题： [##说说线程的生命周期和状态？](#)

27. 介绍下volatile关键词？

解析

可见性，有序性，非原子性

参考口述回答

`volatile`是Java中的一个关键字，主要用于确保多线程环境中变量的可见性和有序性。当一个变量被声明为`volatile`时，它会告诉编译器和处理器，不要对这个变量的读写做优化处理，以确保每次对该变量的读写都是

直接从主内存中进行的，而不是从CPU缓存中进行。这样可以确保每个线程都能立即看到其他线程对该变量的最新修改。

具体来说，`volatile`关键字的作用主要体现在以下两个方面：

一、可见性：当一个线程修改了一个`volatile`变量的值，其他线程能够立即看到这个修改。这是因为`volatile`关键字会禁止CPU缓存和编译器优化，确保每次读写都是直接从主内存中进行的。这样，当一个线程修改了`volatile`变量的值后，其他线程在读取这个变量时，会直接从主内存中读取最新的值，而不是从CPU缓存中读取可能过时的值。

二、有序性：Java内存模型允许编译器和处理器对指令进行重排序，以提高执行效率。但是，在某些情况下，重排序可能会导致多线程环境中的问题。`volatile`关键字通过插入内存屏障来禁止指令重排序，从而确保代码的执行顺序与程序的顺序一致。这可以防止因重排序而导致的并发问题。

需要注意的是，虽然`volatile`关键字可以确保变量的可见性和有序性，但它并不能确保原子性。也就是说，`volatile`不能替代锁来确保复杂操作的原子性。在处理需要多个步骤的复合操作时，仍然需要使用锁或其他同步机制来确保操作的原子性。

推荐学习资料

《JavaGuide》| 并发面试题：##`volatile` 关键字

28. 介绍下synchronized关键字？

解析

如何使用；底层实现；锁升级

参考口述回答

synchronized关键字的介绍

`synchronized`是Java中的一个关键字，它用于控制多线程对共享资源的访问，以防止并发问题。具体来说，它可以确保被修饰的方法或代码块在任意时刻只能有一个线程执行。

synchronized的使用

1. 修饰实例方法：作用于当前实例加锁，进入同步方法前要获得当前实例的锁。
2. 修饰静态方法：作用于当前类对象加锁，进入同步方法前要获得当前类对象的锁。也就是给Class进加锁，注意是给这个类加锁，而不是类的某个实例加锁。
3. 修饰代码块：指定加锁对象，对给定对象加锁，进入同步代码块前要获得给定对象的锁。这种用法比较灵活，可以用于对任意代码块进行同步。

synchronized的底层实现

`synchronized`关键字的底层实现依赖于Java虚拟机（JVM）中的监视器锁（Monitor Lock）或称为内部锁。每个Java对象都有一个与之关联的监视器锁。当线程进入一个`synchronized`方法或代码块时，它会自动获取该锁，而在离开时则会自动释放该锁。

锁升级过程

JVM对`synchronized`的优化包括锁升级机制，具体过程如下：

1. **无锁状态**：一开始并没有任何锁。
2. **偏向锁**：当一个线程首次访问`synchronized`块时，会尝试使用偏向锁。偏向锁是一种优化手段，它的思想是偏向第一个获取它的线程，该线程在后续访问时无需再进行同步操作。如果在此期间没有其他线程访问该锁，则线程无需进行任何同步操作。
3. **轻量级锁**：如果有其他线程尝试访问已经被偏向锁定的资源，偏向锁会升级为轻量级锁。轻量级锁使用CAS（Compare and Swap）操作尝试获取锁，如果成功则顺利执行，否则进行自旋等待（即不断重试）。这个过程不需要线程挂起，因此效率相对较高。
4. **重量级锁**：如果自旋过程中仍有大量线程竞争锁，轻量级锁会升级为重量级锁。此时，未获取到锁的线程将被阻塞，进入等待状态。当锁被释放时，这些线程会被唤醒并尝试重新获取锁。重量级锁涉及到线程的挂起和唤醒，这需要通过操作系统来完成，因此性能开销相对较大。

总的来说，`synchronized`关键字的锁升级机制使得Java能够在运行时根据具体的竞争情况动态调整同步策略，从而在保障线程安全的同时提供尽可能高的性能。

推荐学习资料

《JavaGuide》| 并发面试题：##`synchronized` 关键字

29. 线程池参数？饱和策略？

解析

参考口述回答

1. 线程池参数？

1. `corePoolSize`（核心线程数）：

- 线程池中始终保持的线程数量，即使这些线程处于空闲状态，也不会被销毁。例如，可以设置为5，表示线程池将始终保持5个活跃线程。

2. `maximumPoolSize`（最大线程数）：

- 线程池中允许同时运行的最大线程数量。当工作队列已满，且线程数未达到这个最大值时，线程池会创建新的线程来处理任务。例如，设置为10，表示线程池最多可以有10个线程同时运行。

3. `keepAliveTime`（线程空闲时间）：

- 当线程池中的线程数量超过`corePoolSize`时，这是多余的空闲线程在被终止之前等待新任务的最长时间。例如，设置为60秒，表示空闲线程在等待60秒后若仍无任务处理则会被销毁。

4. `unit`（时间单位）：

- `keepAliveTime`的时间单位，可以是毫秒、秒等。例如，使用`TimeUnit.SECONDS`表示以秒为单位。

5. `workQueue`（工作队列）：

- 用于存储待执行的任务的阻塞队列。常见的类型有`LinkedBlockingQueue`、`ArrayBlockingQueue`等。例如，可以创建一个容量为10的`LinkedBlockingQueue`作为任务队列。

6. `threadFactory`（线程工厂）：

- 用于创建新线程的工厂类，可以定制线程的名称、优先级等属性。通常情况下，使用默认的线程工厂即可。

7. handler（拒绝策略）：

- 当线程池中的线程数达到maximumPoolSize，且工作队列已满时，用于处理无法执行的新任务的策略。Java线程池提供了四种内置的拒绝策略：`AbortPolicy`（默认，抛出异常）、`CallerRunsPolicy`（调用者执行）、`DiscardPolicy`（丢弃任务）、`DiscardOldestPolicy`（丢弃最旧任务）。

2. 饱和策略？

1. AbortPolicy（中止策略）：

- 这是默认的拒绝策略。当无法处理新任务时，它会直接抛出一个`RejectedExecutionException`异常。这种策略适用于关键业务，因为它能及时反馈程序的运行状态。

2. CallerRunsPolicy（调用者运行策略）：

- 当线程池无法处理新任务时，该策略不会抛出异常，而是直接在提交任务的线程中执行该任务。这提供了一种简单的任务降级机制，但需要注意，如果提交任务的线程是一个重要线程（如UI线程），则可能会导致问题。

3. DiscardPolicy（丢弃策略）：

- 当线程池无法处理新任务时，该策略会默默地丢弃这个任务，不会有任何反馈。这适用于那些无关紧要的任务。

4. DiscardOldestPolicy（丢弃最旧策略）：

- 当线程池无法处理新任务时，该策略会丢弃工作队列中最旧的任务，然后尝试重新提交当前的任务。这种策略适用于需要发布消息等场景。

推荐学习资料

《JavaGuide》| 并发面试题：###线程池常见参数有哪些？如何解释？

《JavaGuide》| 并发面试题：#线程池的饱和策略有哪些？

30. 线程池任务进来处理过程？

解析

参考口述回答

1. 任务提交：

当有新任务提交到线程池时，首先会触发任务的接收过程。这通常通过调用线程池的`execute()`或`submit()`方法实现，其中`execute()`用于执行不需要返回值的任务，而`submit()`则用于执行需要返回值的任务。

2. 检查核心线程数：

线程池在接收任务后，会首先检查当前线程池中的线程数量。如果线程数量小于设定的`corePoolSize`（核心线程数），则线程池会立即创建一个新的工作线程来处理该任务。

3. 任务排队：

如果线程池中的线程数量已经达到`corePoolSize`，但工作队列（如`LinkedBlockingQueue`或`ArrayBlockingQueue`）未满，新提交的任务会被添加到工作队列中等待执行。

4. 线程池扩展：

如果工作队列已满，且当前线程数小于`maximumPoolSize`（最大线程数），线程池会尝试创建新的非核心线程来处理任务。这些非核心线程在空闲时间超过`keepAliveTime`且线程数大于`corePoolSize`时会被销毁。

5. 拒绝策略：

当工作队列已满，且线程数已经达到`maximumPoolSize`时，线程池无法再处理新任务。此时，会根据设定的拒绝策略来处理新提交的任务。常见的拒绝策略包括丢弃任务、丢弃最旧的任务、抛出异常或让提交任务的线程自己执行任务。

推荐学习资料

《JavaGuide》| 并发面试题：[#线程池处理任务的流程了解吗？](#)

31. 什么是死锁？死锁的4个条件？Java死锁后怎么排查？如何避免死锁？

解析

参考口述回答

什么是死锁？

死锁是指两个或两个以上的进程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象。具体来说，当多个进程或线程互相等待对方释放资源时，就会导致死锁。若无外力作用，这些进程都将无法推进下去，此时称系统处于死锁状态或系统产生了死锁，这些永远在互相等待的进程称为死锁进程。

死锁的4个条件

死锁的产生需要满足以下四个必要条件：

1. **互斥条件**：一个资源每次只能被一个进程使用。
2. **请求与保持条件**：一个进程因请求资源而阻塞时，对已获得的资源保持不放。
3. **不剥夺条件**：进程已获得的资源，在未使用完之前，不能强行剥夺。
4. **循环等待条件**：若干进程之间形成一种头尾相接的循环等待资源关系。

Java死锁后怎么排查？

在Java中，如果怀疑出现了死锁，可以通过以下几种方法进行排查：

1. **使用jstack命令**：首先使用jps查看Java进程编号，然后使用jstack查看进程信息。如果出现死锁，jstack会在最后给出进程的分析信息，表示出现了死锁。
2. **使用图形化工具jconsole**：打开jconsole，选择要检测的程序，然后点击检测死锁。检测后，可以看到存在死锁的线程。

3. **使用图形化工具jvisualvm**：在左侧选择要检测的Java程序，如果检测到死锁，右侧会出现红色的字提醒。然后可以查看堆栈信息，堆栈信息和jstack显示的差不多，但堆栈信息中间部分可能比jstack多出一些信息。

如何避免死锁？

避免死锁可以从以下几个方面入手：

1. **破坏互斥条件**：这个条件比较难以破坏，因为某些资源本身就需要独占使用。不过，有些资源可以通过共享的方式使用，比如只读文件。
2. **破坏请求与保持条件**：可以一次性申请所有需要的资源，而不是逐步申请。或者当某个进程申请资源时，如果它之前已经占有了其他资源，那么需要先将之前占有的资源释放掉。
3. **破坏不剥夺条件**：当某个进程获得部分资源而得不到其他资源时，必须放弃已获得的资源，让其他进程使用。等以后再重新申请。
4. **破坏循环等待条件**：可以通过顺序申请资源来预防。即对每个设备编号，进程在申请资源时必须按照编号的顺序进行申请，不能逆序申请。另外，还可以采用避免无限等待的方法，即给每个进程设置一个最大等待时间，超过这个时间就放弃申请。

除了以上方法外，还可以通过设置锁的超时时间、检测到死锁时主动放弃锁等策略来避免死锁的发生。

推荐学习资料

《JavaGuide》| 并发面试题：[#什么是线程死锁?如何避免死锁?](#)

《极客时间网课》| 并发编程78讲：[69 如何用命令行和代码定位死锁?](#)

32. 锁ReentrantLock和synchronized有什么区别？

解析

获取方式；公平性；灵活性；中断；实现层次

参考口述回答

一、锁的获取方式

1. synchronized：隐式获取锁，即当线程进入同步代码块或方法时自动获取锁，并在退出时自动释放锁。
2. ReentrantLock：显式获取锁，需要程序员手动调用lock()方法获取锁，并在使用完毕后调用unlock()方法释放锁。

二、锁的公平性

1. synchronized：是非公平锁，它不能保证等待时间最长的线程最先获取锁。这意味着线程的调度顺序是不确定的，可能会导致某些线程长时间得不到执行。
2. ReentrantLock：可以是公平锁，也可以是非公平锁。当设置为公平锁时，它能够保证等待时间最长的线程最先获取锁，从而提供了一种更公平的线程调度方式。

三、锁的灵活性

1. synchronized：功能相对固定，缺乏一些高级功能。例如，无法设置超时时间，无法判断锁是否被其他线程持有，也不支持使用Condition类实现线程等待/通知机制。

2. ReentrantLock：提供了更多的灵活性。例如，可以设置获取锁的超时时间，可以判断锁是否被其他线程持有，以及支持使用Condition类实现更复杂的线程等待/通知机制。

四、锁的状态判断与中断

1. synchronized：无法判断锁的状态，且不可被中断。即使在发生异常时，它也会自动释放锁。
2. ReentrantLock：可以通过isLocked()方法判断锁的状态（是否被锁定）。此外，它支持可中断的获取锁方式（lockInterruptibly()），这意味着在等待获取锁的过程中，如果线程被中断，它将能够响应这个中断并停止等待。

五、实现层次与锁信息存储

1. synchronized：是JVM层次的锁，锁信息保存在对象头中。这意味着它的实现与Java虚拟机紧密相关。
2. ReentrantLock：是JDK层次的锁，通过代码中int类型的state标识来标识锁的状态。这使得它在某些方面比synchronized更加灵活和可配置。

推荐学习资料

《JavaGuide》| 并发面试题： [synchronized 和 ReentrantLock 有什么区别？](#)

33. 介绍一下AQS?

解析

参考口述回答

一、AQS概述

AQS，即AbstractQueuedSynchronizer，是Java并发包中的一个核心组件。它是一个用于构建锁和同步器的框架，为许多同步器提供了一种高效且灵活的实现方式。AQS的设计简化了并发工具的开发，让开发者能够更容易地构建出复杂的同步器。

二、AQS的主要作用

1. **提供同步基础**：AQS为Java中的各种同步器提供了一个统一的基础框架，如ReentrantLock、Semaphore等。
2. **简化同步器开发**：通过使用AQS，开发人员可以避免重复实现同步器的底层机制，从而更加专注于业务逻辑的实现。
3. **高效且灵活**：AQS提供了一种高效且灵活的实现同步器的方式，使得并发编程更加简单和高效。

三、AQS的工作原理

1. **同步状态**：AQS使用一个int类型的变量来表示同步状态，通过CAS（Compare And Swap）操作来实现状态的改变。这个状态可以被视为一种资源或者锁的占用情况。
2. **队列管理**：AQS内部维护了一个双向队列（基于CLH队列的变体实现），用于管理请求资源的线程。当线程请求资源时，如果资源被占用，线程会被封装成一个Node节点并加入到同步队列中。
3. **线程唤醒与阻塞**：当持有资源的线程释放资源时，AQS会从同步队列中唤醒一个线程来获取资源。这一过程中，AQS会进行线程的阻塞与唤醒操作。

4. **独占与共享模式**：AQS支持独占模式和共享模式。在独占模式下，只有一个线程能够获取资源；在共享模式下，多个线程可以同时获取资源。

四、AQS的特点

1. **可重入性**：AQS支持锁的重入，即同一个线程可以多次获取同一把锁而不会出现死锁。
2. **灵活性**：AQS提供了一个框架，开发者可以基于此框架进行扩展和定制化，以满足不同的同步需求。
3. **高效性**：AQS通过精细化的状态管理和线程唤醒机制，实现了高效的并发控制。

推荐学习资料

《JavaGuide》| 并发面试题：[AQS是什么？AQS原理](#)

《JavaGuide》| 并发面试题：[AQS详解](#)

34. 乐观锁和悲观锁？CAS？并发原子类？

解析

参考口述回答

1. 什么是乐观锁和悲观锁

悲观锁，顾名思义，它持有一种悲观的态度，总是假设最坏的情况会发生，数据在任何时候都可能被其他事务修改。因此，它会在数据处理前先进行锁定，确保数据在处理过程中不会被其他事务修改或访问，这就好像我们在做某件事情之前，先把相关的资源都占住，以防别人抢走。这种方式虽然安全，但可能会降低系统的并发性能，因为它会阻止其他事务访问被锁定的数据。

乐观锁则相反，它持有一种乐观的态度，认为数据在大部分情况下不会被其他事务修改。因此，它不会一开始就锁定数据，而是在数据提交更新的时候检查数据是否被其他事务修改过。这就像是我们先去做自己的事情，等到最后需要提交成果的时候，再检查是否有其他人已经修改过相关数据。如果数据没有被修改过，那么我们的更新就可以成功提交；如果数据已经被其他事务修改过，那么我们的更新就会失败，需要进行相应的处理。这种方式可以提高系统的并发性能，因为它允许多个事务同时访问和修改数据，但需要在数据提交时进行冲突检测和处理。

2. 如何实现乐观锁？（版本号机制+CAS）

乐观锁的实现主要依赖于两种机制：版本号机制和CAS。

首先，我们来说说版本号机制。这种机制的核心思想是在数据中加入一个版本号，每次数据变更时，版本号都会递增。当我们要对数据进行修改时，首先会检查数据的版本号。如果数据的版本号与我们预期的一致，那么我们就可以进行数据修改，并将版本号递增，以此确保数据的一致性。如果版本号与我们预期的不一致，那就说明在我们读取数据后，有其他操作对数据进行了修改，这时我们就需要采取一些策略，比如重新读取数据或者放弃修改，以此避免数据冲突。

接下来，我们谈谈CAS，也就是Compare-And-Swap。CAS是一种原子操作，它涉及到三个关键的值：内存中的当前值，我们预期的值，以及我们想要设置的新值。CAS操作会先比较内存中的值与预期的值是否相等，如果相等，那么就会把内存中的值替换为我们设置的新值。这个过程是原子的，也就是说，在这个操作过程

中，不会有其他线程来打断。如果内存中的值与预期的值不相等，那么CAS操作就会失败，这时我们可以选择重试或者采取其他策略。

3. CAS是怎么实现的（操作系统层面）？CAS ABA问题是什么？Java如何解决ABA问题的？

1. CAS是怎么实现的（操作系统层面）？

CAS，即Compare-And-Swap，是一种无锁的原子操作。在操作系统层面，CAS的实现依赖于硬件支持的原子性指令。这些指令能够确保在操作过程中不会被其他线程打断，从而保证操作的原子性。具体来说，CAS操作会先比较内存中的值与预期的值是否相等，如果相等，则会把内存中的值更新为新值。这个过程是原子的，也就是说，在这个操作过程中，不会有其他线程来打断。

1. CAS的ABA问题是什么？

CAS的ABA问题是一个在多线程环境下可能出现的问题。简单来说，就是当一个线程准备进行CAS操作时，它会先读取某个内存地址的值（假设为A），然后去做一些其他事情。在这期间，另一个线程可能已经将这个内存地址的值从A改成了B，然后又改回了A。当第一个线程回来准备进行CAS操作时，它发现内存中的值还是A，就会误认为这个值没有被其他线程修改过，从而进行错误的CAS操作。这就是所谓的ABA问题。

1. Java如何解决ABA问题的？

Java为了解决CAS的ABA问题，提供了两种主要的类：AtomicStampedReference和AtomicMarkableReference。这两种类都引入了一个额外的标记或版本号来记录数据的变化。

- AtomicStampedReference使用一个标记（stamp）来记录对象的版本号。当对象发生变化时，版本号会自动增加。在进行CAS操作时，不仅需要比较对象的值，还需要比较版本号。只有在值和版本号都匹配的情况下，CAS操作才会成功。
- AtomicMarkableReference则使用一个布尔标记（mark）来表示对象的状态是否发生过改变。同样地，在进行CAS操作时，需要同时比较对象的值和标记。

这两种方式都能有效地解决CAS的ABA问题，因为它们都能检测到数据在CAS操作期间是否发生过变化，即使数据的值最终又回到了原来的状态

4. 并发原子类

Java并发原子类介绍

Java中的并发原子类是一组用于实现线程安全的可变变量的类，这些类位于`java.util.concurrent.atomic`包下。这些原子类提供了一种机制，让多线程可以按照预期的方式进行原子性的操作，从而避免数据竞争问题。

1. 原子类的基本特性：

- 原子性：原子类提供的操作都是原子性的，即在执行过程中不会被其他线程中断，确保操作的完整性和一致性。
- 高效性：相比通过synchronized和Lock等方式实现的线程安全同步操作，原子类通常具有更高的性能，因为它们采用了无锁（lock-free）的方式来实现线程安全访问。
- 简单易用：原子类提供了简洁明了的API，使得并发编程更加简单。

2. 常用的原子类：

- `AtomicInteger`：用于原子性操作int类型的变量。
- `AtomicLong`：用于原子性操作long类型的变量。
- `AtomicBoolean`：用于原子性操作boolean类型的变量。
- 还有其他如`AtomicReference`、`AtomicStampedReference`、`AtomicMarkableReference`等用于引用类型的原子操作。

3. 原子类的实现机制：

- 原子类的实现大多基于底层的硬件支持，如CPU的CAS（Compare and Swap）指令。CAS是一种非阻塞算法，通过比较当前变量的值和期望值是否相等来判断是否可以进行更新操作。
- 如果相等，则执行更新操作；如果不等，则重新尝试，直到成功为止。这种方式避免了加锁带来的性能开销。

4. 使用场景：

- 在需要高性能、高并发的场景下，使用原子类可以显著提升程序的性能。
- 原子类特别适合用于实现计数器、状态标志等需要线程安全访问的变量。

5. 注意事项：

- 虽然原子类可以确保单个操作的原子性，但是多个操作之间的组合并不一定是原子性的。如果需要保证多个操作的原子性，应该使用锁或其他并发控制机制。
- 在使用原子类时，要注意其适用范围和原子操作的特点，合理设计并发控制策略。

推荐学习资料

《JavaGuide》| 并发面试题：[乐观锁和悲观锁](#)

《JavaGuide》| 详解：[# Atomic 原子类总结](#)

35. ThreadLocal介绍下？ThreadLocal内存泄漏问题？

解析

介绍；实现原理；内存泄漏

参考口述回答

ThreadLocal介绍

ThreadLocal是Java中的一个类，它提供了线程局部（thread-local）变量。这些变量与普通的可共享变量不同，因为每一个访问这个变量的线程都有它自己的独立初始化的变量副本。通过ThreadLocal，我们可以避免多线程环境下的数据不一致问题，因为每个线程都操作自己的数据，不存在线程安全问题。ThreadLocal适用于每个线程需要自己独立的数据副本，并且该数据可以在多个方法中使用，但在线程之间应该是隔离的场景。

ThreadLocal底层实现原理

ThreadLocal的底层实现原理主要依赖于Thread类中的threadLocals属性，这是一个ThreadLocalMap类型的变量。ThreadLocal本身并不存储数据，而是作为ThreadLocalMap的管理者。当调用ThreadLocal的set()方法时，会将ThreadLocal的引用作为key，用户传入的值作为value存入ThreadLocalMap中。每个线程内部都

维护着自己的ThreadLocalMap，这样每个线程的读写操作都是基于线程本身的一个私有的变量副本，线程之间的数据是相互隔离的。

具体来说，ThreadLocalMap是通过一个Entry数组实现的，每个ThreadLocal实例在数组中都有对应的索引位置，这个位置是通过ThreadLocal的hashCode()方法和数组长度取模得到的。当线程调用ThreadLocal的get()或set()方法时，它会根据自己线程内的ThreadLocalMap来存取数据。

ThreadLocal内存泄漏问题

ThreadLocal如果使用不当，可能会导致内存泄漏。这主要是因为ThreadLocalMap中的Entry对ThreadLocal实例是弱引用（WeakReference），而对应的值value则是强引用。如果ThreadLocal实例被回收了，但是对应的value还没有被回收，那么这部分数据就可能成为内存泄漏的源头。

为了避免这种情况，有几种方法：

1. 使用完ThreadLocal后，及时调用其remove()方法清除对应的Entry，避免因为线程长时间存活而导致的内存泄漏。
2. 将ThreadLocal变量设置为static和final，以延长其生命周期，减少被垃圾回收的可能性。但这并不是根本的解决办法，因为即使ThreadLocal实例不被回收，其对应的value还是可能被长时间持有而无法释放。

因此，最佳实践是在使用完ThreadLocal后，及时通过remove()方法清除数据，以避免潜在的内存泄漏问题。

推荐学习资料

《JavaGuide》| 并发面试题：[# ThreadLocal 面试题](#)

《JavaGuide》| 详解：[# ThreadLocal 详解](#)

JAVA：JVM

36. 介绍一下JVM的运行时数据区（内存结构）？

解析

线程私有：程序计数器；虚拟机栈；本地方法栈； 线程共享：Java堆区；方法区

参考口述回答

首先是**程序计数器**，这是一块较小的内存空间。它可以看作是当前线程所执行的字节码的行号指示器。每个线程都有一个独立的程序计数器，用于记录下一条要执行的指令的地址。

接下来是**虚拟机栈**，这也是线程私有的内存区域。每个线程在创建时都会创建一个虚拟机栈，它描述的是Java方法执行的内存模型。栈中的每个栈帧都存储着局部变量、操作数栈、动态链接和方法出口等信息。当方法被调用时，一个新的栈帧会被压入栈中，当方法执行完毕后，对应的栈帧会被弹出。

与虚拟机栈相似的是**本地方法栈**，它为虚拟机使用到的Native方法提供内存空间。它也是线程私有的，与虚拟机栈不同的是，虚拟机栈为执行Java方法服务，而本地方法栈为执行Native方法服务。

再来看**Java堆区**，这是JVM内存中最大的一块区域，用于存储Java对象实例。堆区是所有线程共享的，它还可以被细分为年轻代和老年代，其中年轻代又可以分为Eden区和Survivor区。堆区的大小是在虚拟机启动时

就已经设定好的，但可以通过JVM参数来进行调整。

最后是**方法区**，这也是一个线程共享的内存区域。它用于存储已被虚拟机加载的类信息、常量、静态变量、即时编译器编译后的代码等数据。方法区中有一个非常重要的部分叫做运行时常量池，它存放着编译期生成的各种字面量和符号引用。

推荐学习资料

《JavaGuide》| JVM面试题： [# Java内存区域详解（重点）](#)

37. JVM内存分配策略有哪些？

解析

参考口述回答

首先，**对象优先分配在Eden区**。当一个新的对象需要被创建时，JVM会尝试在Eden区域进行分配。这是最常见的内存分配策略。

其次，**大对象直接进入老年代**。为了避免在Eden区和Survivor区之间发生大量的内存复制，对于那些需要大量连续内存空间的Java对象，如很长的字符串以及数组，JVM会直接将它们分配到老年代。

再者，**长期存活的对象会进入老年代**。JVM为了识别哪些是长期存活的对象，会给每个对象定义一个对象年龄计数器。当一个对象经过多次Minor GC后仍然存活，其年龄计数器会增加，当达到一定的阈值（默认是15），该对象就会被移动到老年代。

此外，还有**动态对象年龄判定**的策略。如果Survivor空间中相同年龄所有对象大小的总和大于Survivor空间的一半，那么年龄大于或等于该年龄的对象可以直接进入老年代，而无需等到年龄阈值中要求的年龄。

最后，为了保障内存分配的顺利进行，JVM还采用了**空间分配担保机制**。在进行Minor GC之前，JVM会检查老年代是否有足够的空间来容纳Survivor区中的所有对象，以确保不会发生内存溢出。

推荐学习资料

《JavaGuide》| JVM面试题： [#内存分配和回收原则](#)

38. 可达性分析算法？哪些对象可以作为 GC Roots对象？ 引用类型？

解析

可达性分析；GC Roots; 强软弱虚引用

参考口述回答

1. 可达性分析算法？哪些对象可以作为 GC Roots对象？

可达性分析算法是JVM中用于确定哪些对象应该被垃圾收集的重要算法。它的核心思想是从一组被称为GC Roots的对象开始，沿着引用链向下搜索，找到所有可达的对象。如果某个对象从GC Roots出发无法被搜索到，即没有引用链相连，那么这个对象就被视为垃圾，可以被回收。

接下来，关于哪些对象可以作为GC Roots，主要包括以下几类：

- 一、虚拟机栈中的引用对象。这包括当前活跃线程中的本地变量和正在执行的方法中的参数对象，这些都是由活动线程所持有的引用。
- 二、方法区中的类静态属性引用的对象。这类对象是通过类的静态变量引用的，它们在程序运行期间一直存在。
- 三、方法区中的常量引用的对象。比如字符串常量池中的引用，这些常量在程序运行期间是不可变的。
- 四、本地方法栈中的JNI引用对象。这些是在Java代码中与本地代码进行交互时创建的对象引用。
- 五、JVM内部的一些特殊对象，如系统类加载器、一些重要的异常类、预分配的对象等，也可以作为GC Roots。这些对象是由JVM为了特殊目的而保留的。

2. 引用类型？

首先，我们来谈谈**强引用**。强引用是Java中最常见的引用类型，它是默认的引用方式。当一个对象被强引用所指向时，垃圾回收器是不会回收这个对象的，即使系统内存不足。只要强引用存在，对象就不会被判定为可回收。例如，当我们通过**new**关键字创建一个对象并将其赋值给一个变量时，这个变量就是对该对象的一个强引用。

接下来是**软引用**。软引用是通过**SoftReference**类来实现的。它用于描述那些还有用但并非必需的对象。当系统内存足够时，软引用指向的对象不会被回收；但当系统内存不足时，垃圾回收器会开始考虑回收这些对象。这种特性使得软引用非常适合用于实现内存敏感的缓存，比如图片缓存等。当内存紧张时，可以自动释放这些缓存，从而避免内存溢出。

再来说说**弱引用**。弱引用通过**WeakReference**类来实现，它比软引用更加“脆弱”。只要垃圾回收器开始运行，无论当前内存是否充足，弱引用指向的对象都会被回收。弱引用的生命周期相对较短，主要用于解决内存泄漏问题，如缓存、监听器等情况。当对象只被弱引用所指向时，垃圾回收器会毫不犹豫地将其回收。

最后是**虚引用**。虚引用是最弱的一种引用类型，它不会对对象的生存时间构成影响，也无法通过虚引用来获取一个对象实例。虚引用的主要作用是跟踪对象被垃圾回收的状态。当垃圾回收器准备回收一个被虚引用指向的对象时，会将该虚引用加入到与之关联的引用队列中。这样，我们可以通过检查引用队列来了解对象何时被回收。虚引用通常用于实现一些特殊的内存管理需求。

推荐学习资料

《JavaGuide》| 并发面试题：[#死亡对象的判断方法？](#)

39. 垃圾回收算法？

解析

标记-清除算法；复制算法；标记-整理算法；其他

参考口述回答

首先是**标记-清除算法**，这是最基本的垃圾回收算法。它的原理很简单，分为两个阶段：标记阶段和清除阶段。在标记阶段，垃圾回收器会从根对象开始，递归地访问这些对象的引用，将访问到的对象都标记为‘存活’。然后在清除阶段，垃圾回收器会遍历整个堆内存，将没有被标记的对象全部清除。这种算法的优点是实现简单，但缺点是会造成内存碎片化，导致后续分配大对象时可能找不到连续的内存空间。

接下来是**复制算法**，这种算法将内存划分为两个等大小的区域。在任意时间点，只有其中一个区域被使用，另一个区域是空闲的。当当前使用的区域内存耗尽时，垃圾回收器会将所有存活的对象复制到另一个空闲区域中，然后清空当前区域。这种算法的优点是垃圾回收速度快，但缺点是内存使用率只有50%。复制算法通常用于新生代的垃圾回收，特别是针对大量短命对象进行回收时效率很高。

还有一种算法是**标记-整理算法**，它标记存活对象的处理方式与标记-清除算法相同，但在清除阶段，它会将所有存活的对象都移动到一端，然后清理掉边界以外的内存。这种算法的优点是不会造成内存碎片化，但缺点是移动存活对象的成本较高。这种算法通常用于老年代的垃圾回收，因为老年代中存活的对象较多，不适合使用复制算法。

除了以上几种基本算法外，还有一些更先进的垃圾回收算法，如**分代收集算法**和**区域化垃圾回收算法**等。这些算法根据对象的存活周期和内存布局特点进行优化，以提高垃圾回收的效率和性能。

推荐学习资料

《JavaGuide》| JVM面试题：[垃圾收集算法](#)

40. 介绍下垃圾收集器？

解析

参考口述回答

首先是**Serial收集器**，这是最基本的收集器，采用单线程方式进行垃圾回收。它在执行时会暂停所有工作线程，因此不适用于需要高并发处理的系统。Serial收集器简单高效，在单核CPU或小内存环境下表现良好，适用于Client模式下的虚拟机。

接下来是**ParNew收集器**，它是Serial收集器的多线程版本，适用于多核CPU环境。ParNew收集器通过多个线程并行执行垃圾回收，从而提高了回收效率。它通常与CMS收集器配合使用，以减少用户线程的停顿时间。

Parallel Scavenge收集器则是一个以吞吐量优先的并行收集器。它使用多线程和复制算法进行垃圾回收，旨在达到一个可控制的吞吐量。Parallel Scavenge收集器适用于后台运算等弱交互场景，能够高效利用CPU时间，尽快完成程序运算任务。

对于老年代，我们有**Serial Old收集器**，它是Serial收集器的老年代版本，同样采用单线程方式进行垃圾回收。此外**Parallel Old收集器**则是Parallel Scavenge的老年代版本，使用多线程和“标记-整理”算法进行垃圾回收。

CMS收集器是一种以获取最短停顿时间为目标的收集器，非常适合对响应时间有要求的系统。它使用“标记-清除”算法，并分为初始标记、并发标记、重新标记和并发清除四个阶段。然而，CMS收集器可能会产生内存碎片，并且对CPU资源敏感。

G1收集器则是一个面向大堆内存和多核处理器的收集器，它将堆内存分割成多个区域进行回收，旨在提供可预测的停顿时间。G1收集器适用于大堆内存和需要低延迟的应用程序，能够在多核处理器上发挥更好的性

能。

最后，**ZGC收集器**是一个面向大堆内存、低延迟的垃圾收集器。它通过并发压缩来减少内存碎片，并且能够在多核环境下实现较低的停顿时间。ZGC收集器适用于需要快速响应和低停顿时间的应用场景。

推荐学习资料

《JavaGuide》| JVM面试题：[#垃圾收集器](#)

41. 类的生命周期？（含类加载的过程）

解析

加载、验证、准备、解析、初始化、使用和卸载

参考口述回答

首先是**加载阶段**。这个阶段的主要任务是把类的.class文件从各种介质，如文件系统、网络或其他来源读入到JVM的方法区中。加载过程是通过类的全限定名来获取定义此类的二进制流，然后将这个字节流所代表的静态存储结构转化为方法区的运行时数据结构，最后在内存中生成一个代表这个类的java.lang.Class对象，作为方法区这个类的各种数据的访问入口。

接下来是**验证阶段**。这一阶段的目的是为了确保被加载的类的正确性，防止因类文件被篡改或损坏而导致系统崩溃。验证阶段会进行文件格式验证、元数据验证、字节码验证和符号引用验证等。

准备阶段是紧随验证阶段之后的，它的主要任务是为类的静态变量分配内存，并将其初始化为默认值。这里需要注意的是，仅为静态变量分配内存，实例变量将会在对象实例化时随着对象一起分配在堆中。

然后是**解析阶段**。解析阶段的任务是把类中的符号引用转换为直接引用。符号引用是以一组符号来描述所引用的目标，而直接引用是可以直接指向目标的指针、相对偏移量或一个能间接定位目标的句柄。如果符号引用指向的是一个方法，那么这个符号引用会被解析为该方法的直接引用。

接下来是**初始化阶段**。初始化阶段是执行类构造器方法clinit()的过程。此方法由编译器自动收集类中的所有类变量的赋值动作和静态代码块集合来的。当初始化一个类的时候，如果发现其父类还没有进行过初始化，则需要先触发其父类的初始化。虚拟机会保证一个类的clinit()方法在多线程环境中被正确地加锁和同步。

使用阶段就是类的实际应用过程，包括创建类的对象、调用类的方法等。这个阶段没有特别的操作，主要是根据程序的需求来使用类。

最后是**卸载阶段**。当一个类不再被使用时，或者系统内存不足时，JVM就会回收其占用的空间，这个过程就是类的卸载。类的卸载是由垃圾回收器来完成的，它会检测类是否还在被使用，如果没有被使用，就会将其卸载以释放内存空间。

推荐学习资料

《JavaGuide》| JVM面试题：[类的生命周期](#)

42. 类加载器？双亲委派模型？如何打破双亲委派模型？

解析

参考口述回答

一、类加载器

类加载器是Java虚拟机（JVM）的一部分，它的职责是将类的字节码加载到内存中，并生成对应的Class对象。Java中的类加载器主要分为三种类型：

1. **启动类加载器（Bootstrap Class Loader）**：这是JVM的一部分，负责加载Java的核心类库，如rt.jar中的类。它是由C++实现的，因此在Java代码中无法直接获取。
2. **扩展类加载器（Extension Class Loader）**：它用于加载Java的扩展类库，这些类库位于JRE的lib/ext目录下。扩展类加载器是由启动类加载器加载的Java类。
3. **应用程序类加载器（Application Class Loader）**：也称为系统类加载器，是加载应用程序类的默认类加载器。它负责加载应用程序classpath下的类，包括用户自定义的类和第三方库。

二、双亲委派模型

双亲委派模型是Java的类加载机制，用于保证Java类的稳定性和安全性。当类加载器收到类加载请求时，它首先会将这个请求委派给父类加载器去完成，只有当父类加载器无法完成加载时，才会由当前类加载器自己去加载。这个模型的工作流程如下：

1. 当前类加载器首先检查是否已经加载过这个类，如果已经加载则直接返回。
2. 如果没有加载过，则委派给父类加载器去加载。
3. 父类加载器按照同样的方式去检查和委派，直到到达顶层的启动类加载器。
4. 如果顶层的启动类加载器仍无法完成加载，则返回当前类加载器，尝试自己去加载。

双亲委派模型可以防止类的重复加载，保证了类加载的唯一性和一致性，同时也能够防止恶意的代码注入和破坏。

三、如何打破双亲委派模型

自定义类加载器：通过自定义类加载器，我们可以自定义类的加载顺序和逻辑，从而实现对双亲委派模型的改变。自定义类加载器需要继承`java.lang.ClassLoader`类，并重写`findClass`，如果想不遵循双亲委派的类加载顺序，还需要重写`loadClass`。

推荐学习资料

《JavaGuide》| JVM面试题：[#打破双亲委派模型方法](#)

《CSDN》| 博客：[# Java双亲委派模型：为什么要双亲委派？如何打破它？破在哪里？](#)

43. JVM线上问题排查和性能调优？

解析

参考口述回答

1. 如何定位排查OOM问题？

当Java虚拟机（JVM）出现OutOfMemoryError（OOM）时，通常意味着JVM没有足够的内存空间来分配新对象。OOM可能由多种原因引起，包括但不限于内存泄漏、过大的垃圾回收开销、不恰当的堆大小设置等。为了排查和定位OOM问题，我们可以按照以下步骤进行：

一、收集错误信息

1. 当OOM发生时，JVM通常会输出堆栈跟踪信息和堆转储（heap dump）。这些信息对于后续的问题分析至关重要。
2. 如果没有自动生成这些信息，可以通过添加JVM参数来启用。例如，使用`-XX:+HeapDumpOnOutOfMemoryError`参数可以在OOM时自动输出堆转储文件。

二、初步分析

1. 查看OOM的错误类型，如Java heap space、GC overhead limit exceeded、Permgen space等，这有助于缩小问题范围。
2. 根据错误类型，可以初步判断OOM的原因。例如，Java heap space通常意味着堆内存不足，可能是内存泄漏或堆大小设置不当导致的。

三、使用工具进行深入分析

1. **jVisualVM和jConsole**：这两个工具可以实时监控JVM的性能指标，包括内存使用情况、线程状态和垃圾回收情况等。它们有助于发现内存泄漏和性能瓶颈。
2. **MAT（Memory Analyzer Tool）**：通过分析堆转储文件，MAT可以帮助我们找到潜在的内存泄漏和占用大量内存的对象。
3. **GCViewer或GCEasy**：这些工具可以分析GC日志，查看垃圾收集器的行为和效率，从而判断是否存在过度的垃圾回收或内存泄漏等问题。

四、定位和排查问题

1. **内存泄漏定位**：使用MAT等工具打开堆转储文件，查找占用内存最大的对象。通过分析这些对象的引用链，可以确定是否有对象不应该被保留但却无法被GC清理，即内存泄漏。
2. **检查代码**：审查代码中是否存在不必要的大对象分配或内存泄漏的情况。特别注意那些创建了大量对象或使用了大量内存的代码段。
3. **调整配置**：根据应用的需求和初步分析的结果，调整JVM的配置，如堆大小、垃圾回收策略等。有时候，简单的配置调整就能解决OOM问题。

五、解决方案实施与验证

1. 在定位到具体问题后，采取相应的解决措施，如修复内存泄漏、优化代码、调整JVM配置等。
2. 实施解决方案后，需要进行充分的测试以验证问题是否已得到解决。这包括功能测试、性能测试以及长时间的稳定性测试。

2.如何定位排查频繁GC的问题？

首先，我们要明确GC的频繁发生可能会导致应用程序性能下降，甚至出现停顿现象，严重影响用户体验。因此，及时定位和解决这个问题至关重要。

第一步，确认GC问题的存在。

- 通过监控工具或日志分析，观察GC的频率和持续时间是否异常。
- 注意应用程序的性能是否出现明显下降，如响应时间延长、吞吐量降低等。

第二步，收集诊断信息。

- 开启并收集JVM的GC日志。这些日志会详细记录每次GC发生的时间、类型、持续时间和回收的对象数量等信息。
- 使用jstat、jmap等工具获取JVM的运行时状态，包括堆内存的使用情况、各个内存池的大小和使用率等。

第三步，**分析诊断信息**。

- 通过分析GC日志，找出GC频繁发生的原因。可能是堆内存不足、内存泄漏、代码中存在大量临时对象的创建等。
- 使用jmap等工具生成的堆转储文件（heap dump）来分析内存中的对象分布，找出占用内存大的对象或疑似内存泄漏的对象。

第四步，**调整和优化**。

- 根据分析结果，可以尝试调整JVM的参数配置，如堆大小、新生代与老年代的比例等，以优化GC性能。
- 检查代码中是否存在不必要的对象创建或长时间持有的对象，导致内存占用过高。如果有，考虑优化代码逻辑，减少内存消耗。

推荐学习资料

《JavaGuide》| JVM: [# 最重要的JVM参数总结](#)

《JavaGuide》| JVM: [# JDK监控和故障处理工具总结](#)

《JavaGuide》| JVM: [# JVM线上问题排查和性能调优案例](#)

JAVA: SSM框架

44. 谈谈你对Spring IOC的理解?

解析

控制反转，依赖注入

参考口述回答

Spring的IOC，也就是控制反转，是Spring框架的核心特性之一。要理解IOC，我们首先需要了解两个概念：控制反转和依赖注入。

控制反转，即Inversion of Control，简称IoC。它的主要思想是将原本由代码直接操控的对象的调用权交给第三方，比如一个容器来控制，以解耦代码。在传统的程序设计中，我们直接在对象内部通过new关键字来创建依赖的对象，这种方式会导致代码之间高度耦合，不利于测试和维护。而IoC的思想是，将这些对象创建和绑定的控制权交由Spring容器来管理，从而降低系统的耦合性。

具体来说，在Spring中实现IoC主要有两种方式：基于XML的配置和基于注解的配置。在XML配置中，我们可以在Spring的配置文件中，通过bean元素来定义需要Spring管理的对象，并通过property元素来注入依赖。而在注解配置中，我们可以使用如@Autowired、@Resource等注解来自动注入依赖。

依赖注入，即Dependency Injection，简称DI，是实现IoC的一种方式。它指的是通过外部的配置或者设定来将依赖关系注入到对象中。在Spring中，这通常是通过配置文件或者注解来实现的。这样，对象之间的依赖

关系就由Spring容器来管理，而不是由对象自身来管理。

Spring IoC容器负责管理Bean的生命周期、依赖注入等。我们只需要配置好相关Bean，就可以轻松实现对象之间的解耦和依赖管理。总的来说，IoC的思想和Spring的IoC容器极大地提高了代码的灵活性和可维护性，降低了系统的复杂度。

另外，Spring IoC还提供了很多高级特性，如Bean的作用域控制、生命周期回调、事件传播等，这些特性可以帮助我们更好地管理和控制Bean的行为。

总的来说，我对Spring IoC的理解是，它是一种强大的解耦和管理对象依赖关系的工具，通过把对象的创建和依赖关系的控制权交给Spring容器来管理，从而提高了代码的灵活性和可维护性。

推荐学习资料

《JavaGuide》| 常用框架：[#Spring IOC](#)

45. 谈谈你对 Spring AOP的理解？

解析

作用和优势；相关概念；实现方式；适用场景

参考口述回答

首先，AOP是Aspect Oriented Programming的缩写，意为面向切面编程。它是Spring框架中的一个重要内容，也是OOP（面向对象编程）的一种延续。与OOP关注对象不同，AOP关注的是在程序运行中的某个切入点进行特定的操作，比如日志记录、事务管理等。

关于Spring AOP，我认为可以从以下几个方面来理解：

一、AOP的作用和优势

AOP的主要作用是在不修改源代码的情况下，对方法进行功能增强。这使得我们可以在不改变原有业务逻辑的基础上，动态地添加一些额外的功能，如日志记录、性能监控等。其优势主要体现在减少重复代码，提高开发效率，同时便于后期维护。

二、AOP的关键概念

在AOP中，有几个关键的概念需要理解。首先是连接点（Joinpoint），它指的是程序执行过程中的某个明确点，如方法的调用或异常的抛出。其次是目标（Target），即被通知或被代理的对象，它完成具体的业务逻辑。通知（Advice）是在某个特定的连接点上执行的动作，它包含了具体的程序代码实现。代理（Proxy）是将通知应用到目标对象后创建的对象，它结合了目标和通知的功能。最后是切入点（Pointcut），它定义了通知应该应用到哪些连接点。

三、AOP的实现方式

在Spring中，AOP的实现主要依赖于动态代理技术。Spring提供了两种动态代理技术：JDK代理和CGLIB代理。JDK代理是基于接口的动态代理技术，它要求目标对象必须实现一个接口。而CGLIB代理是基于父类的动态代理技术，它可以对没有实现接口的目标对象进行代理。在运行时，Spring会根据目标对象的情况自动选择使用哪种代理技术。

四、AOP的适用场景

Spring AOP广泛应用于日志记录、事务管理、安全性检查、性能监控、异常处理、缓存管理以及参数校验等场景。这些场景中的功能都可以通过AOP在不修改原有业务逻辑的情况下动态添加进来。

总的来说，Spring AOP提供了一种灵活且强大的方式来增强程序的功能，同时降低了代码的耦合度，提高了程序的可重用性和开发效率。在实际开发中，我们应该根据具体的业务需求和系统架构来合理利用Spring AOP的功能。

推荐学习资料

《JavaGuide》| 常用框架： [#Spring AOP](#)

46. 讲一讲Spring的循环依赖？

解析

作用和优势；相关概念；实现方式；适用场景

参考口述回答

首先，循环依赖指的是在Spring框架中，两个或多个Bean之间相互依赖，形成一个环状依赖的情况。例如，Bean A依赖于Bean B，而Bean B又依赖于Bean A，这样就形成了一个循环依赖。这种情况在复杂的系统中比较容易出现，特别是当模块之间的耦合度较高时。

关于Spring循环依赖，可以从以下几个方面进行理解：

一、循环依赖的类型

循环依赖主要有三种类型：构造函数注入的循环依赖、Setter注入的循环依赖和Singleton Bean之间的循环依赖。其中，构造函数注入的循环依赖通常会导致问题，因为在实例化Bean时需要先满足其依赖关系，而如果存在循环依赖，就会导致无法满足依赖关系，从而引发异常。

二、Spring对循环依赖的处理

Spring容器通过其依赖注入机制来处理循环依赖问题。对于Singleton Bean，Spring容器会先实例化一个Bean，但并不立即进行依赖注入。当发现循环依赖时，Spring会利用一个三级缓存机制来解决这个问题。首先，Spring会将正在创建的Bean放入“当前创建Bean池”中，如果发现该Bean已经在池中，则表示存在循环依赖。此时，Spring会从三级缓存中获取一个早期的Bean引用（也称为原始类型的Bean），并将其注入到依赖它的其他Bean中，从而打破循环依赖。

然而，对于构造函数注入的循环依赖，Spring无法解决，因为这需要在实例化Bean之前满足所有的依赖关系。因此，如果存在构造函数注入的循环依赖，项目在启动时会报错。

三、如何避免循环依赖

为了避免循环依赖，可以采取以下措施：首先，尽量使用Setter注入而不是构造函数注入，因为Setter注入允许在Bean实例化后再设置依赖关系，从而更容易解决循环依赖问题。其次，合理设计系统架构和模块之间的依赖关系，降低模块之间的耦合度。例如，可以使用接口或抽象类来定义模块之间的交互方式，从而减少直接依赖。最后，利用Spring的@Lazy注解来延迟Bean的初始化，这样也可以在一定程度上避免循环依赖问题的发生。

推荐学习资料

47. Bean生命周期?

解析

实例化阶段；属性赋值阶段；初始化阶段；销毁阶段

参考口述回答

Spring Bean的生命周期大体上可以分为四个阶段：实例化、属性赋值、初始化和销毁。

一、实例化阶段：

这个阶段主要是通过反射技术创建Bean的实例。Spring会根据配置文件中Bean的定义，利用Java Reflection技术创建对应的实例。

二、属性赋值阶段：

实例化后的对象被封装在BeanWrapper对象中，此时对象仍然是一个原生状态，并没有进行依赖注入。Spring根据BeanDefinition中的信息进行依赖注入，通过populateBean方法完成属性的注入。此外，如果Bean实现了xxxAware接口，Spring会调用相应的set方法注入相应的内容，如BeanNameAware、BeanClassLoaderAware、BeanFactoryAware和ApplicationContextAware等。

三、初始化阶段：

初始化阶段是Bean生命周期中非常关键的一个环节，它包含了多个扩展点。

- 1. 执行Aware接口的方法：**如果Bean实现了相应的Aware接口，Spring会调用这些接口定义的方法，如setBeanName、setBeanClassLoader、setBeanFactory和setApplicationContext等。
- 2. 执行BeanPostProcessor的前置处理：**如果Bean实现了BeanPostProcessor接口，Spring会在初始化方法调用前，先调用其postProcessBeforeInitialization方法，这是用户自定义处理逻辑的一个重要扩展点。
- 3. 执行InitializingBean接口的方法：**如果Bean实现了InitializingBean接口，Spring会调用其afterPropertiesSet方法，用户可以在这个方法中进行一些初始化操作。
- 4. 执行自定义的init-method：**如果在Spring的配置文件中为Bean指定了init-method属性，Spring会自动调用这个指定的初始化方法。
- 5. 执行BeanPostProcessor的后置处理：**在初始化方法调用后，如果Bean实现了BeanPostProcessor接口，Spring还会调用其postProcessAfterInitialization方法，这是另一个用户自定义处理逻辑的扩展点。

四、销毁阶段：

当Bean不再需要时，如果Bean实现了DisposableBean接口，Spring会调用其destroy方法进行销毁前的清理工作。同时，如果在配置文件中指定了destroy-method属性，Spring也会在销毁前调用这个方法。

综上所述，Spring Bean的生命周期包括实例化、属性赋值、初始化和销毁四个阶段，其中每个阶段都提供了不同的扩展点，允许用户自定义处理逻辑。这些扩展点增强了Spring框架的灵活性和可扩展性，使得开发者能够更加灵活地控制Bean的生命周期行为。

推荐学习资料

48. 讲一下SpringBoot的自动装配原理？

解析

1. @SpringBootApplication注解；
2. @EnableAutoConfiguration注解；
3. AutoConfigurationImportSelector类；
4. SpringFactoriesLoader工具类

参考口述回答

首先，我们要明确什么是Spring Boot的自动装配。自动装配是指Spring Boot通过分析项目的依赖和配置，自动配置Spring应用程序所需的组件，而开发者无需手动配置大量的XML配置文件或Java代码。这大大简化了Spring应用的开发和配置过程。

那么，Spring Boot是如何实现自动装配的呢？这主要得益于以下几个关键部分：

1. **@SpringBootApplication注解**：这是启动Spring Boot应用的核心注解，它实际上是一个复合注解，包括了@Configuration、@EnableAutoConfiguration和@ComponentScan等。其中，@EnableAutoConfiguration是实现自动装配的关键。
2. **@EnableAutoConfiguration注解**：这个注解告诉Spring Boot基于你所添加的jar依赖项自动配置你的Spring应用。例如，如果你在classpath下有HSQLDB，那么Spring Boot就会自动配置一个内存数据库。这个注解内部使用了@Import(AutoConfigurationImportSelector.class)，这是实现自动装配的核心类。
3. **AutoConfigurationImportSelector类**：这个类负责加载META-INF/spring.factories文件中配置的自动装配类。Spring Boot在启动时，会实例化这个类，并加载配置文件中指定的自动装配类。这些类通常包含了一些Bean的定义和配置。
4. **条件装配**：Spring Boot的自动装配还支持基于条件的装配。这意味着只有在某些条件满足时，特定的自动装配才会发生。这是通过@Conditional注解实现的。例如，如果classpath下有某个特定的类库，那么与之相关的自动装配就会被触发。
5. **SpringFactoriesLoader**：这是一个工具类，用于加载META-INF/spring.factories文件中的配置。这个文件包含了各种自动装配类的全限定名，供AutoConfigurationImportSelector类加载和使用。

综上所述，Spring Boot的自动装配原理可以归纳为以下几个步骤：

- 通过@SpringBootApplication注解启动Spring Boot应用；
- @EnableAutoConfiguration注解触发自动装配机制；
- AutoConfigurationImportSelector类加载META-INF/spring.factories中配置的自动装配类；
- 根据条件装配的规则，决定是否进行特定的自动装配；
- 最后，通过SpringFactoriesLoader工具类加载并实例化自动装配类中的Bean。

这就是Spring Boot自动装配的基本原理。通过这种方式，Spring Boot能够极大地简化Spring应用的开发和配置过程，提高开发效率。

推荐学习资料

《JavaGuide》| 常用框架：[#SpringBoot 自动装配原理详解](#)

49. 讲一下SpringMVC的处理流程？

解析

整个SpringMVC的处理流程可以概括为：用户请求->DispatcherServlet捕获并解析请求->HandlerMapping查找Handler->HandlerAdapter执行Handler->返回ModelAndView->ViewResolver解析视图->View渲染并返回响应页面

参考口述回答

一、用户请求与前端控制器的接收

1. 当用户发送一个请求时，这个请求首先会被SpringMVC的前端控制器DispatcherServlet捕获。
2. DispatcherServlet会对请求的URL进行解析，获取请求的URI，并根据这个URI来决定后续的处理流程。

二、处理器映射与处理器执行链的获取

1. DispatcherServlet会将解析得到的URI发送给HandlerMapping（处理器映射器），由其找到对应的Handler（控制器中的具体方法）以及相关的拦截器，并将它们封装成一个HandlerExecutionChain（处理器执行链）返回。
2. 这个过程中，HandlerMapping会根据请求的URL和HTTP方法（如GET、POST等）来查找并匹配对应的Handler。

三、处理器适配与执行

1. DispatcherServlet在获取到HandlerExecutionChain后，会选择一个合适的HandlerAdapter（处理器适配器）来执行这个链中的Handler。
2. HandlerAdapter会提取Request中的模型数据，填充Handler的入参，并开始执行Handler。在这个过程中，SpringMVC还会进行一系列的数据转换、数据格式化和数据验证等工作。

四、处理器执行结果与模型视图的返回

1. Handler执行完毕后，会返回一个ModelAndView对象给HandlerAdapter。这个对象包含了处理的结果数据（模型）以及要展示的视图名称或视图对象。
2. HandlerAdapter接收到ModelAndView后，会将其返回给DispatcherServlet。

五、视图解析与渲染

1. DispatcherServlet在接收到ModelAndView后，会请求对应的ViewResolver（视图解析器）进行解析。
2. ViewResolver会根据ModelAndView中指定的视图名称找到对应的View（视图）对象，并返回给DispatcherServlet。
3. 最后，由View对象负责将模型数据渲染到视图中，生成最终的响应页面，并返回给客户端用户。

推荐学习资料

《JavaGuide》| 常用框架：[#SpringMVC 工作原理了解吗？](#)

操作系统：进程管理

50. 进程，线程，协程的区别

解析：

进程、线程和协程在资源分配、调度、通信和开销等方面存在显著的差异，可以从这几个方面进行回答

**参考口述回答

首先，我们来谈谈**进程**。进程是操作系统中进行资源分配和调度的基本单位，它拥有自己的独立内存空间和系统资源。每个进程都有独立的堆和栈，不与其他进程共享。进程间通信需要通过特定的机制，如管道、消息队列、信号量等。由于进程拥有独立的内存空间，因此其稳定性和安全性相对较高，但同时上下文切换的开销也较大，因为需要保存和恢复整个进程的状态。

接下来是**线程**。线程是进程内的一个执行单元，也是CPU调度和分派的基本单位。与进程不同，线程共享进程的内存空间，包括堆和全局变量。线程之间通信更加高效，因为它们可以直接读写共享内存。线程的上下文切换开销较小，因为只需要保存和恢复线程的上下文，而不是整个进程的状态。然而，由于多个线程共享内存空间，因此存在数据竞争和线程安全的问题，需要通过同步和互斥机制来解决。

最后是**协程**。协程是一种用户态的轻量级线程，其调度完全由用户程序控制，而不需要内核的参与。协程拥有自己的寄存器上下文和栈，但与其他协程共享堆内存。协程的切换开销非常小，因为只需要保存和恢复协程的上下文，而无需进行内核级的上下文切换。这使得协程在处理大量并发任务时具有非常高的效率。然而，协程需要程序员显式地进行调度和管理，相对于线程和进程来说，其编程模型更为复杂。

推荐学习资料：

《博客》 | 腾讯云开发者社区: [#一文读懂什么是进程、线程、协程](#)

《小林Coding》 | 图解系统: [# 进程、线程基础知识](#)

51. 进程上下文切换和线程上下文切换？操作系统都做了什么？

解析：

进程上下文切换和线程上下文切换在切换内容和发生场景上有所不同。进程上下文切换涉及的内容更多、开销更大，主要发生在进程调用系统调用或时间片用完等场景；而线程上下文切换主要涉及线程的执行状态和条件，发生在线程主动让出CPU、时间片用完或线程阻塞等场景

参考口述回答：

首先是**进程**上下文切换。在**切换内容**方面，进程上下文切换涉及的内容较为广泛。由于进程是操作系统中进行资源分配和调度的基本单位，它拥有自己的独立内存空间和系统资源，因此进程上下文切换不仅包含了虚拟内存、栈、全局变量等用户空间的资源，还包括了内核堆栈、寄存器等内核空间的资源。这些资源在切换时都需要被保存和恢复，以确保新进程能够在切换后顺利执行。

在**发生场景**上，进程上下文切换主要发生在以下几种情况：一是进程主动调用某些系统调用时，如等待IO完成或者获取锁，这时进程无法继续运行，操作系统会触发上下文切换；二是进程分配到的时间片用完，或者有更高优先级的进程需要抢占CPU时，也会发生上下文切换。

接下来是**线程**上下文切换。在**切换内容**方面，线程上下文切换主要涉及线程在执行过程中的运行条件和状态，如程序计数器、栈信息、寄存器的值等。由于线程共享进程的内存空间，因此线程上下文切换不需要像进程上下文切换那样涉及大量的内存和资源管理。

线程上下文切换**发生场景**主要包括：线程主动让出CPU，例如调用了Thread.sleep()或Object.wait()等方法；当一个线程的时间片用完，需要切换到另一个线程继续执行；或者线程因为阻塞或等待某个事件而无法继续执行时，调度器会切换到另一个线程继续执行。

推荐学习资料：

《小林Coding》 | 图解系统：[# 进程的上下文切换](#)

《小林Coding》 | 图解系统：[# 线程的上下文切换](#)

52. 进程间通信方式？

解析：

管道，消息队列，共享内存，信号量，套接字，信号

参考口述回答：

首先，管道（Pipe）是一种常见的进程间通信方式，它允许一个进程的输出作为另一个进程的输入。管道是半双工的，数据只能单向流动，且通常用于具有亲缘关系的进程之间，如父子进程。此外，还有命名管道（FIFO），它与管道类似，但允许无亲缘关系的进程间进行通信。

其次，消息队列（Message Queue）是另一种重要的IPC机制。进程可以将消息发送到消息队列，其他进程则可以从队列中检索消息。这种方式克服了管道的一些限制，如只能承载无格式的字节流以及缓冲区大小受限等。消息队列允许进程之间发送和接收具有特定格式的消息，且可以异步地进行通信。

共享内存（Shared Memory）也是一种高效的进程间通信方式。多个进程可以访问同一块内存区域，从而直接读写共享的数据。这种方式速度非常快，因为数据不需要在不同进程之间复制。然而，它也需要更复杂的同步机制来防止数据冲突和不一致性。

信号量（Semaphore）则是一种用于控制多个进程对共享资源的访问的同步机制。它可以被视为一个计数器，用于实现进程间的互斥和同步操作。信号量常用于保护对共享内存或其他资源的访问，以防止发生竞态条件。

套接字（Socket）通信则是一种更为通用的进程间通信方式，它不仅适用于同一台计算机上的进程间通信，还适用于网络中的不同计算机上的进程间通信。套接字提供了一种标准的接口来发送和接收数据，支持多种协议（如TCP、UDP等），并具有跨平台和可靠性高的特点。

最后，信号（Signal）也是一种进程间通信的方式，但它主要用于通知接收进程某个事件已经发生。信号是一种软件中断，可以由操作系统或其他进程发送。接收进程在收到信号后，可以根据信号的类型执行相应的操作。

推荐学习资料：

《小林Coding》 | 图解系统：[# 进程间有哪些通信方式？](#)

53. 管道的原理是什么？有名管道和匿名管道有啥区别？

解析：

参考口述回答：

管道（Pipe）是一种特殊的文件类型，具有读写两个端口，数据从一个端口流入，从另一个端口流出。管道的原理是基于内核缓冲区实现的，其本质是一个伪文件。当进程向管道写入数据时，数据首先被存储在内核缓冲区中，然后其他进程可以从该缓冲区读取数据，实现了进程间的数据共享和传输。

有名管道（Named Pipe）和匿名管道（Anonymous Pipe）之间的主要区别在于它们的存在范围和使用方式。

有名管道也被称为FIFO，它在文件系统中具有一个路径名，并且可以被多个进程同时打开和使用。这意味着不相关的进程可以通过读写这个共享的文件来交换数据。有名管道是基于磁盘上实际的文件进行操作的，因此即使进程退出，管道依然存在。此外，多个进程可以同时向管道写入数据或从管道读取数据，这种特性使得有名管道成为进程间通信的重要手段之一。它常用于解决在异步场景下进程间通信的问题。

相比之下，**匿名管道**是单向的，只能在具有亲缘关系的进程间使用，例如父子进程。它没有磁盘上对应的文件，只存在于内核中，因此读写速度非常快。匿名管道是基于内存进行操作的，主要用于实现进程间的同步和互斥，避免竞争条件和死锁等问题。当数据被写入匿名管道时，它会被存储在内核的缓冲区中，等待其他进程来读取。如果没有数据可读或管道已满，则相应的读写进程会被阻塞，直到有数据可读或管道中有空间可写为止。

推荐学习资料：

《小林Coding》 | 图解系统：[# 管道](#)

54. 信号和信号量的区别？

解析：

信号主要用于异步事件通知，而信号量则主要用于同步和互斥操作，以确保对共享资源的正确访问

参考口述回答：

信号（Signal）是一种处理异步事件的方式，用于通知接收进程有某种事件发生。信号是比较复杂的通信方式，它不仅可以用于进程间的通信，还可以发送给进程本身。在Unix和Linux系统中，信号被广泛应用于处理各种异步事件，如中断、异常或用户自定义的通知。信号是一种软件中断，允许一个进程向另一个进程或自身发送简短消息，通知某些事件发生，例如，一个程序可能因用户按下Ctrl+C而接收到SIGINT信号，表示需要中断执行。

信号量（Semaphore）则是一种进程间通信处理同步互斥的机制，常用于多线程或多进程环境中。信号量负责协调各个线程或进程，以确保它们能够正确、合理地使用公共资源或关键代码段。信号量通常被用作一个计数器，用于控制对共享资源或临界区域的访问。它提供了两种基本操作：Wait（等待，有时也称为P操作）和

Release（释放，有时也称为V操作）。当某个线程或进程需要使用共享资源时，它会执行Wait操作，这将减少信号量的值；当资源使用完毕后，它会执行Release操作，增加信号量的值。通过这种方式，信号量能够确保在同一时刻只有一个线程或进程能够访问特定资源，从而防止资源冲突和数据不一致。

推荐学习资料：

《小林Coding》 | 图解系统：[# 信号](#)

《小林Coding》 | 图解系统：[# 信号量](#)

55. 死锁怎么产生的？怎么避免死锁？

解析：

死锁必要条件：互斥条件；持有并等待条件；不可剥夺条件；环路等待条件； 避免死锁：打破其中一个条件

参考口述回答：

产生死锁：

死锁，是指在多进程或多线程系统中，两个或多个进程或线程因争夺系统资源而陷入无限等待的状态，无法继续执行下去。产生死锁必须同时满足以下四个条件：

首先是互斥条件。这意味着至少有一个资源必须处于非共享状态，即一次只能被一个进程或线程占用。如果其他进程或线程试图访问该资源，它们将被阻塞，直到资源被释放。

其次是持有并等待条件。这指的是进程或线程在持有至少一个资源的同时，还在等待其他资源。这个过程中，进程或线程并不会放弃自己已经持有的资源，这就可能导致资源的占用和等待形成僵持状态。

第三个是不可剥夺条件。它表明已分配给进程或线程的资源不能被强制性地剥夺，只能由持有资源的进程或线程主动释放。这意味着即使其他进程或线程急需该资源，也无法从当前持有者那里强制取回。

最后是循环等待条件。存在一个进程或线程的资源申请序列，使得每个进程或线程都在等待下一个进程或线程所持有的资源。这形成了一个闭环，其中每个参与者都在等待链中的下一个参与者释放资源，从而导致所有参与者都无法继续执行。

当这四个条件同时满足时，就会发生死锁。

避免死锁：

为了避免死锁的发生，我们可以采取一些策略来破坏这些条件中的至少一个。例如，我们可以规定进程或线程在申请资源时必须一次性申请所有需要的资源，从而破坏持有并等待条件；或者我们可以实现资源的超时机制，使得资源在一段时间后可以被强制剥夺，从而破坏不可剥夺条件；另外，我们还可以对资源进行编号，并规定进程或线程必须按照编号的顺序申请资源，这样可以破坏循环等待条件。通过这些方法，我们可以有效地预防和避免死锁的发生。

推荐学习资料：

《小林Coding》 | 图解系统：[# 死锁](#) | [怎么避免死锁](#)

操作系统：系统结构

56. 内核态和用户态的区别，为什么要划分？

解析：

划分内核态和用户态是为了提高系统的安全性、稳定性和性能。这种划分使得操作系统能够更好地管理和保护系统资源，同时提供了一个有效的机制来平衡系统的性能和安全性

参考口述回答：

内核态和用户态是操作系统中的两种运行级别，它们之间有着明显的区别。简单来说，内核态是操作系统内核运行的模式，具有极高的权限，可以访问所有硬件资源和执行所有指令，包括特权指令。而用户态则是用户程序运行的模式，其权限受到限制，只能执行非特权指令，且不能直接访问操作系统内核数据结构和程序。

这两种状态的主要差别在于，处于用户态执行的程序不能直接访问操作系统内核数据结构和程序，其所处于占有的处理器是可被抢占的，且其能访问的内存空间和对象受到限制。而内核态下运行的程序则能访问所有的内存空间和对象，且所占有的处理器不允许被抢占。

那么，为什么要划分内核态和用户态呢？这主要是出于以下几个原因：

1. 安全性：通过划分内核态和用户态，可以防止用户程序对系统造成不可修复的破坏。因为用户程序在用户态下运行，其操作受到严格的限制，不能直接操作关键系统资源，从而保护了系统的安全性。
2. 稳定性：如果用户程序崩溃或出现错误，由于它运行在用户态，因此不会影响整个系统的稳定性。内核态下运行的操作系统可以继续正常工作，而不受用户程序的干扰。
3. 性能：虽然内核态和用户态之间的切换会耗费一定的性能资源，但这种划分也带来了性能上的优势。因为内核态具有更高的权限，可以执行一些需要较高特权级别的操作，如直接访问硬件，这有助于提高系统的性能。同时，通过将关键操作限制在内核态，可以确保这些操作的高效和准确执行。

推荐学习资料：

《技术文章摘抄》：[用户态和内核态：用户态线程和内核态线程有什么区别？](#)

《视频资料》：[#用户态和内核态](#)

57. 用户态陷入系统调用，切换到内核态的过程是怎样的？哪些命令和函数会陷入系统调用？

解析：

用户态陷入系统调用并切换到内核态的过程是一个由用户态程序主动发起、通过软中断触发、由操作系统内核接管并执行相应操作的过程

参考口述回答：

切换过程

当用户态陷入系统调用，切换到内核态的过程，首先是由用户态的进程主动发起一个系统调用请求。这个请求通常是由于用户态的程序需要执行一些只有操作系统内核才能完成的敏感或关键操作，比如访问硬件资源、进行进程间通信等。

在发起系统调用时，CPU会通过一个特殊的指令，比如x86架构中的`int`指令或者`syscall`指令，来触发一个软中断。这个软中断会导致CPU暂停当前用户态程序的执行，转而跳转到预先设定好的中断处理程序中去执行。

中断处理程序是操作系统内核的一部分，它在内核态中运行。当中断处理程序开始执行时，就意味着操作系统已经从用户态切换到了内核态。在内核态中，操作系统可以访问所有的硬件资源和系统数据，拥有最高的权限。

哪些命令和函数会陷入系统调用？

关于哪些命令和函数会陷入系统调用，实际上在Linux等类Unix系统中，很多常见的命令和函数最终都会陷入系统调用。比如，当我们使用`open`函数打开一个文件时，这个函数内部最终会调用系统提供的`open`系统调用来完成实际的文件打开操作。同样地，像`read`、`write`、`close`等文件操作函数，以及`socket`、`bind`、`listen`、`accept`等网络编程相关的函数，它们底层都会陷入相应的系统调用来完成实际的工作。

推荐学习资料：

《博客园》FynnWang: [#用户态到内核态的切换](#)

操作系统：内存管理

58. 讲讲虚拟内存是什么？为什么要有虚拟内存？操作系统是如何管理虚拟内存的？

解析：

管理：分段，分页，多级页表，TLB

参考口述回答：

1. 虚拟内存

虚拟内存是计算机系统内存管理的一种重要技术。简单来说，它使得应用程序认为自己拥有连续可用的内存空间，而实际上，这部分内存可能被分隔成多个物理内存碎片，甚至有部分数据被暂时存储在外部磁盘存储器上。当程序需要这些数据时，操作系统会将其从磁盘中读取到物理内存中。

2. 为什么要有虚拟内存？

这种技术为我们提供了很多便利和优势。

首先，虚拟内存扩展了可用的内存容量。通过结合物理内存和硬盘空间，它允许大型应用程序运行，并为多任务操作系统提供了更好的支持。即使物理内存不足，系统也可以通过将未使用的数据移动到磁盘上来继续运行程序。

其次，虚拟内存提供了内存隔离，增强了数据的安全性和系统的稳定性。每个进程都有自己的虚拟地址空间，这样一个进程就无法读取或修改另一个进程的数据。这种隔离保护了操作系统和应用程序免受潜在的安全威胁。

此外，虚拟内存还减少了程序之间的冲突。由于每个程序都在自己的地址空间中运行，因此不同的程序之间不会产生地址冲突。如果两个程序试图使用相同的内存地址，操作系统会检测到这种情况并防止数据损坏。

虚拟内存还通过页面置换技术提高了I/O效率。当物理内存中的某些页面不再活跃时，它们会被移动到硬盘上，从而释放出物理内存空间给更活跃的应用程序使用。这不仅减少了I/O操作的次数，还降低了硬盘访问时间，从而提高了整体的系统性能。

最后，虚拟内存技术还简化了内存管理，减轻了程序员的负担。程序员无需关心实际的物理内存布局和管理细节，可以更加专注于应用程序的开发和实现。

3. 操作系统如何管理虚拟内存的？

虚拟内存管理是现代操作系统中非常重要的一部分，它允许程序使用比实际物理内存更大的内存空间。这种技术主要涉及到分段、分页、多级页表以及TLB（Translation Lookaside Buffer）等关键概念。

首先，我们来谈谈分段。在分段机制下，虚拟地址由两部分组成：段选择子和段内偏移量。段选择子用于指定要访问的段的起始地址和长度，而段内偏移量则表示在该段内的具体位置。操作系统会维护一个段表，记录了每个段的起始地址和长度等信息。当程序访问一个虚拟地址时，操作系统会通过段选择子从段表中找到对应的段描述符，进而计算出物理地址。

接下来是分页。分页是另一种重要的虚拟内存管理技术。操作系统会将物理内存和磁盘等辅助存储器中的空间划分为固定大小的区块，称为“页”。当程序需要更多的内存空间时，操作系统会将部分数据从物理内存中移到磁盘等辅助存储器上，而将当前需要的数据加载到物理内存中。这种机制使得程序可以透明地使用更大的内存空间，而无需关心实际物理内存的大小。

然而，随着程序规模的扩大，页表也会变得非常庞大，这导致了查找效率的问题。为了解决这个问题，我们引入了多级页表的概念。多级页表将页表进行分级，通过分级索引的方式来定位具体的页框号。这种方式可以有效地减少页表所占用的空间，并提高查找效率。

最后，我们来谈谈TLB（Translation Lookaside Buffer）。TLB是一种高速缓存结构，用于加速虚拟地址到物理地址的转换过程。它保存了最近使用的虚拟地址和物理地址之间的映射关系。当CPU发出虚拟地址时，TLB会首先检查其中是否包含所需的物理地址。如果命中，则直接返回对应的物理地址；如果未命中，则需要访问操作系统的页表以获取映射关系，并将这些信息加载到TLB中以便下次快速访问。通过这种方式，TLB可以显著提高内存访问的性能。

综上所述，虚拟内存管理通过分段、分页、多级页表和TLB等技术实现了对物理内存的抽象和扩展，为程序提供了更大的内存空间并提高了内存访问的性能。

推荐学习资料：

《小林Coding》 | 图解系统：[为什么要有虚拟内存？](#)

《小林Coding》 | 图解系统：[虚拟内存管理总结](#)

更多优质题目尽在牛面Offer小程序：

牛面 一站式学习与面试

2000+热门题库免费刷！题目全栈覆盖，助你拿offer！



海量题库刷不停，
微信扫码立即开始！



趣味闯关

告别死记硬背，闯关练习陪你
轻松进步，每一关都在悄悄变强。

1000+精品关卡等你来战

技术题库

春招、秋招最热题型全覆盖
多名大厂老鸟精心把关筛选

2k+道命中率超高必刷题

简历押题

大厂标准提取核心问题
问题-思路-答案三位一体

个性化精准预测

模拟面试

专项突破，垂直领域靶向强化
简历深挖，六大维度全面拷问

真实场景完整演练

牛面题库 让面试准备更高效 → niumianoffer.com



学习过程中有任何问题，都欢迎联系牛面小助手，不定时会发放福利哦：



扫码添加 牛面小助手

59. IO多路复用? select,poll,epoll的区别?

解析:

参考口述回答:

1. IO多路复用?

IO多路复用是一种高效的IO处理方式，它允许单个进程或线程同时监视多个文件描述符，如网络连接或文件句柄。当这些描述符中的任何一个就绪时，比如有数据可读或可写，多路复用机制就能够通知应用程序进行相应的读写操作。这种机制的核心优势在于，它可以在不增加额外线程或进程的情况下，处理大量的并发连接，从而显著地提高系统的并发性和响应能力。

常见的IO多路复用技术包括select、poll和epoll等。这些技术各有特点，但核心思想都是通过一个线程来管理多个连接，减少系统资源的消耗，并提高程序运行的效率。

2. select,poll,epoll的区别

select是最早的一种I/O多路复用技术。它使用一个fd_set数据结构来表示所有被监视的文件描述符。然而，select有几个明显的缺点。首先，它监视的文件描述符数量有限，默认上限是1024。其次，每次调用select时，都需要将fd_set从用户态拷贝到内核态，并且在内核中需要遍历所有被监视的文件描述符来检查是否有就绪的，这个开销在文件描述符数量很多时会变得非常大。最后，select返回后，还需要再次遍历fd_set来找出哪些文件描述符是就绪的，这进一步增加了处理开销。

poll则是select的一种改进方案。它使用一个pollfd结构来表示被监视的文件描述符及其事件。与select相比，poll没有文件描述符数量的限制，因为它基于链表来存储。然而，poll仍然需要在每次调用时从用户态拷贝所有的pollfd结构到内核态，并在内核中遍历检查每个文件描述符的状态。因此，当文件描述符数量很多时，poll的性能也会下降。

epoll则是Linux特有的I/O多路复用技术，它在很多方面都优于select和poll。首先，epoll只在初始时完成一次文件描述符的注册，避免了每次调用时的拷贝开销。其次，epoll采用回调函数的方式，只有当一个或多个文件描述符就绪时，才会调用回调函数并通知用户空间，这使得epoll在处理大量文件描述符时仍然能保持高效。最后，epoll返回时已经明确指出了哪些文件描述符是就绪的，因此无需再像select和poll那样进行额外的遍历操作。

推荐学习资料:

《小林Coding》 | 图解系统: [IO多路复用](#)

3. epoll的具体实现? epoll的事件触发有几种方式?

解析:

参考口述回答:

1. epoll的实现?

epoll通过一种高效的数据结构来管理大量的文件描述符，这个数据结构就是**红黑树**。红黑树是一种自平衡的二叉搜索树，它在添加、删除和搜索操作中都能保持相对平衡，从而保证这些操作的时间复杂度接近 $O(\log n)$ 。在epoll中，每个文件描述符作为红黑树的一个节点被存储，这样可以高效地管理和检索大量的文件描述符。

epoll的另一个关键特性是其事件驱动的回调机制。当注册的文件描述符上发生特定事件（如数据可读、可写或异常）时，内核不会立即通知应用程序，而是将这些事件添加到**就绪链表**中。这个链表包含了所有已经就绪（即有事件发生）的文件描述符。当应用程序调用`epoll_wait()`函数时，内核会检查就绪链表，并返回其中已经就绪的文件描述符列表，从而实现了事件驱动的回调通知。

此外，epoll还使用了**内存映射技术**来减少不必要的内存拷贝。通过将用户空间和内核空间的部分内存映射到同一个物理内存页，epoll能够在内核空间和用户空间之间高效地传递事件信息，进一步降低了系统调用的开销。

总的来说，epoll通过结合红黑树、就绪链表和内存映射技术，实现了一种高效、可扩展且低开销的I/O多路复用机制，特别适用于需要处理大量并发连接的高性能服务器和网络应用

2. epoll的事件触发有几种方式？

epoll的事件触发主要有两种方式。第一种是边缘触发，也就是说，只有当文件描述符的状态从不就绪变为就绪时，epoll才会发出通知。一旦通知过后，除非状态再次发生变化，否则不会再次通知。第二种是水平触发，这种模式下，只要文件描述符处于就绪状态，epoll就会持续发出通知，直到数据处理完毕

推荐学习资料：

《小林Coding》 | 图解系统：[Epoll](#)

60. 零拷贝技术是什么？

解析：

减少数据在用户态和内核态之间的拷贝次数

参考口述回答：

零拷贝技术是一种高效的数据传输技术，它的主要目的是**减少数据在用户态和内核态之间的拷贝次数**，从而降低CPU和内存的开销，提高数据传输的效率。在传统的传输过程中，数据通常需要在用户空间和内核空间之间进行多次拷贝，这不仅消耗了大量的CPU资源，还增加了内存带宽的占用。而零拷贝技术则能够避免这些不必要的拷贝。

具体来说，零拷贝技术可以通过一些特殊的系统调用来实现，如`sendfile`、`mmap`以及`splice`等。这些系统调用允许数据在内核空间和用户空间之间直接传输，而无需经过中间的拷贝过程。例如，`sendfile`系统调用可以在网络传输中实现零拷贝，它允许数据直接从内核缓冲区发送到网络接口，避免了数据在用户空间和内核空间之间的额外拷贝。

除了网络传输，零拷贝技术在文件操作、数据库操作、图像处理以及消息队列等场景中也有广泛的应用。在这些场景中，零拷贝技术同样可以减少数据在应用程序和底层系统之间的拷贝次数，从而提高系统的性能和效率。

当然，零拷贝技术也并非没有缺点。例如，在某些情况下，由于需要直接操作内核空间的数据，因此可能会增加系统的复杂性和安全风险。此外，如果应用程序需要频繁地进行用户空间和内核空间之间的数据交换，那么零拷贝技术可能并不适合，因为它可能会引入额外的系统调用开销。

推荐学习资料：

《小林Coding》 | 图解系统：[什么是零拷贝？](#)

操作系统：Linux命令

61. Linux常用命令有哪些？

解析

查看资源；查看日志；查看网络

参考回答

查看系统资源

- top命令**：这是一个实时显示系统中各个进程的资源占用状况的监视器。它可以展示CPU使用率、内存使用率、进程状态等信息。通过top命令，用户可以快速识别哪些进程占用了过多的系统资源。
- ps命令**：用于显示当前系统中活动进程的信息。通过执行“ps -ef”或“ps aux”，用户可以查看所有进程的详细信息，包括进程ID、CPU和内存使用率等。此外，结合grep命令，可以方便地查找特定进程。
- free命令**：用于显示系统的内存使用情况。执行“free -m”可以以MB为单位显示内存使用情况，包括总内存、已用内存、空闲内存等信息。
- df命令**：用于显示磁盘分区上的可用和已使用的磁盘空间。执行“df -h”可以以人类可读的格式（如GB、MB）显示磁盘空间使用情况。
- sar命令**：这是一个系统活动报告工具，可以用于收集、报告和保存系统活动信息。通过sar命令，用户可以查看CPU使用率、内存使用率、I/O等历史数据。
- vmstat命令**：用于显示关于系统虚拟内存、进程、CPU活动等的统计信息。通过vmstat命令，用户可以实时监控系统的资源使用情况。

查看日志

- tail命令**：常用于查看实时变化的日志文件。例如，“tail -f /var/log/syslog”可以实时查看系统日志的最新内容。此外，“tail -n [行数] 文件名”可用于查看文件的最后几行。
- cat命令**：用于显示文件内容。结合grep命令，可以搜索关键字附近的日志内容。例如，“cat /var/log/syslog | grep '关键字’”可以搜索包含特定关键字的日志行。
- less命令**：这是一个强大的文本查看器，允许用户向前和向后翻页查看文件内容。对于大型日志文件，less命令提供了方便的导航和搜索功能。

4. **head命令**：与tail命令相反，head命令用于查看文件的开头部分。例如，“head -n [行数] 文件名”可以查看文件的前几行。
5. **more命令**：以全屏幕的方式按页显示文件内容，适合查看长文件。

查看网络状态

1. **ifconfig**：此命令用于显示和配置网络接口的详细信息，包括IP地址、MAC地址、网络掩码等。例如，ifconfig 可以列出所有活动网卡的信息。
2. **ip**：ip 命令是一个多功能的网络工具，用于显示或操作路由、网络设备、策略路由和隧道。例如，ip addr 可以显示网络接口的详细信息，类似于 ifconfig；ip route 可以显示路由表信息。
3. **netstat**：此命令用于显示网络连接、路由表、接口统计等网络相关信息。例如，netstat -an 可以显示所有活动的网络连接和监听的端口。
4. **ss**：ss 命令是一个比 netstat 更强大的工具，用于查看系统的socket统计信息。它可以提供更详细的信息，并且比 netstat 更快。例如，ss -tuln 可以列出所有TCP和UDP的监听端口。
5. **ping**：此命令用于测试网络的连通性，通过发送ICMP回显请求数据包到目标主机，并等待接收回显回复数据包来判断网络是否连通。例如，ping www.baidu.com 可以测试到Baidu的网络连通性。

推荐学习资料

《小林Coding》 | 公众号：[2万字系统总结，带你实现 Linux 命令自由](#)

62. Java程序线上CPU使用率100%如何排查？

解析

top->jstack->arthas

参考回答

当Java程序在线上环境中出现CPU使用率达到100%的情况时，需要系统地进行排查以确定问题所在。以下是一个清晰的排查流程：

1. 使用top命令查看进程CPU占用情况

- 首先，运行top命令，观察各个进程的CPU占用情况。
- 通过top命令，可以快速找到CPU占用率最高的Java进程。注意记录下该进程的PID（进程ID）。

2. 使用top -Hp命令查看线程CPU占用情况

- 确定高CPU占用的Java进程后，使用top -Hp [PID]命令查看该进程中各个线程的CPU占用情况。
- 这里的[PID]是之前通过top命令找到的Java进程的PID。
- 通过这个命令，我们可以找到具体是哪个线程占用了过多的CPU资源。

3. 使用jstack命令查看线程堆栈信息

- 确定了高CPU占用的线程后，我们需要获取该线程的堆栈信息。这时可以使用jstack命令。
- 运行jstack [PID] > stack.txt将Java进程的线程堆栈信息输出到文本文件中。

- 在stack.txt文件中搜索之前通过top -Hp命令找到的线程ID（需要将线程ID转换成16进制进行搜索），以定位到具体线程的堆栈信息。

4. 分析线程堆栈信息定位问题

- 通过分析堆栈信息，我们可以找到线程中正在执行的方法调用链，从而定位到可能导致CPU占用率上升的代码段。
- 常见的导致CPU占用率上升的原因包括无限循环、复杂的算法和计算、频繁的IO操作以及线程问题（如竞争条件、死锁等）。

5. 使用Arthas进行更深入的分析（可选）

- 如果通过jstack分析仍然无法定位问题，或者需要更实时的监控和分析工具，可以使用Arthas这样的Java诊断工具。
- Arthas提供了丰富的监控和诊断功能，如monitor可以监控方法的执行情况，watch可以检测函数返回值等。
- 通过Arthas的thread命令，我们可以查看当前Java进程的线程信息，包括线程的状态、堆栈等，有助于进一步定位问题。

6. 解决问题并验证

- 在定位到具体问题后，进行代码修改或优化以解决问题。
- 修改完成后，需要重新部署并验证修改是否有效，确保CPU占用率恢复正常。

总结来说，当Java程序线上CPU使用率达到100%时，我们可以通过top命令找到高CPU占用的进程和线程，然后使用jstack命令查看线程堆栈信息定位问题所在。如果需要更深入的分析，可以使用Arthas这样的诊断工具。在定位并解决问题后，需要重新部署并验证修改的有效性。

推荐学习资料

《博客》：[#线上cpu使用率100%如何排查](#)

Redis：架构

63. Redis是单线程还是多线程？单线程为什么还这么快？

解析

参考口述回答

1. redis是单线程还是多线程？

首先，关于Redis是单线程还是多线程的问题，这实际上取决于Redis的版本。在Redis的早期版本，比如3.x版本，它确实是单线程的。这里的单线程主要是指Redis处理客户端请求的主线程是单线程的，即网络IO和键值对读写是由一个线程来完成的。这种设计使得Redis能够避免多线程切换带来的开销，提高了执行效率，同时也避免了线程安全问题。

然而，在后续的版本中，Redis开始逐渐引入多线程的概念。例如，在Redis 4.x版本中，虽然处理客户端请求的主线程仍然是单线程的，但一些辅助功能如异步删除等已经开始使用多线程。到了Redis 6.0版本，多线程的支持得到了进一步的加强，主要体现在处理大量写操作的场景上。这种多线程的设计主要是为了提高Redis的吞吐量，更好地利用现代多核CPU的性能，以及减少在写操作时的延迟。

2. 单线程为什么还这么快？

1. 基于内存操作：Redis的所有数据都存储在内存中，这使得数据的读写速度非常快。内存操作的延迟远远低于磁盘操作，因此Redis能够迅速响应客户端的请求。
2. 简单的数据结构：Redis的数据结构是专门设计的，这些简单的数据结构的查找和操作的时间复杂度大部分都是 $O(1)$ ，这意味着无论数据量多大，操作的时间都是恒定的。
3. IO多路复用和非阻塞IO：Redis使用IO多路复用功能来监听多个socket连接客户端。这样，它就可以使用一个线程连接处理多个请求，从而减少了线程切换带来的开销，并避免了IO阻塞操作。
4. 避免上下文切换：由于Redis采用单线程模型（在早期版本中），因此可以避免不必要的上下文切换和多线程竞争。这样可以省去多线程切换带来的时间和性能上的消耗，同时单线程也不会导致死锁问题的发生。

推荐学习资料

《小林Coding》 | 图解Redis: [Redis 线程模型](#)

64. 过期策略与内存淘汰策略？

解析

过期删除策略：惰性删除；定期删除

内存淘汰策略：noeviction；allkeys-lru；allkeys-lfu；allkeys-random；volatile-lru；volatile-lfu；volatile-ttl；volatile-random

参考口述回答

1. 过期删除策略？

Redis的过期删除策略主要涉及两种机制：**惰性删除**和**定期删除**。这两种策略各有特点，且被Redis结合起来使用，以实现内存的有效管理和CPU资源的合理利用。

首先是惰性删除策略。这种策略在Redis中的实现是，当某个键被访问时，Redis会检查该键是否设置了过期时间以及是否已过期。如果键已过期，Redis会删除该键并返回不存在给客户端。这种策略的优点在于它节约CPU性能，只在必要时进行删除操作。然而，它的缺点也很明显，就是如果某些过期键长时间没有被访问，它们会一直占用内存，造成资源浪费。

为了弥补惰性删除的不足，Redis还采用了定期删除策略。这种策略是Redis每隔一段时间主动对数据库进行一次检查，删除里面的过期键。具体实现上，Redis会随机抽取一定数量的键进行检查，并删除其中的过期

键。这个过程的运行频率和检测数量都是可以通过配置进行调整的。定期删除策略的优点在于它可以有效地释放过期键占用的内存，减少内存浪费。同时，由于它的执行是定期的，也可以在一定程度上平滑CPU的使用率。

总的来说，Redis通过结合惰性删除和定期删除两种策略，既保证了过期键的及时删除，又避免了CPU资源的过度占用。这种设计使得Redis在处理大量数据时仍能保持良好的性能。

2. 内存淘汰策略？

Redis的内存淘汰策略，主要是用来处理内存不足的情况。当Redis使用的内存快要达到我们设定的最大值时，就需要用这些策略来决定哪些数据应该被删除，以释放出内存空间。

首先，有一种叫做**noeviction**的策略，这是Redis的默认设置。这种策略在内存满时不会淘汰任何数据，而是直接拒绝新的写入操作。这适合那些不能丢失任何数据的情况。

然后，我们有**allkeys-lru**策略，它会淘汰最近最少使用的数据，不管这些数据有没有设置过期时间。如果你有很多数据，并且希望保留最常用的数据，这个策略就很有用。

还有**allkeys-lfu**策略，这个策略会淘汰访问频率最低的数据，它也是全局的，不考虑数据是否过期。

接下来是**volatile-lru**和**volatile-lfu**策略，它们只会淘汰那些设置了过期时间的数据。如果你用Redis来做缓存，并且只希望淘汰那些有过期时间的数据，这两种策略就很合适。

volatile-ttl策略则会淘汰设置了过期时间并且即将过期的数据，这样可以最大化利用缓存空间。

除了这些，Redis还有**allkeys-random**和**volatile-random**策略。**allkeys-random**会随机淘汰任意数据，而**volatile-random**则是随机淘汰设置了过期时间的数据。这两种策略在数据选择上没有明确的优先级，适合在数据访问模式没有明显规律或者需要保持数据随机性的场景下使用。

总的来说，选择哪种策略要看你的具体需求，比如数据的重要性、使用频率，以及你是否给数据设置了过期时间等。

推荐学习资料

《小林Coding》 | 图解Redis: [Redis过期删除与内存淘汰](#)

Redis：数据类型

65. 常见数据类型适用场景？

解析

常用5种：字符串String; 哈希Hash; 列表List; 集合Set; 有序集合ZSet

后面新增4种：BitMap; HyperLogLog; GEO; Stream

参考口述回答

1. 5中常用数据类型：

首先，**字符串 String**是Redis最基本的数据类型，它可以存储任意类型的数据，包括文本、数字和二进制数据，且其值最大可以达到512MB。字符串在缓存、计数器、分布式锁等场景下有着广泛的应用。例如，在缓存场景中，我们可以将热点数据存储在Redis的字符串类型中，以提高数据的访问速度；在计数器场景中，我们可以利用Redis的原子性操作来实现高效、准确的计数功能。

其次，**哈希类型 Hash**是一种键值对的集合，它适合存储对象的多个属性。在哈希类型中，每个键都与一个值相关联，这使得我们可以方便地获取或修改对象的某个属性。哈希类型的值最多可以包含 $2^{32}-1$ 个键值对，这使得它能够存储大量的数据。哈希类型在存储用户信息、配置信息等场景下非常有用。例如，我们可以将用户的个人信息存储在一个哈希值中，通过键值对的方式来访问和更新这些信息。

第三，**列表类型 List**是一种有序的字符串集合，它可以存储多个字符串值。列表类型支持在头部或尾部插入元素，以及获取指定范围内的元素等操作。这使得列表类型非常适合用于实现队列、栈等数据结构。例如，在消息队列场景中，我们可以利用列表类型来存储待处理的消息，并通过LPUSH和RPOP等命令来实现消息的入队和出队操作。

第四，**集合类型 Set**是一种无序、唯一的字符串集合。它支持添加元素、删除元素以及判断元素是否存在等操作。此外，集合类型还支持求交集、并集和差集等操作。这使得集合类型非常适合用于存储不重复的元素，并进行快速的集合运算。例如，在标签系统中，我们可以利用集合类型来存储用户的标签信息，并通过集合运算来查找具有相同标签的用户或者计算用户之间的相似度等。

最后，**有序集合类型 ZSet**是一种有序的字符串集合，每个元素都关联着一个分数用于排序。有序集合支持添加元素、删除元素以及获取指定范围内的元素等操作，并且还支持根据分数进行范围查找和计算元素的排名等操作。这使得有序集合非常适合用于排行榜、优先级队列等场景。例如，在排行榜场景中，我们可以利用有序集合来存储用户的得分信息，并通过ZRANK等命令来获取用户的排名情况。

2. 后面新增4种

首先是**Bitmap数据类型**，它是一种基于String类型实现的特殊数据结构，用于存储大量二进制位（0或1）的数据。这些位可以代表不同的状态或标志。Bitmap非常适合用于签到打卡等场景，因为这类场景通常只需要记录用户是否进行了某个操作，例如签到（1）或未签到（0）。使用Bitmap可以极大地节省存储空间，并提高处理速度。

接下来是**HyperLogLog数据类型**，它是Redis 2.8.9版本引入的，用于做数据基数计算。HyperLogLog的优势在于它占用的内存非常少，并且不保存原始数据，而是通过一定的算法估计数据的基数。尽管它有一定的误差（标准误差为0.81%），但在对数据准确度要求不是特别高的场景下，如统计网站注册IP数、每日访问IP数或页面实时UV/PV数等，HyperLogLog是一个非常好的选择。

再来说说**GEO数据类型**，这是Redis 3.2版本新增的特性，用于存储经纬度格式的地理坐标，并对这些坐标执行距离计算、范围查找等操作。GEO数据类型非常适合用于地理位置相关的业务场景，如各类社交软件中的“附近的人”功能、外卖配送服务中的距离计算和最优配送员分配等。

最后是**Stream数据类型**，它类似于消息队列或日志系统，可以记录一系列的事件或消息，并按照时间顺序进行存储。Stream数据类型非常适合用于实时监控、大数据处理、数据流转、实时数据分析和视频流处理等场景。例如，在实时监控中，可以使用Stream来记录服务器的日志或网络流量数据；在大数据处理中，可以使用Stream来对一批数据进行筛选、排序和统计等操作；在实时数据分析中，可以使用Stream来处理网站访问记录并了解用户行为。

66. 5种常见数据类型是如何实现的?

解析

参考口述回答

首先,我们来看String类型。在Redis中,String类型的底层实现根据存储内容的不同,会有三种编码方式:**int**、**raw**和**embstr**。如果字符串的内容可以解析为整数,Redis会将其编码为int类型,直接存储整数值。对于长度较短的字符串,Redis会使用embstr编码,这是一种紧凑的字符串表示方式。而对于长度较长的字符串,Redis则会使用raw编码,即简单的动态字符串。

接下来是Hash类型。在Redis中,Hash的底层实现主要依赖于**哈希表**。哈希表是一种根据键直接访问在内存存储位置的数据结构,它通过计算哈希值来快速定位数据。在Redis 7中,Hash类型的编码方式有所变化,使用了listpack或hashtable来实现。(注意:如果哈希类型元素个数小于 512 个,所有值小于 64 字节的话,Redis 会使用**压缩列表**作为 Hash 类型的底层数据结构)

对于List类型,Redis的底层实现主要使用了**QuickList**。在Redis 3.2版本之前,主要使用压缩列表ZipList和双向链表LinkedList来实现。但是,在3.2版本之后,Redis统一只采用QuickList来实现List,它结合了LinkedList和ZipList的优点,提供了更高的存储上限和更好的性能。

Set类型的底层实现主要是基于**哈希表**。Redis将每个元素作为哈希表的一个键,并将其值设置为NULL。由于哈希表的键具有唯一性,所以Set类型中的元素也是唯一的。这种实现方式保证了Set的高效性和唯一性。

(注意:如果集合中的元素都是整数且元素个数小于 512 个,Redis 会使用**整数集合**作为底层数据结构)

最后是Sorted Set类型,它在Redis中的底层实现依赖于跳跃表(SkipList)和 哈希表。跳跃表是一种可以进行二分查找的有序链表,它提高了数据的查找效率。同时,哈希表用于快速查找元素。这种结合使得Sorted Set在保持元素有序的同时,也提供了高效的查找性能。(注意:如果有序集合的元素个数小于 128 个,并且每个元素的值小于 64 字节时,Redis 会使用**压缩列表或listpack**作为 Zset 类型的底层数据结构;)

推荐学习资料

《小林Coding》 | 图解Redis: [Redis 常见数据类型底层实现](#)

67. 介绍一下ZSET与跳表? 描述一个元素的插入过程, 随机层高?

解析 参考口述回答

1. ZSET与跳表

ZSET,即有序集合,是Redis提供的一种数据结构。它类似于Set,但其成员是唯一的,且每个成员都关联着一个分数。这个分数用于对集合中的元素进行排序。ZSET在实现上采用了跳表(SkipList)和哈希表(HashTable)两种数据结构,其中跳表用于维护元素的排序,而哈希表则提供快速的元素查找。

跳表(SkipList)是一种随机化的数据结构,实质上就是一种可以进行二分查找的有序链表。跳表在原有的有序链表上面增加了多级索引,通过索引来实现快速查找。它不仅提高了搜索性能,同时也提高了插入和删除操作的性能。

1. 跳表的插入过程

Redis中跳表的插入过程：

一、查找插入位置

1. 从跳表的最高层开始，从左到右依次检查每个节点的下一个节点。
2. 在查找的过程中，记录每一层中最后一个小于或等于待插入元素的节点，这些节点被称为“前驱”。
3. 当找到插入位置或者到达跳表的底层时停止查找。

二、创建新节点并设置值

1. 根据待插入元素创建一个新的节点。
2. 将新节点的值设置为待插入元素的值。

三、更新前驱节点的指针

1. 在每一层中，将新节点的指针指向前驱节点的下一个节点。
2. 将前驱节点的指针指向新节点。

四、决定是否提升新节点

1. 通过一个随机算法决定是否将新节点提升到更高层，以便加快查找的速度。
2. 如果新节点被提升到更高层，需要对新节点进行连接操作。具体来说，就是在每一层中找到前驱节点的上一层节点，然后将新节点的指针指向前驱节点的上一层节点的下一个节点，接着将前驱节点的上一层节点的指针指向新节点。

这个过程确保了跳表在插入新元素后仍然保持有序，并且通过**随机化的层高**设计，使得跳表在查询、插入和删除操作时都能保持高效的性能。

总的来说，Redis中跳表的插入过程是一个结合了**有序链表和二分查找**思想的优化算法，它通过增加多级索引和随机化的层高设计来提高操作效率。

2. 跳表节点层数设置？

首先，跳表节点的层数设置具有随机性。在Redis的跳表中，每个节点的层数是随机生成的，这种随机性有助于保持跳表的平衡，防止某一层级的节点过多，从而影响查询效率。具体来说，每个节点在初始化时都会被赋予一个基本的层数，通常是1层。然后，通过一个随机数生成器，决定该节点是否增加额外的层级。

其次，层数的增加遵循一定的概率分布规则。在Redis中，这个概率通常是由一个固定的概率因子 P （25%）来控制的。每当需要增加一个新的层级时，系统会生成一个介于0和1之间的随机数 p ，并将其与概率因子 P 进行比较。如果 p 小于 P ，则节点的层数会增加一层。这个过程会重复进行，直到生成的随机数 p 大于或等于 P 为止。

值得注意的是，虽然每个节点的层数是随机设置的，但整体上跳表的层数分布是稳定的。这是由于大量的节点在层数设置上遵循相同的概率规则，因此从统计学的角度来看，跳表的层级结构会趋于稳定。

综上所述，跳表节点的层数设置是通过随机化和概率控制来实现的。

68. 介绍下哈希表的rehash?

解析 参考口述回答

1. rehash介绍?

当哈希表中的键值对数量超过一定阈值时,为了保证哈希表的性能和稳定性,Redis会自动进行rehash操作。具体来说, rehash就是将哈希表中的所有键值对重新散列到新的哈希表中。

在rehash过程中, Redis会创建一个新的哈希表,其大小通常是原哈希表大小的两倍(扩容时)。然后, Redis会将原哈希表中的所有键值对重新散列到新哈希表中。这个过程中, Redis会使用新的哈希表大小来计算每个键值对的新哈希值,并将其插入到新哈希表的相应位置。

2. 渐进式Rehash

为了避免一次性rehash导致的长时间阻塞, Redis采用了渐进式rehash的策略。这种策略将rehash操作分散到多个指令的执行过程中,从而降低了对系统性能的影响。

在渐进式rehash过程中, Redis会维护两个哈希表:原哈希表和新哈希表。当触发rehash操作时,新的键值对会直接存储到新哈希表中,而原有的键值对则会在后续的指令执行过程中逐渐被迁移到新哈希表中。这个过程是逐步完成的,因此称为渐进式rehash。

3. Rehash触发条件

扩容条件:当哈希表中的键值对数量超过一定阈值时,会触发扩容rehash。具体来说,在Redis中,这个阈值通常是哈希表大小的某个比例(如负载因子超过某个值)。当达到这个阈值时,为了保证查询效率和性能, Redis会进行扩容操作。

推荐学习资料

《小林Coding》 | 图解Redis: [Redis 数据结构-哈希表-rehash](#)

Redis: 持久化

69. Redis持久化怎么做的?

解析

RDB; AOF; 混合持久化

参考口述回答

首先是RDB持久化，也就是Redis DataBase的缩写。这种方式是通过在指定的时间间隔内，将Redis内存中的数据生成快照并写入到一个二进制文件中，通常是一个名为dump.rdb的文件。RDB的优点是生成的文件较小，且恢复速度快。比如，我们可以通过配置如“save 60 1000”这样的参数，告诉Redis在60秒内有1000个键被修改时，就自动保存一次数据集。当然，我们也可以手动执行save或bgsave命令来生成快照。但RDB的缺点是它可能会丢失一些数据，因为它是基于时间点的快照，而不是实时的。

接下来是AOF持久化，即Append Only File。与RDB不同，AOF是通过记录Redis的所有写命令并追加到一个文件中来实现的。这意味着每当Redis执行一个写命令，这个命令就会被记录到AOF文件中。当Redis重启时，它会重新执行AOF文件中的命令来恢复数据。AOF的优点是数据安全性更高，因为它记录了所有的写操作。但缺点是文件体积可能会很大，且恢复速度相对较慢。

除了单独的RDB和AOF持久化外，Redis还支持混合持久化。这种方式结合了RDB和AOF的优点。它首先使用RDB方式创建一个快照，然后将后续的写命令追加到AOF文件中。这样，在恢复数据时，Redis可以先加载快照以快速恢复到某个时间点，然后再通过执行AOF文件中的命令来恢复到最新状态。

推荐学习资料

《小林Coding》 | 图解Redis: Redis 持久化: RDB 和 AOF 合体

70. AOF | AOF刷盘策略 | AOF瘦身（重写机制）？

解析

参考口述回答

一、AOF基本概念

AOF，即Append Only File，是Redis的一种持久化方式。与RDB不同，AOF不是通过生成数据快照来保存数据，而是通过将每一个写命令追加到一个文件中，从而在Redis重启时可以通过重新执行这些命令来恢复数据。这种方式提供了更高的数据安全性，因为即使在Redis突然宕机的情况下，已经追加到AOF文件中的命令仍然可以用来恢复数据。

二、AOF刷盘策略

AOF的刷盘策略决定了何时将写命令从缓冲区同步到磁盘上的AOF文件中。Redis提供了三种刷盘策略：

1. **always**：每个写命令都会立即同步到磁盘。这种方式虽然最安全，但性能开销也最大，因为每次写操作都需要进行磁盘I/O。
2. **everysec**：每秒同步一次。这是默认且推荐的策略，它在数据安全性和性能之间达到了一个平衡。
3. **no**：由操作系统决定何时同步到磁盘。这种方式性能最好，但数据安全性最低。

三、AOF瘦身（AOF重写机制）

随着时间的推移，AOF文件会变得越来越大，这可能会影响到Redis的启动速度和恢复时间。为了解决这个问题，Redis提供了AOF重写（瘦身）的功能。AOF重写的过程是创建一个新的AOF文件，这个文件只包含恢复当前数据集所需的最小命令集。重写完成后，旧的AOF文件会被新的AOF文件替换。这个过程可以在不中断服务的情况下进行，并且通常会显著减小AOF文件的大小。AOF重写可以通过手动触发（使用**bgrewriteaof**命令）或根据配置的自动触发条件（如文件大小增长比例和时间间隔）来执行。

推荐学习资料

《小林Coding》 | 图解Redis: [AOF 持久化是怎么实现的?](#)

Redis: 高可用

71. Redis是怎么实现服务的高可用的? (集群 | 主从 | 哨兵 | 故障转移)

解析

集群 | 主从 | 哨兵

参考口述回答

Redis实现服务的高可用性主要依赖于几种关键机制: 集群、主从模式、哨兵机制和故障转移过程。下面我将逐一详细介绍这些机制。

首先是**集群**。Redis Cluster是Redis的分布式解决方案,它通过分片将数据分布在多个Redis节点上。每个节点都存储着一部分数据,并且节点之间可以相互通信。这种分布式架构不仅提高了数据的可扩展性,还增强了服务的可用性。当一个节点出现故障时,其他节点仍然可以继续提供服务,从而确保整体服务的可用性。

其次是**主从模式**。在这种模式下,我们有一个主节点和多个从节点。主节点负责处理写入操作,而从节点则复制主节点的数据并处理读取请求。这种读写分离的设计可以提高系统的吞吐量和可用性。如果主节点出现故障,我们可以手动或通过自动化工具将一个从节点提升为主节点,以确保服务的连续性。

接下来是**哨兵机制**。Redis Sentinel是一个高可用的解决方案,它用于监控Redis节点的状态并在主节点出现故障时进行故障转移。哨兵会定期检查Redis节点的状态,并在检测到主节点故障时触发故障转移过程。这个过程包括选举一个新的主节点、通知其他从节点更新配置以及将客户端重定向到新的主节点等步骤。哨兵机制能够自动处理故障转移,从而大大提高了Redis服务的可用性。

综上所述,Redis通过集群、主从模式、哨兵机制实现了服务的高可用性。这些机制能够确保在Redis节点出现故障时,服务仍然能够正常运行,并且数据不会丢失

推荐学习资料

《博客》segmentfault: [#\[Redis\] 你了解 Redis 的三种集群模式吗?](#)

72. 热key | 大key | 防护方案

解析

参考口述回答

问题介绍

在Redis中,大Key指的是那些占用内存较大或者包含大量元素的Key,而热Key则是指访问频率非常高的Key。这两种情况都可能对Redis的性能和稳定性造成影响。大Key可能导致内存占用过高、网络传输效率低下

以及备份恢复困难等问题；而热Key则可能引发读写倾斜，导致某些节点负载过高，甚至引发缓存击穿等风险。

如何发现大Key和热Key

对于大Key的发现，我们可以采用以下几种方法：

1. 使用Redis自带的命令，如`--bigkeys`选项，对Redis进行扫描，找出各种数据类型中最大的Key。
2. 利用`MEMORY USAGE`命令查看特定Key的内存使用情况，从而判断其是否为大Key。
3. 借助第三方工具进行扫描和分析，如`redis-memory-analyzer`等。

对于热Key的发现，可以考虑以下途径：

1. 通过Redis的`MONITOR`命令监控实时命令流，分析哪些Key被频繁访问。
2. 使用第三方工具或自定义脚本统计每个Key的访问频率。
3. 根据业务特点和历史数据，预判哪些Key可能会成为热Key。

如何解决大Key和热Key问题

针对大Key问题，我们可以采取以下措施：

1. 拆分大Key：将一个大Key拆分成多个小Key，分散存储和访问压力。
2. 优化数据结构：选择更合适的数据结构来减少内存占用和提高访问效率。
3. 设置过期时间：为大Key设置合理的TTL（Time To Live），以便在一段时间后自动清理。

对于热Key问题，可以尝试以下方法：

1. 利用二级缓存：将热Key的数据缓存到离用户更近的地方，如使用本地缓存或CDN来减少对Redis的访问压力。
2. 数据分片：通过一定的策略将热Key分散到多个Redis节点上，实现负载均衡。
3. 优化查询逻辑：减少不必要的查询和重复查询，降低对热Key的访问频率。

推荐学习资料

《阿里云》技术文档：[发现并处理Redis的大Key和热Key](#)

Redis：应用

73. 数据库和缓存如何保证一致性？

解析

1. 删除缓存和更新数据库顺序
2. 延迟双删除
3. 消息队列重试

参考口述回答

先删除缓存还是先更新数据库？

1. 先删除缓存，再更新数据库：

- 问题：在删除缓存后、更新数据库前的这段时间里，如果有读请求，会因为缓存已删除而读取数据库中的旧数据，然后这些数据又会被写入缓存，导致数据不一致。

2. 先更新数据库，再删除缓存：

- 问题：在更新数据库后、删除缓存前的这段时间里，如果有读请求，会读取到缓存中的旧数据。此外，如果删除缓存操作失败，也会导致数据不一致。

解决方案

为了解决上述问题，我们可以采用以下策略：

1. 延迟双删策略：

- 操作步骤：首先删除缓存，然后更新数据库，稍微延迟几百毫秒（这个延迟时间需要根据具体的业务场景来定），再次删除缓存。
- 优点：这种方法能够较大程度地保证数据库和缓存的一致性。
- 注意事项：延迟时间的设置是关键，太长会影响性能，太短则可能达不到预期效果。

2. 结合消息队列和重试机制：

- 当更新数据库成功后，可以将删除缓存的操作放入消息队列中。
- 如果删除缓存失败，可以利用消息队列的重试机制来重新尝试删除操作。
- 优点：这种方法能够确保即使初次删除缓存失败，也能通过重试机制最终达到数据一致性。
- 注意事项：需要合理配置消息队列和重试策略，以避免对系统造成过大的压力。

推荐学习资料

《小林Coding》 | 图解Redis: [数据库和缓存如何保证一致性？](#)

74. 什么是缓存雪崩、击穿、穿透？如何解决？

解析

参考口述回答

缓存雪崩

缓存雪崩是指缓存由于某些原因（如大量数据同时过期或Redis 故障宕机）整体崩溃，导致大量请求到达后端数据库，从而导致数据库崩溃，整个系统崩溃，发生灾难。

解决方案：

1. 使用分布式缓存部署，避免单点故障。
2. 设置不同的过期时间，避免缓存集中失效。
3. 缓存不过期，使用缓存预热和自动刷新机制。

4. 应用程序限流，避免过多请求同时到达数据库。

缓存击穿

缓存击穿是指一个热点数据失效后，大量请求同时涌入后端存储，导致后端存储负载增大、响应时间变慢，甚至瘫痪。

解决方案：

1. 使用互斥锁或者分布式锁对并发请求进行控制，避免对同一资源的并发读写竞争。
2. 热点数据预加载，提前将热点数据加入缓存，在其失效时快速刷新缓存。

缓存穿透

缓存穿透是指查询一个数据库一定不存在的数据。正常的使用缓存流程大致是，数据查询先进行缓存查询，如果key不存在或者key已经过期，再对数据库进行查询，并把查询到的对象，放进缓存。如果数据库查询对象为空，则不放进缓存。持续查询不存在的数据会导致所有请求都落到后端存储上，导致系统瘫痪。

解决方案：

1. 使用布隆过滤器或者黑白名单等方式过滤掉无效请求。
2. 在应用程序中加入缓存预热等机制，预先加载常用数据到缓存中。
3. 对于查询结果为空的数据，也可以将其key对应的value设置为一个默认值（如null），并设置一个较短的缓存过期时间，以减少对数据库的无效查询。

推荐学习资料

《小林Coding》 | 图解Redis：什么是缓存雪崩、击穿、穿透？

75. redis 如何实现分布式锁？除了redis还有哪些可以做分布式锁？

解析

参考口述回答

1. redis如何实现分布式锁？

1. 使用SetNX实现基本的锁机制

Redis的SETNX命令（SET if Not eXists）是实现分布式锁的基础。当一个客户端尝试获取锁时，它会使用SETNX来设置一个键。如果键不存在，SETNX会设置该键并返回1，表示成功获取了锁。如果键已经存在，SETNX不会做任何事情并返回0，表示获取锁失败。

设置过期时间防止死锁

但是，仅仅使用SETNX是不够的。如果持有锁的客户端在处理过程中崩溃或网络断开，那么锁可能永远不会被释放，导致其他客户端永远无法获取锁，这就是所谓的死锁。为了防止这种情况，我们需要为锁设置一个过期时间。在Redis中，这可以通过EXPIRE命令来实现。

使用Lua脚本确保解锁操作的原子性

解锁操作需要谨慎处理。为了确保解锁操作的原子性，我们通常使用Lua脚本来执行。Lua脚本在Redis中是原子执行的，这意味着在脚本执行期间，其他命令不会被插入执行。这确保了即使在并发环境中，解锁操作也是

安全的。

看门狗机制自动续期

为了确保锁在客户端处理过程中不会因过期而被意外释放，我们可以实现一个看门狗机制来自动续期锁的过期时间。这可以通过在客户端创建一个定时任务来实现，该任务在锁过期前不断地重新设置锁的过期时间。

2. 使用Redisson库简化分布式锁的实现

手动实现上述所有功能可能相当复杂且容易出错。幸运的是，有一些库如Redisson已经为我们封装了这些复杂的逻辑。使用Redisson，我们可以非常简单地实现分布式锁，而无需关心底层的细节。Redisson还提供了很多高级功能，如锁的重试机制、公平锁等。

2. 除了Redis还有哪些方式可以实现分布式锁？

1. 基于数据库的分布式锁

我们可以利用数据库来实现分布式锁。例如，在MySQL中，可以使用SELECT ... FOR UPDATE语句来锁定某一行数据。当一个事务在对数据执行修改操作前，先使用此语句锁定该行，其他事务在此期间无法对该行进行修改，从而实现了锁的效果。但需要注意的是，数据库锁可能会引起性能瓶颈，特别是在高并发场景下。

2. 基于Zookeeper的分布式锁

Zookeeper是一个开源的分布式协调服务，它可以帮助我们实现分布式锁。在Zookeeper中，我们可以创建一个临时的顺序节点来表示锁。当客户端需要获取锁时，它会在指定的目录下创建一个顺序节点。然后，客户端会检查自己创建的节点是否是最小的节点，如果是，则获取到锁；如果不是，则监听比自己序号小的前一个节点。当前一个节点被删除时，客户端会收到通知并重新检查自己是否是当前最小的节点。这种方式可以有效地避免死锁，并且具有较好的性能。

3. 其他第三方库或服务

除了上述两种常见的方式外，还有一些第三方库或服务也提供了分布式锁的功能，例如etcd等。这些库或服务通常提供了简单易用的API，使我们能够快速实现分布式锁。

推荐学习资料

《博客》ITPUB: [Redis 分布式锁的正确实现原理演化历程与 Redisson 实战总结](#)

《博客》Dunwu Blog: [分布式锁基本原理: 含非redis实现方式](#)

计算机网络：基础

76. 介绍下OSI七层模型？

解析：

OSI七层模型。这个模型从下到上依次为：物理层、数据链路层、网络层、传输层、会话层、表示层和应用层。每一层都有其特定的功能和协议。

参考口述回答：

首先是**物理层**，它主要定义物理设备标准，比如网线的接口类型和传输介质的传输速率等。物理层的主要作用就是透明地传输比特流，也就是传输最原始的数据信号。

紧接着是**数据链路层**，这一层主要将从物理层接收的数据进行MAC地址的封装与解封装，数据在这一层被称为帧。数据链路层还负责处理流控制、物理地址寻找以及数据的重发等。

再往上是**网络层**，它的主要功能是选择合适的网间路由和交换节点，确保数据及时传送。网络层将从下层接收到的数据进行IP地址的封装和解封装，数据在这一层被称为数据包。

接下来是**传输层**，它主要负责建立连接、处理数据包错误以及数据包次序。传输层将从下层收到的数据进行分段和传输，达到目的地址后进行重组。常见的传输层协议有TCP和UDP。

之后是**会话层**，它主要负责在网络中的两个节点之间建立、维持和终止通信。会话层还负责解决节点链接的协调和管理，包括同步两个节点之间的对话，并决定通信是否被中断以及通信中断时从何处重新发送。

紧接着是**表示层**，它主要是对接收的数据进行解释、压缩与解压等处理，确保数据能够以合适的格式进行传输和呈现。

最后是**应用层**，它是OSI七层模型中的最高层，主要负责为网络用户或应用程序提供各种服务。应用层确定通信对象，提供访问网络服务的接口。常见的应用层协议有Telnet、FTP、HTTP和SNMP等。

推荐学习资料：《博客》 | CloudFlare: [什么是 OSI 模型？](#)

77. 介绍下TCP/IP 网络模型？

解析：

TCP/IP网络模型由四个层次构成，从下到上分别是网络接口层、网络层、传输层和应用层

参考口述回答：

首先，网络接口层，也被称为链路层，它主要负责数据的物理传输，包括将数据包从一个节点传输到另一个节点。这一层处理的是如何在物理链路上传输数据，涉及到数据的成帧、差错控制等问题。

接下来是网络层，它的主要任务是负责数据包在网络中的路由选择。网络层通过IP协议为数据包选择最佳的路径，确保数据能够准确、高效地到达目的地。同时，网络层还处理数据包的分片与重组，以适应不同网络环境的传输需求。

再往上是传输层，它提供了端到端的数据传输服务。传输层通过TCP或UDP等协议，确保数据在发送端和接收端之间可靠、有序地传输。其中，TCP协议通过建立连接、确认、重传等机制，保证了数据传输的可靠性和顺序性；而UDP协议则提供了一种无连接、不可靠的数据报服务，适用于对实时性要求较高、可以容忍一定数据丢失的应用场景。

最后是应用层，它位于TCP/IP模型的最顶层，直接为用户提供各种网络服务。应用层包含了多种应用协议，如HTTP、FTP、SMTP等，这些协议定义了不同应用程序之间的通信规则和数据格式。通过应用层协议，用户可以轻松地实现各种网络应用，如浏览网页、上传下载文件、发送电子邮件等。

推荐学习资料：

《小林Coding》 | 图解网络：[#TCP/IP 网络模型有哪几层？](#)

78. 浏览器键入网址到网页显示，期间发生了什么？

解析：

按照HTTP、DNS、TCP、IP、MAC以及相关的网络设备（交换机、路由器）和服务端、客户端的数据处理流程来详细阐述这个过程

参考口述回答：

首先，我们输入的是网址，也就是URL，这是HTTP协议的使用入口。浏览器会首先检查是否有缓存，如果没有，就会通过**HTTP协议**向服务器发起请求。但在此之前，需要通过DNS协议将网址解析为服务器对应的IP地址。

DNS协议工作在应用层，浏览器会向DNS服务器发送查询请求，获取网址对应的IP地址。一旦获取到IP地址，就可以准备建立与服务器的连接了。

接下来是**TCP协议**的工作。TCP位于传输层，它负责建立和管理连接，确保数据的可靠传输。浏览器会与服务器的指定IP地址和端口号进行TCP三次握手，建立连接。

在TCP连接建立后，浏览器就可以通过HTTP协议发送具体的网页请求了。HTTP请求会被封装在TCP数据包中发送出去。

接下来是**IP协议**和**MAC协议**的工作。IP协议在网络层，它负责为数据包提供路由信息，即数据包应该如何从源地址传输到目的地址。每个数据包都会包含源IP地址和目标IP地址。而MAC协议则工作在数据链路层，它负责处理数据包在局域网内的传输。每个网络设备都有一个唯一的MAC地址，MAC协议确保数据包能够准确到达局域网内的目标设备。

当数据包从客户端发送到网络中时，首先会经过**交换机**。交换机根据数据包的MAC地址来决定数据包应该转发到哪个端口。然后数据包会经过**路由器**，路由器会根据IP地址来决定数据包应该转发到哪个网络。这样，数据包就能够逐级跳转，最终到达目标服务器。

服务器收到数据包后，会进行**拆包**处理。首先拆除的是MAC层的数据包，然后是IP层的数据包，接着是TCP层的数据包，最后是HTTP层的数据。服务器会根据HTTP请求的内容来准备响应数据，并将响应数据按照相反的顺序**封装成数据包**发送回客户端。

客户端收到服务器的响应数据包后，也会进行类似的**拆包**处理，最终提取出HTTP响应的内容。浏览器会根据HTTP响应的内容来**渲染网页**，展示给用户。

这就是整个从浏览器键入网址到网页显示的过程中所涉及的网络层协议和主要过程。

推荐学习资料：

《小林Coding》 | 图解网络：[键入网址到网页显示，期间发生了什么？](#)

计算机网络：TCP&UDP

79. 说一下TCP和UDP的区别以及各自的应用场景

解析

这种相对开放的问题，回答的时候需要掌握技巧。如果面试官是倾听聊天型这种，我们可以系统回答，1是让面试官意识到你对知识的理解掌握程度，2是掌控面试节奏，也消磨了时间，这样面试官能考察的其他内容就更少了。留给面试官的印象就是，考察的问题都会，并且掌握得也很系统。

如果面试官是输出型，打断型这种，我们就简单回答，等他追问。

参考口述回答（简单版）

TCP和UDP的主要区别体现在它们的连接方式和数据传输的可靠性上。

首先，TCP是一个面向连接的协议，它在数据传输之前需要先建立连接，确保数据的可靠传输，像打电话一样需要事先拨号建立通话。而UDP则是无连接的，它不需要在传输数据之前建立连接，就像发短信一样直接发送，速度可能更快，但数据可靠性可能不如TCP。

其次，TCP提供可靠的数据传输服务，通过数据校验、重传等机制确保数据完整性和顺序性。而UDP不保证数据包的可靠传输，数据包可能会丢失或乱序。

总的来说，TCP注重数据的可靠性和顺序性，适用于文件传输等需要稳定可靠传输的场景；UDP则更注重传输速度，适用于实时性要求较高的场景，如在线视频等。在选择使用TCP还是UDP时，需要根据具体的应用需求和场景来做决策。

参考口述回答（系统版）

这两种协议都是传输层协议，但它们在确实许多关键方面有着显著的不同。

首先，从连接方式上来看，TCP是一个面向连接的协议。这意味着，在数据传输开始之前，发送方和接收方必须先进行“三次握手”以建立连接。这个过程确保了双方都已准备好进行数据传输，并提供了一种同步机制。一旦连接建立，数据就可以在一个可靠的、有序的通道中进行传输。这种连接方式使得TCP非常适合需要稳定、可靠数据传输的应用场景，如文件传输、电子邮件等。

与此相反，UDP是一个无连接的协议。这意味着UDP在发送数据之前不需要与接收方建立连接。发送方可以直接将数据包发送到网络上，而无需等待接收方的确认或同步。这种无连接的方式使得UDP具有较低的延迟和较高的传输效率，但同时也意味着数据包可能会在传输过程中丢失或乱序。因此，UDP更适合于那些对实时性要求较高、能够容忍一定数据包丢失的应用场景，如在线视频、实时游戏等。

其次，从可靠性上来看，TCP提供了可靠的数据传输服务。它通过数据校验、序列号、确认应答、重传机制等一系列措施来确保数据的完整性和准确性。如果数据包在传输过程中丢失或损坏，TCP会要求发送方重新发送数据包，直到接收方正确接收到数据为止。这种可靠性机制使得TCP在传输重要数据时具有很高的可信度。

而UDP则不提供这样的可靠性保证。它仅仅是将数据包发送到网络上，而不关心数据包是否能够成功到达接收方。如果数据包在传输过程中丢失或损坏，UDP并不会进行重传或修复操作。这种“尽力而为”的传输方式使得UDP具有较低的延迟和较高的效率，但同时也需要应用层来处理可能的数据包丢失或乱序问题。

除了连接方式和可靠性之外，TCP和UDP还在其他方面有所不同。例如，TCP是一个基于字节流的协议，它将数据划分为较小的数据包进行传输，并在接收端将这些数据包重新组装成原始数据流。这种方式使得TCP能够

灵活地处理各种大小的数据传输需求。而UDP则是一个基于数据报的协议，它将应用层传来的数据直接打包成数据报进行传输。这种方式使得UDP能够更加高效地处理单个数据包，但也可能导致数据包的丢失或乱序问题。

另外，TCP还具有流量控制和拥塞控制机制，能够根据网络状况动态调整发送速率，避免网络拥堵和数据丢失。而UDP则没有这些控制机制，需要应用层自行处理相关问题。

总的来说，TCP和UDP在连接方式、可靠性、数据传输方式以及控制机制等方面都有着显著的不同。这些差异使得它们适用于不同的应用场景和需求。例如，对于需要稳定、可靠数据传输的文件传输、电子邮件等应用，我们通常会选择TCP；而对于需要低延迟、高效率的在线视频、实时游戏等应用，我们可能会选择UDP。当然，在实际应用中，我们也需要根据具体情况进行权衡和选择。

推荐学习资料

《小林 Coding》 / 图解网络：UDP 和 TCP 有什么区别呢？分别的应用场景是？

80. TCP数据包和UDP数据包内容上的区别？

参考口述回答

关于TCP数据包和UDP数据包内容上的区别，可以这样来看：

TCP数据包的内容相对复杂，它包括了诸如源端口号、目的端口号、序号、确认号、数据偏移、保留位、控制位、窗口大小、检验和以及紧急指针等字段。这些字段共同构成了TCP数据包的头部，用于确保数据的可靠、顺序传输。特别地，TCP数据包中的序列号和确认号用于数据的序列化和确认机制，这是TCP提供可靠传输的关键。此外，窗口大小字段则用于流量控制，以确保数据的平稳传输。

相比之下，UDP数据包的内容则相对简单。它主要包含源端口、目的端口、数据包的长度以及校验和等字段。UDP数据包没有像TCP那样的序列化和确认机制，因此它不保证数据的可靠传输。这也意味着UDP数据包在传输过程中可能会出现数据丢失或乱序的情况。然而，正是由于UDP数据包的简单性，它在传输效率上通常优于TCP。

总的来说，TCP数据包和UDP数据包在内容上的主要区别在于TCP数据包包含了更多的控制和管理信息，以确保数据的可靠传输；而UDP数据包则更加简洁，注重传输效率。这些差异使得TCP和UDP在各自适用的场景中发挥着不同的作用。

推荐学习资料

《小林 Coding》 / 图解网络：

TCP 头格式有哪些？

UDP 头格式

81. TCP和UDP可以共用端口吗？

参考回答

可以的 TCP 和 UDP 可以同时绑定到相同的端口上，因为它们在内核中是作为两个完全独立的软件模块来处理的。当一个数据包到达时，操作系统会检查 IP 包头的协议号字段来确定该数据包是属于 TCP 还是 UDP。根据这个信息，数据包会被送到相应的协议模块进行处理。然后，模块会根据端口号将数据包转发给相应的应用程序。

由于 TCP 和 UDP 是独立的协议，它们可以并行运行而不会相互干扰，即使它们都绑定到同一个端口上。这种设计允许在同一台计算机上的不同应用程序可以同时使用 TCP 和 UDP 协议，而不会发生端口冲突。

推荐学习资料

《小林 Coding》 / 图解网络：TCP 和 UDP 可以使用同一个端口吗？

82. 讲一下TCP三次握手？

参考口述回答（简单版）

TCP三次握手是建立TCP连接的一个过程。首先，客户端向服务器发送一个SYN包，询问服务器是否愿意建立连接，这可以看作是客户端的打招呼。接着，服务器回应一个SYN+ACK包，表示愿意建立连接，并对客户端的初始序列号进行确认，这相当于服务器的回应和打招呼。最后，客户端再发送一个ACK包，对服务器的SYN包进行确认，这就像是客户端对服务器的回应说‘好的，我也准备好了’。这样，三次握手完成后，TCP连接就建立了。这个过程确保了双方都能准确地知道对方已准备好进行数据传输，是TCP协议中保证数据可靠传输的重要机制之一。

参考口述回答（系统版）

TCP三次握手是TCP协议中建立连接的重要过程，它涉及客户端和服务器的状态变化以及发送请求的具体内容。

一开始，客户端和服务端都处于CLOSE状态。当服务端准备好接受连接时，它会进入LISTEN状态，等待客户端的连接请求。

第一次握手时，客户端会初始化一个随机序号，并发送一个SYN请求连接报文给服务端，这个报文中包含了客户端的初始序列号。此时，客户端的状态变为SYN-SENT，等待服务端的确认。

服务端收到SYN请求后，会进行第二次握手。服务端也会随机初始化一个序号，并将客户端的序列号加1作为确认应答报文ACK，同时发送自己的SYN报文。这样，服务端发送的就是一个SYN+ACK的报文，此时服务端的状态变为SYN-RCVD。

客户端收到服务端的SYN+ACK报文后，会进行第三次握手。客户端会再次发送一个ACK报文给服务端，确认服务端的SYN报文，并将服务端的ACK序列号加1。此时，客户端的状态变为ESTABLISHED，表示连接已建立。服务端收到客户端的ACK报文后，状态也变为ESTABLISHED。

这个三次握手的过程确保了客户端和服务端都能准确地知道对方已准备好进行数据传输，是TCP协议中保证数据可靠传输的重要机制。

推荐学习资料

《小林 Coding》 / 图解网络：[TCP 建立连接](#) | [三次握手](#)

83. TCP三次握手的状态变化？

参考口述回答

首先，握手开始之前，客户端和服务器的TCP连接都处于CLOSED状态。当客户端想要建立连接时，它会首先进入SYN-SENT状态，并发送一个SYN报文给服务器，等待服务器的确认。

服务器收到SYN报文后，会进入SYN-RECEIVED状态，表示已经收到客户端的连接请求，并等待确认。然后，服务器会回应一个SYN+ACK报文给客户端，其中ACK报文用来确认客户端的SYN报文。

客户端收到服务器的SYN+ACK报文后，会再次发送一个ACK报文给服务器，然后进入ESTABLISHED状态，表示连接已成功建立且可以进行数据传输了。

服务器收到客户端的ACK报文后，也会进入ESTABLISHED状态，此时双方都已准备好进行数据传输。

推荐学习资料

《小林 Coding》 / 图解网络：[TCP 三次握手过程是怎样的？](#)

84. 为什么是3次握手，能否改成2次握手或者4次？

参考口述回答

为什么不是2次？

首先，三次握手能确保双方都具有接收和发送数据的能力。在第一次握手时，客户端发送SYN报文，服务端若能接收则证明客户端发送、服务端接收能力正常；第二次握手时，服务端发送SYN+ACK报文，客户端若能接收则证明服务端发送、客户端接收能力正常；第三次握手时，客户端发送ACK报文，服务端若能接收则证明客户端发送、服务端接收能力也都正常。这样，通过三次握手，可以全面地验证双方的通信能力。

其次，三次握手还可以防止已失效的连接请求报文突然又传送到了服务器，从而产生错误连接。如果使用两次握手，在第一次握手时客户端发送SYN报文，服务端接收到并发送SYN+ACK报文后，就进入了已连接状态。如果此时客户端的SYN报文在网络中滞留了一段时间才到达服务端，服务端会误认为是新的连接请求，从而建立错误的连接。而三次握手可以避免这种情况的发生，因为在第三次握手时，客户端会发送ACK报文来确认服务端的SYN报文，这样即使网络中有滞留的SYN报文，也不会导致错误的连接建立。

通过三次握手，客户端和服务端可以同步彼此的初始序列号。具体来说，在第一次握手中，客户端发送SYN报文，其中包含自己的初始序列号；在第二次握手中，服务端回复SYN+ACK报文，确认客户端的序列号，并发

送自己的初始序列号；在第三次握手中，客户端发送ACK报文，确认服务端的序列号。这样，通过三次握手，双方就完成了初始序列号的同步

因此，虽然两次握手看起来更简单，但三次握手提供了更高的可靠性和安全性。所以，TCP协议选择了三次握手而不是两次握手来建立连接

为什么不是4次？

三次握手就已经理论上最少可靠连接建立，所以不需要使用更多的通信次数

推荐学习资料

《小林 Coding》 / 图解网络：[为什么是三次握手？不是两次、四次？](#)

85. 讲一下TCP四次挥手？

参考口述回答（简单版）

首先，当客户端决定关闭连接时，它会向服务器发送一个FIN报文，这是第一次挥手，表示客户端没有数据要发送了。

接着，服务器收到FIN报文后，会发送一个ACK报文给客户端，确认关闭请求，这是第二次挥手。此时，连接进入半关闭状态，服务器还可以继续发送数据。

然后，当服务器也准备关闭连接时，它会发送一个FIN报文给客户端，告知自己也没有数据要发送了，这是第三次挥手。

最后，客户端收到服务器的FIN报文后，会发送一个ACK报文进行确认，这是第四次挥手。此后，客户端会等待一段时间后关闭连接，而服务器收到ACK后则立即关闭连接。

参考口述回答（系统版）

首先，我们要明确TCP连接是全双工的，这意味着数据可以在两个方向上同时传输。因此，当一方想要关闭连接时，它需要通知另一方，并确保双方的数据传输都已完成，然后才能完全关闭连接。这就是四次挥手的主要目的。

第一次挥手是由客户端发起的。当客户端没有更多的数据要发送时，它会发送一个FIN报文段给服务端。这个FIN报文段的作用是告诉服务端：“我已经没有数据要发送了，你可以开始关闭你那边到我这边的连接了”。同时，客户端会进入FIN_WAIT_1状态，等待服务端的确认。

第二次挥手发生在服务端收到FIN报文段后。服务端会发送一个ACK报文段给客户端，作为对FIN报文段的确认。这个ACK报文段的作用是告诉客户端：“我已经收到你的关闭请求了，我这边会准备关闭连接，但是在我发送完所有的数据之前，连接还不会完全关闭”。此时，服务端进入CLOSE_WAIT状态，等待所有数据发送完毕。而客户端在收到ACK后，会进入FIN_WAIT_2状态，等待服务端的FIN报文段。

第三次挥手发生在服务端发送完所有数据后。此时，服务端会发送一个FIN报文段给客户端，告知客户端它也没有数据要发送了，可以关闭连接了。这个FIN报文段的作用是告诉客户端：“我这边也没有数据要发送了，我们可以关闭连接了”。服务端在发送FIN报文段后，会进入LAST_ACK状态，等待客户端的确认。

第四次挥手是客户端在收到服务端的FIN报文段后发起的。客户端会发送一个ACK报文段给服务端，作为对FIN报文段的确认。这个ACK报文段的作用是告诉服务端：“我已经收到你的关闭请求了，我会关闭连接”。在发送完ACK报文段后，客户端会进入TIME_WAIT状态。这个状态会持续一段时间（通常是2MSL，即数据包在网络中的最大生存时间），以确保关闭请求和确认报文段能够被对方正确接收。如果在这段时间内没有收到对方的重新发送请求，那么客户端会最终关闭连接，进入CLOSED状态。而服务端在收到ACK报文段后，会立即关闭连接，进入CLOSED状态。

这就是TCP四次挥手的全过程。通过这个过程，TCP连接能够在确保双方数据传输完成的前提下安全地关闭。这个过程的每一步都是必要的，以确保数据的完整性和可靠性。

推荐学习资料

《小林 Coding》 / 图解网络：TCP 四次挥手过程是怎样的？

86. 四次挥手状态流转？四次挥手为什么是4次？

参考口述回答

四次挥手状态流转？

首先，客户端会发送一个带有FIN标志的报文段给服务端，这表示客户端想要关闭连接。在发送FIN报文段之后，客户端的状态会从ESTABLISHED转变为FIN_WAIT_1，等待服务端的确认。

接着，服务端在收到FIN报文段后会回应一个ACK报文段，并进入CLOSE_WAIT状态。这意味着服务端已经收到了关闭连接的请求，但它可能还需要发送一些剩余的数据。此时，客户端在收到ACK后会进入FIN_WAIT_2状态，继续等待服务端的FIN报文段。

然后，当服务端发送完所有剩余数据后，它会发送一个FIN报文段给客户端，并进入LAST_ACK状态。这是第三次挥手，表示服务端也准备关闭连接了。

客户端在收到服务端的FIN报文段后，会发送一个ACK报文段进行确认，并进入TIME_WAIT状态。在这个状态下，客户端会等待一段时间，以确保服务端接收到了关闭连接的请求和确认。这段时间通常是数据包在网络中的最大生存时间的两倍，即2MSL。等待这段时间过后，客户端会关闭连接，状态变为CLOSED。

最后，服务端在收到客户端的ACK报文段后，会立即关闭连接，状态也变为CLOSED。

四次挥手为什么是4次？

当客户端想要关闭连接时，它会发送一个FIN报文段给服务端，这表示客户端不再发送数据了，但还可以接收数据。服务器收到客户端的FIN报文时，内核会马上回一个ACK应答报文，但是服务端应用程序可能还有数据要发送，所以并不能马上发送FIN报文，而是将发送FIN报文的控制权交给服务端应用程序，所以服务端的ACK和FIN一般都会分开发送。这就是挥手是4次不是3次的原因。

推荐学习资料

TCP 四次挥手过程是怎样的？

为什么挥手需要四次？

87. TIME_WAIT系列问题：TIME_WAIT了解吗？服务器什么情况下会产生大量的TIME_WAIT呢？如果TIME_WAIT太多会有什么问题吗？你怎么解决大量TIME_WAIT这个问题？为什么 TIME_WAIT 等待的时间是2MSL？

TIME_WAIT了解吗？

当TCP连接中主动关闭连接的一方发送最后一个ACK确认包后，它会进入TIME_WAIT状态。这个状态的主要作用是等待一段时间，以确保最后的ACK确认包能够被对方正确接收。

TIME_WAIT状态持续的时间是2MSL，也就是两个最大报文段生存时间。这个时间的选择是为了确保在网络中可能存在的延迟或重传的报文段有足够的时间消失，从而避免对后续新建立的连接造成干扰。

在TIME_WAIT状态下，系统会等待足够的时间以确保对方接收到最后的ACK确认包，并且确保网络中不会再有该连接的残留报文。一旦过了这个时间，连接就会完全关闭，资源也会被释放。

服务器什么情况下会产生大量的TIME_WAIT呢？

首先要知道 TIME_WAIT 状态是主动关闭连接才会出现的状态，所以如果服务器出现大量的 TIME_WAIT 状态的 TCP 连接，就是说明服务器主动断开了很多 TCP 连接。而服务器主动断开链接的场景有：1. HTTP 没有使用长连接；2.HTTP 长连接超时；3. HTTP 长连接的请求数量达到上限等

如果TIME_WAIT太多会有什么问题吗？

首先端口资源耗尽是一个主要的问题。由于每个处于TIME_WAIT状态的连接都会占用一个端口，当这种状态的连接数量过多时，可能会导致服务端可用端口不足。一旦所有可用端口都被占满，新的连接就无法建立，从而导致服务拒绝新的客户端请求。

其次，大量的TIME_WAIT状态也会消耗服务端的系统资源，如内存和CPU。虽然每个TIME_WAIT连接消耗的资源相对较少，但当数量庞大时，这些资源的累积消耗就会变得显著，可能会影响到服务端的整体性能。

最后，大量的TIME_WAIT状态还可能导致一些管理上的困扰。例如，监控和排查问题时需要更多的精力来处理这些连接。同时，如果服务端频繁出现大量TIME_WAIT状态，可能需要对服务端的网络配置或应用程序逻辑进行优化，以减少这种情况的发生。

你怎么解决大量TIME_WAIT这个问题？

首先，考虑到TIME_WAIT状态主要是为了确保关闭连接的请求和最后的ACK能够被对方正确接收，我会尝试优化TCP参数配置。具体来说，可以调整TCP的SO_REUSEADDR选项，允许端口复用，这样即使端口处于TIME_WAIT状态，也可以被新的连接所使用，从而避免了端口资源耗尽的问题。

其次，我会从应用程序的设计层面进行优化。例如，对于频繁进行短连接操作的场景，我会考虑使用长连接或者连接池技术来减少连接的频繁建立和关闭。这样不仅可以减少TIME_WAIT状态的数量，还能提高系统的整体性能。

另外，我也会关注服务端和客户端之间的网络通信质量。如果网络通信不稳定或者存在延迟，可能会导致连接在关闭后仍然长时间处于TIME_WAIT状态。因此，我会尝试优化网络环境，减少网络延迟和丢包率，从而降低TIME_WAIT状态的数量。

最后，如果服务端确实需要频繁地关闭和打开连接，并且无法避免大量的TIME_WAIT状态，我会考虑增加服务端的端口范围，以确保即使有大量TIME_WAIT状态存在，也不会耗尽所有可用端口。

为什么 TIME_WAIT 等待的时间是 2MSL？

首先，设置2MSL的等待时间可以确保最后的ACK报文能够到达对方并被正确接收。网络传输中可能存在延迟或丢包的情况，所以给予足够的时间窗口，让对方能够接收到确认消息，从而保证连接的可靠关闭。

其次，这个时间的设置也有助于避免新旧连接之间的混淆。在网络中，可能存在多个相同源IP地址和目的IP地址的连接。等待2MSL的时间可以确保之前连接的所有报文都已经在网络中消失，这样新的连接就不会错误地接收到之前连接的残留数据。

总的来说，TIME_WAIT等待的时间是2MSL，是为了确保网络连接的可靠关闭，并防止新旧连接之间的混淆。

推荐学习资料

《小林 Coding》 / 图解网络：

[为什么需要 TIME_WAIT 状态？](#)

[服务器出现大量 TIME_WAIT 状态的原因有哪些？](#)

[TIME_WAIT 过多有什么危害？](#)

[如何优化 TIME_WAIT？](#)

[为什么 TIME_WAIT 等待的时间是 2MSL？](#)

88. TCP可靠性相关问题：TCP可靠性怎么实现的？滑动窗口机制是怎样的？TCP怎么进行流量控制的？TCP的拥塞控制怎么做的？拥塞控制算法有哪些？

解析

TCP通过重传机制、滑动窗口、流量控制和拥塞控制等机制共同保证了数据的可靠传输。这些机制使得TCP能够在复杂的网络环境中实现稳定、可靠的数据传输服务

参考口述回答

TCP，即传输控制协议，是一种面向连接的、可靠的、基于字节流的传输层通信协议。TCP的可靠性主要是通过一系列机制来实现的，这些机制共同确保了数据包能够准确、有序地从发送端传输到接收端。具体来说，TCP的可靠性主要通过以下几个方面来保证和实现：

1. **重传机制**：TCP通过序列号和确认应答机制来实现数据的可靠传输。当发送端发送数据后，会等待接收端的确认应答。如果在一定时间内没有收到确认应答，发送端会重传数据，直到收到确认应答为止。这种重传机制确保了数据的可靠传输，避免了数据丢失的问题。
2. **滑动窗口机制**：TCP使用滑动窗口机制来实现流量控制。发送端维护一个发送窗口，接收端也维护一个接收窗口。发送端根据接收端的接收能力来动态调整发送窗口的大小，从而控制数据的发送速率，避免发送端发送数据过快而导致接收端来不及处理。这种机制确保了数据的稳定、有序传输。
3. **流量控制**：TCP的流量控制是基于滑动窗口实现的。接收端会在确认应答中告知发送端自己当前的接收窗口大小，发送端根据这个信息来调整自己的发送速率。这样，当接收端的处理能力下降时，发送端会相应地降低发送速率，从而避免数据丢失或乱序。
4. **拥塞控制**：TCP还通过拥塞控制机制来避免网络拥塞。具体来说，TCP使用了慢开始、拥塞避免、快重传和快恢复等算法来调整发送速率。这些算法可以根据网络的实时状况动态地调整发送端的发送速率，从而避免网络拥塞和数据丢失。

综上所述，TCP通过重传机制、滑动窗口、流量控制和拥塞控制等机制共同保证了数据的可靠传输。这些机制使得TCP能够在复杂的网络环境中实现稳定、可靠的数据传输服务。

滑动窗口机制是怎样的？

滑动窗口机制是TCP协议中用于流量控制和数据传输效率提升的一种重要机制。在TCP连接建立后，发送方和接收方都会维护一个窗口，这个窗口可以看作是一个缓冲区，用于暂存待发送或接收的数据。

发送方的滑动窗口表示在未收到确认之前，可以连续发送的数据量。发送方会不断地将数据发送到窗口中，而不需要等待每一个数据段的确认。当数据被成功发送并确认后，这个窗口会“滑动”向前，允许发送方继续发送新的数据。

接收方的窗口大小则表示它还能接收多少数据，这个信息会通过确认报文段（ACK）中的窗口字段反馈给发送方。这样，发送方就能根据接收方的接收能力来动态调整自己的发送速率，既不会发送得太快导致接收方来不及处理，也不会发送得太慢浪费网络资源。

简单来说，滑动窗口机制就像是一个可移动的“窗口”，在这个“窗口”范围内的数据可以被连续发送，而无需等待每个数据包的确认。这种机制大大提高了TCP协议的数据传输效率，同时也保证了数据的可靠传输。

TCP怎么进行流量控制的？

TCP进行流量控制主要通过**滑动窗口**机制来实现。具体来说，流量控制是为了防止发送方发送数据过快，导致接收方来不及处理而丢失数据。为了实现这一目标，TCP使用了基于滑动窗口的流量控制方法。

在TCP连接建立后，接收方会根据自己的接收缓冲区大小，通过确认报文段（ACK）中的窗口字段来告知发送方自己当前能够接收的最大数据量，即**接收窗口**的大小。发送方在收到这个信息后，就会根据接收窗口的大小

来调整自己的发送速率，确保发送的数据量不会超过接收方的处理能力。

随着数据传输的进行，接收方的接收窗口大小可能会动态变化。比如，当接收方的应用程序从接收缓冲区中读取数据时，接收窗口的大小就会相应增加；而当接收缓冲区中的数据积累过多时，接收窗口的大小就会减小。接收方会实时更新窗口大小，并通过ACK报文通知发送方。

发送方在收到新的窗口大小信息后，会相应地调整自己的**发送窗口**，以确保发送的数据量始终在接收方的处理能力范围之内。这样，TCP就能够通过滑动窗口机制实现有效的流量控制，防止数据丢失和网络拥塞的发生。

TCP的拥塞控制怎么做的？拥塞控制算法有哪些？

TCP的拥塞控制是为了防止因过多的数据注入网络而产生的网络拥塞，从而避免网络性能下降和数据丢失。拥塞控制主要是通过一系列算法来动态调整数据的发送速率，以达到网络资源的合理利用。

TCP的拥塞控制算法主要包括**慢开始**、**拥塞避免**、**快重传**和**快恢复**。

首先是**慢开始**算法。在TCP连接刚建立时，发送方会初始化一个较小的拥塞窗口，并随着每个确认报文的到来逐渐增大这个窗口，通常是每次翻倍。这个阶段的目的是逐渐探测网络的拥塞程度，避免一开始就发送大量数据导致网络拥塞。

当拥塞窗口增大到一定程度后，TCP会进入**拥塞避免**阶段。在这个阶段，发送方每收到一个确认报文，拥塞窗口只增加一点点，而不是继续翻倍，以避免发送速率增长过快导致网络拥塞。

如果发送方连续收到三个相同的确认报文（即三个重复的ACK），它会认为有数据包丢失，此时会触发**快重传**算法。发送方会立即重传被认为丢失的数据包，而不必等待超时。

在快重传之后，TCP会进入**快恢复**阶段。在这个阶段，发送方会将拥塞窗口减半，以降低发送速率，并尝试恢复正常的数据传输。然后，发送方会采用与拥塞避免算法相同的策略来增加拥塞窗口。

推荐学习资料

《小林 Coding》 / 图解网络：[可靠性保证：TCP 重传、滑动窗口、流量控制、拥塞控制](#)

《小林 Coding》 / 图解网络：[TCP：滑动窗口](#)

《小林 Coding》 / 图解网络：[TCP：流量控制](#)

《小林 Coding》 / 图解网络：[TCP：拥塞控制](#)

计算机网络：HTTP

89. http和https的区别？

解析：

HTTP和HTTPS的主要区别在于安全性，端口使用，处理速度和申请证书方面。

参考口述回答：

首先，安全性是这两者之间最显著的区别。HTTP是超文本传输协议，它是以明文方式传输数据的，这意味着在数据传输过程中，如果遭到拦截，信息将以明文形式暴露，存在安全风险。而HTTPS则是HTTP的安全版，它在HTTP的基础上加入了SSL/TLS协议，可以对数据进行加密传输和身份认证，从而保证了数据传输的安全性。

其次，从端口号上来看，HTTP和HTTPS使用的端口也不同。HTTP默认使用80端口，而HTTPS则使用443端口。

再者，HTTPS将加密，解密，验证，解压缩等流程都交给了SSL/TLS来处理，因此相对于HTTP来说，HTTPS在处理速度上会有所降低，但由于目前硬件性能的大幅提升，这种影响在实际使用中几乎可以忽略不计。

最后，HTTPS需要向认证机构（CA）申请证书，一般需要一定的费用，而HTTP则不需要。

在当今这个信息安全至关重要的时代，HTTPS的重要性不言而喻，它能为我们的数据传输提供强有力的安全保障。虽然HTTPS在处理速度和费用方面相较HTTP有所不足，但这些都看作是保障信息安全而做出的必要牺牲。

推荐学习资料：

《小林Coding》 | 图解网络：[HTTP 与 HTTPS 有哪些区别？](#)

90. 说说http协议的报文结构

解析：

HTTP协议的报文结构是客户端与服务器之间通信的基础，它确保了双方能够准确、高效地交换信息。在处理HTTP请求和响应时，我们需要关注报文的各个部分，以确保正确地解析和生成HTTP消息

参考口述回答：

首先，HTTP请求报文主要由请求行、请求头部和请求体三个部分构成。请求行包含了请求的方法，比如GET或POST，请求的URL路径，以及HTTP协议的版本。紧接着请求行的是请求头部，它由多个键值对组成，每个键值对代表一种信息或设置，比如Host字段标明请求的目标主机，User-Agent字段描述发出请求的用户代理信息，Accept字段则列出客户端能够理解的内容类型。请求头部之后，可能跟随一个请求体，它通常用于POST或PUT请求中，包含客户端提交给服务器的数据。

其次，HTTP响应报文的结构与请求报文相似，由状态行、响应头部和响应体组成。状态行包含HTTP协议版本、状态码以及状态描述信息。比如，状态码200表示请求成功，404则表示资源未找到。响应头部也包含多个键值对，提供有关响应的附加信息，如Content-Type描述响应体的内容类型，Content-Length指明响应体的长度。响应体则包含服务器返回给客户端的实际数据内容，如HTML文件、图片或其他媒体资源。

推荐学习资料：

《博客：JavaScript Guidebook》：[HTTP 报文格式](#)

91. 常见header字段有哪些，content-type知道吗？

解析：

参考口述回答：

在HTTP请求和响应中，常见的header字段包括：

1. **Host**：指定请求的目标主机和端口号。
2. **User-Agent**：标识发出请求的用户代理（如浏览器类型和版本）。
3. **Accept**：告知服务器客户端能够处理的内容类型，如text/html、application/json等。
4. **Accept-Encoding**：告诉服务器客户端支持的内容编码方式，如gzip、deflate等，这有助于减少传输数据的大小。
5. **Accept-Language**：表明客户端希望接收的语言类型，如en-US、zh-CN等。
6. **Authorization**：在需要用户验证的情况下，此字段包含认证信息，如Bearer Token或其他认证方式。
7. **Content-Length**：在请求或响应中，标明正文内容的长度。
8. **Content-Type**：这个字段非常重要，它定义了请求或响应正文的媒体类型。例如，在POST请求中，如果发送的是JSON数据，那么Content-Type应该设置为application/json。同样，如果响应正文是HTML，则Content-Type可能设置为text/html。

推荐学习资料：

《小林Coding》 | 图解网络：[HTTP 常见字段有哪些？](#)

92. get和post方法有什么区别？get不可以用来提交数据吗？

解析：

参考口述回答：

get和post方法有什么区别？

首先，根据 RFC 规范，GET方法主要用于从服务器获取资源，而POST方法则用于向服务器提交数据，通常用于表单的提交或文件的上传等操作。

其次，在数据传输方面，GET方法将请求参数附加在URL之后，作为查询字符串传递，这些参数在浏览器地址栏中可见，因此GET请求不适合传输敏感信息。而POST方法则将请求参数放在请求体中传输，对用户来说地址栏是不可见的。（当然因为http来说都是明文传输，抓包之后都可见）

此外，GET请求的参数有长度限制，因为浏览器和Web服务器对URL长度有限制，而POST请求由于参数放在请求体中，没有这样的限制。

GET方法是否可以用来提交数据？

至于GET方法是否可以用来提交数据，从技术上说，是可以通过查询字符串提交一些简单的数据的。但是，由于GET请求的参数可见性和长度限制，它不适合用于提交大量或敏感的数据。在实际应用中，我们通常使用POST方法来提交数据，以确保数据的安全性和完整性。

推荐学习资料：

《小林Coding》 | 图解网络：[GET 与 POST](#)

93. http常用的状态码？

解析：

参考口述回答：

首先，200状态码是非常常见的成功状态码，表示请求已经成功处理，服务器已成功处理了请求并提供了相应的资源。

其次，301和302状态码是重定向状态码。301代表永久性重定向，意味着资源已经被永久移动到新的URL，而302则表示临时性重定向，即资源临时移动到新的位置。

另外，404状态码是一个客户端错误状态码，表示服务器无法找到请求的资源。这通常发生在用户尝试访问不存在的页面时。

最后，500状态码是一个服务器错误状态码，表示服务器在处理请求时遇到了错误。这可能是由于服务器内部的问题导致的。

推荐学习资料：

《小林Coding》 | 图解网络：[HTTP 常见的状态码有哪些？](#)

94. cookie和session的区别

解析：

Cookie和Session都是用来跟踪浏览器用户身份的会话方式。当客户端第一次发送请求到服务器时，服务器会创建一个Session，并同时创建一个名为JSESSIONID的特殊Cookie发送到客户端。此后，客户端每次发送请求都会携带这个Cookie，服务器则通过识别这个Cookie中的SessionID来找到对应的Session，从而获取用户的会话信息。

关于区别，可以从如下几个方面阐述：**存储位置，存储容量，数据类型，安全性，生命周期，服务器压力**

参考口述回答：

1. **存储位置**：这是二者最明显的区别。Cookie的数据信息存放在客户端的浏览器上，而Session的数据信息则是存放在服务器端的。
2. **存储容量**：单个Cookie保存的数据量是有限的，通常不超过4KB，而且一个站点最多可以保存20个Cookie。相比之下，Session的存储容量没有明确的上限，但出于服务器性能考虑，我们不会在其中存放过多的数据，并会设置适当的Session删除机制。

3. **数据类型**：Cookie中只能存储ASCII字符串，如果需要存储其他类型的数据，则需要进行编码。而Session则能够存储任何类型的数据，包括字符串、整数、列表、字典等。
4. **安全性**：由于Cookie的数据存储在客户端，因此它对于客户端是可见的，这带来了一定的安全风险，如Cookie欺骗等。而Session数据存储在服务器端，对客户端是透明的，因此不存在敏感信息泄漏的风险。
5. **生命周期**：Cookie的生命周期可以通过设置其属性来实现长期有效。而Session的生命周期则依赖于名为JSESSIONID的Cookie，该Cookie的过期时间默认为-1，意味着只要关闭浏览器窗口，该Session就会失效。
6. **服务器压力**：由于Cookie数据保存在客户端，因此不会给服务器带来额外的负担。然而，对于Session来说，每个用户都会产生一个Session，如果并发访问的用户非常多，那么服务器上将会产生大量的Session，这可能会消耗大量的内存资源。

推荐学习资料：

《掘金博客》：[cookie、session、token的那些事儿](#)

95. HTTPS连接建立的过程？（HTTPS四次握手,TLS握手）

解析：

首先，我们需要明确HTTPS是基于HTTP上建立的SSL/TLS加密层，并对传输内容进行加密。HTTPS将对称加密与非对称加密结合起来，充分利用两者之长。在交换密钥环节，HTTPS使用非对称加密方式，之后的报文交换则使用对称加密方式。

参考口述回答：

HTTPS连接建立的四次握手过程如下：

第一次握手：Client Hello

这个过程是客户端向服务端发起连接请求，客户端会发送一个Client Hello消息给服务端。这个消息里面包含了客户端支持的TLS版本、加密套件（Cipher Suites）列表，以及一个随机数（Client Random）。这个随机数将在后续生成会话密钥时使用。同时，客户端还会发送一个支持的压缩算法列表和其他一些扩展字段，比如SNI（Server Name Indication），用于指示客户端想要连接的服务名。

第二次握手：Server Hello

服务端在收到客户端的Client Hello消息后，会回应一个Server Hello消息。这个消息中会包含服务端选择的TLS版本，从客户端提供的加密套件列表中选择的一个合适的加密套件，以及一个随机数（Server Random）。同时，服务端还会发送自己的证书链到客户端，用于身份验证。如果服务端需要客户端证书进行身份验证，也会在这一步中请求。

第三次握手：Client Key Exchange + Change Cipher Spec + Encrypted Handshake Message

在这一步中，客户端首先会生成一个预主密钥（Pre-Master Secret），并使用服务端证书中的公钥进行加密，然后发送给服务端。这个预主密钥在后续将被用来生成会话密钥。接着，客户端会发送一个Change Cipher Spec消息，通知服务端后续通信将使用新的加密套件和密钥。最后，客户端会发送一个Encrypted Handshake Message，这是一个使用新密钥加密的握手消息认证码（HMAC），用于验证密钥交换和之前握手消息的完整性。

第四次握手：Change Cipher Spec + Encrypted Handshake Message

服务端在收到客户端的第三次握手消息后，会使用自己的私钥解密预主密钥，并使用这个预主密钥生成会话密钥。然后，服务端也会发送一个Change Cipher Spec消息给客户端，通知客户端后续通信将使用新的加密套件和密钥。最后，服务端同样会发送一个Encrypted Handshake Message消息给客户端，这也是一个使用新密钥加密的握手消息认证码，用于验证密钥交换和之前握手消息的完整性。

至此，HTTPS的四次握手过程就完成了。在这个过程中，双方通过非对称加密方式安全地交换了密钥，并生成了会话密钥。后续的数据传输将使用这个会话密钥进行对称加密通信，保证了通信的安全性和机密性。同时，通过使用握手消息认证码（HMAC）还可以验证握手消息的完整性防止被篡改或伪造。这个过程充分体现了HTTPS协议在安全通信方面的优势。

推荐学习资料：

《小林Coding》 | 图解网络：[HTTPS 是如何建立连接的？其间交互了什么？](#)

96. HTTPS采用对称加密和非对称加密？为什么后续换成对称密钥？

解析：

混合加密：对称加密（性能好）+ 非对称加密（安全）

参考口述回答：

HTTPS确实采用了对称加密和非对称加密结合的方式。在HTTPS连接建立的过程中，非对称加密主要用于密钥交换和安全认证，而对称加密则用于实际的数据传输。

HTTPS之所以在密钥交换阶段使用非对称加密，是因为非对称加密具有很高的安全性。非对称加密算法使用一对密钥，即公钥和私钥。公钥可以公开，任何人都可以用公钥对数据进行加密，但只有持有相应私钥的人才能解密。这种特性使得非对称加密在密钥交换过程中非常有用，因为它可以确保只有服务器能解密客户端发送的数据，从而保证了通信的安全性。

然而，非对称加密的计算复杂度较高，加密和解密速度相对较慢。因此，在大量的数据传输中，如果一直使用非对称加密，会导致性能下降，影响用户体验。

为了提高通信效率，HTTPS在密钥交换完成后，会生成一个对称密钥。这个对称密钥将用于后续的数据传输加密。对称加密的特点是加密和解密使用相同的密钥，其加密速度快，适合用于大量数据的加密传输。

所以，HTTPS在密钥交换阶段使用非对称加密来确保安全性，而在后续的数据传输阶段则切换到对称加密以提高效率。这种结合使用的方式既保证了通信的安全性，又兼顾了性能需求。

推荐学习资料：《小林Coding》 | 图解网络：[HTTPS 混合加密](#)

97. HTTP/1.1相对HTTP/1.0做了哪些优化改进

解析：

改进：连接管理、请求管道化、错误处理、缓存机制等

参考口述回答：

首先，HTTP/1.1引入了持久连接的概念，也就是所谓的**长连接**。在HTTP/1.0中，每次请求都需要建立一个新的TCP连接，请求处理完毕后立即断开连接。这种方式在处理包含大量图片、链接等资源时，会导致大量的连接建立和断开操作，造成资源浪费和时间延迟。而HTTP/1.1则允许在同一个TCP连接上连续发送多个请求，从而减少了连接建立和断开的开销，提高了网络资源的利用率。

其次，HTTP/1.1支持**请求管道化**，这意味着客户端可以同时发送多个请求，而不需要等待前一个请求的响应。这样，服务器就可以按照请求到达的顺序依次进行回应，从而减少了等待时间，提高了整体的响应速度。需要注意的是，虽然请求管道化可以并行发送请求，但服务器仍然需要按照顺序回应。

另外，HTTP/1.1还增加了更多的**错误状态码**，提供了更详细的错误信息，这有助于开发人员更好地定位问题并进行调试。同时，HTTP/1.1还引入了更多的请求头和响应头，为开发人员提供了更多的控制和灵活性。

最后，值得一提的是，HTTP/1.1还改进了**缓存机制**，允许服务器发送“Last-Modified”和“ETag”等响应头，以便客户端进行条件性请求，这避免了不必要的资源传输，提高了网络的利用率。

总的来说，HTTP/1.1相对于HTTP/1.0在连接管理、请求管道化、错误处理、缓存机制等方面都做了显著的优化和改进，从而提高了网络应用的性能和效率。

推荐学习资料：

《小林Coding》 | 图解网络：[HTTP/1.1 相比 HTTP/1.0 提高了什么性能？](#)

98. HTTP/2.0相对HTTP/1.1做了哪些优化改进

解析：

改进点：二进制分帧、多路复用、头部压缩和服务器推送等

参考口述回答：

HTTP/2.0在保持与HTTP/1.1的语义兼容性的同时，引入了多项重大改进，显著提升了网络性能。

首先，HTTP/2.0采用了二进制分帧层，这使得客户端和服务端可以将HTTP消息分解为多个互不依赖的帧，并可以乱序发送。这些帧在另一端被重新组合起来，从而实现了低延迟和高吞吐量。

其次，HTTP/2.0引入了多路复用技术，允许同时通过单一的HTTP/2连接发起多重请求-响应消息。这解决了HTTP/1.1中的队头阻塞问题，使得多个数据流可以并行传输，从而更有效地使用TCP连接，提高了网络资源的利用率。

此外，HTTP/2.0还支持头部压缩，通过HPACK算法对请求头进行压缩传输，节省了消息头占用的网络流量，降低了带宽消耗。

最后，HTTP/2.0还引入了服务器推送技术，即服务器可以对一个请求发送多个响应。这种机制使得服务器能够更快地推送资源到客户端，从而提高了页面加载速度。

综上所述，HTTP/2.0通过二进制分帧、多路复用、头部压缩和服务器推送等技术，显著提升了网络性能，降低了网络延迟，为用户提供了更好的网络体验。

推荐学习资料：

《小林Coding》 | 图解网络：[HTTP/2 做了什么优化？](#)

99. HTTP/3.0相对HTTP/2.0做了哪些优化改进

解析：

HTTP/3.0通过采用QUIC协议、支持真正的多路复用、提供更好的流量控制和安全性能等方面的优化和改进

参考口述回答：

首先，HTTP/3.0使用了基于UDP协议的QUIC作为传输层协议，而不是传统的TCP协议。这一改变显著减少了连接建立的延迟，因为QUIC协议在建立连接时无需进行多次握手，从而加速了数据传输的速度。

其次，HTTP/3.0支持真正的多路复用，这意味着在同一连接上可以并行处理多个请求和响应，而不会像HTTP/2那样存在队头阻塞的问题。这大大提高了网络资源的利用率和传输效率。

此外，HTTP/3.0还具备更好的流量控制能力，可以有效防止单一流的资源占用过多带宽，从而保证网络传输的稳定性和公平性。

在安全性方面，HTTP/3.0默认使用TLS 1.3加密，为数据传输提供了更高的安全性。同时，QUIC协议还提供了更好的错误恢复机制，当数据包丢失或损坏时，能够更快速地恢复连接并进行数据传输。

最后，值得一提的是，HTTP/3.0还支持0-RTT连接，即客户端可以在不需要等待服务器响应的情况下直接发送数据给服务器。这种连接方式可以大幅度缩短连接建立时间，但在某些场景下可能存在一定的安全风险，因此在使用时需要谨慎考虑。

总的来说，HTTP/3.0通过采用QUIC协议、支持真正的多路复用、提供更好的流量控制和安全性能等方面的优化和改进，进一步提升了网络传输的效率和性能。这些改进使得HTTP/3.0成为现代网络中更具优势和应用前景的协议。

推荐学习资料：

《小林Coding》 | 图解网络：[HTTP/3 做了什么优化？](#)

计算机网络：其他协议

100. DNS是如何进行解析的？

解析：

DNS，全称是Domain Name System，即域名系统，它的主要作用是将便于人类记忆的域名转换为机器能识别的IP地址。

参考口述回答：

DNS的解析过程是一个分布式、层次化的查询过程。当用户在浏览器中输入一个域名时，首先会由本地计算机的DNS客户端发起一个DNS查询请求。这个请求首先会到达本地DNS服务器，也称为LDNS服务器。

本地DNS服务器在收到请求后，会首先查看自身的缓存中是否已经有该域名的记录。如果有，则直接返回对应的IP地址；如果没有，则会开始一个递归或迭代的查询过程。

在这个过程中，本地DNS服务器会向根域名服务器发出查询请求。根域名服务器会返回顶级域名服务器的IP地址，如.com、.net等域的服务器地址。然后，本地DNS服务器会再向这些顶级域名服务器发出查询请求，顶级域名服务器会返回负责管理该域名的权威DNS服务器的IP地址。

最后，本地DNS服务器会向权威DNS服务器发出查询请求，权威DNS服务器会返回该域名对应的IP地址。本地DNS服务器在得到IP地址后，会将其缓存起来，并返回给DNS客户端。这样，用户就可以通过这个IP地址访问对应的网站了。

总的来说，DNS的解析过程是一个分布式的、层次化的查询过程，它通过多个DNS服务器之间的协作，将用户输入的域名解析为对应的IP地址。这个过程确保了用户在访问网站时能够准确地找到目标服务器，从而实现了网络的互联互通。

推荐学习资料：

《小林Coding》 | 图解网络：[真实地址查询 —— DNS](#)

更多优质题目尽在牛面Offer小程序：

牛面 一站式学习与面试

2000+热门题库免费刷！题目全栈覆盖，助你拿offer！



海量题库刷不停，
微信扫码立即开始！



趣味闯关

告别死记硬背，闯关练习陪你
轻松进步，每一关都在悄悄变强。

1000+精品关卡等你来战



技术题库

春招、秋招最热题型全覆盖
多名大厂老鸟精心把关筛选

2k+道命中率超高必刷题



简历押题

大厂标准提取核心问题
问题-思路-答案三位一体

个性化精准预测



模拟面试

专项突破，垂直领域靶向强化
简历深挖，六大维度全面拷问

真实场景完整演练

牛面题库 让面试准备更高效 → niumianoffer.com



学习过程中有任何问题，都欢迎联系牛面小助手，不定时会发放福利哦：



扫码添加 牛面小助手